UNCUYO
UNIVERSIDAD NACIONAL DE CUYO

FACULTAD DE INGENIERÍA

Semáforos Inteligentes

Un Enfoque con Aprendizaje por Refuerzo Multiagente

Tesina

Licenciatura en Ciencias de la Computación

Autor

Juan Martín Morales

Directora

Dra. Ana Carolina Olivera

Ciudad de Mendoza, Argentina

2026

Resumen

Este trabajo aborda la congestión vehicular urbana mediante el control semafórico con aprendizaje por refuerzo multiagente. Cada intersección se representa por un agente que selecciona fases a partir de una observación local del tránsito. Se consideran MAPPO y HAPPO con entrenamiento centralizado y ejecución descentralizada, e IPPO e IDQN con aprendizaje independiente; además, se describe una adaptación de HAPPO con actualización secuencial de los actores. La espera acumulada por carril se codifica mediante una transformación logarítmico-lineal acotada y discretizada, con mayor resolución en valores bajos. El diseño de búsqueda de hiperparámetros comprende los cuatro algoritmos en tres redes sintéticas y en una red del microcentro de Mendoza; la comparación local y vecinal de MAPPO se circunscribe a las redes sintéticas. La red urbana se construye con información cartográfica y conteos parciales, y su comprobación con los programas de SUMO caracteriza su operación, no el desempeño de políticas aprendidas.

Abstract

This work addresses urban traffic congestion through multi-agent reinforcement learning for traffic-signal control. Each intersection is represented by an agent that selects phases from a local observation of traffic. The algorithms considered are MAPPO and HAPPO with centralized training and decentralized execution, and IPPO and IDQN with independent learning; the method also describes an adaptation of HAPPO with sequential actor updates. Accumulated lane waiting is represented by a bounded, discretized log-linear transformation with finer resolution at lower values. The hyperparameter-search design covers the four algorithms in three synthetic networks and an urban network in downtown Mendoza; the MAPPO comparison of local and neighborhood variants is limited to the synthetic networks. The urban network is built from cartographic information and partial traffic counts, and its operational check with SUMO signal programs characterizes network operation rather than learned-policy performance.

Agradecimientos

Página de agradecimientos

Índice general

1. Introducción **1**
1.1. Motivación . . . 3
1.2. Organización del documento . . . 4

2. Marco teórico **5**
2.1. Ingeniería de tránsito y simulación . . . 5
2.1.1. Terminología de tránsito . . . 6
2.1.2. Fundamentos de la dinámica vehicular . . . 8
2.1.3. Sistemas de control de tránsito . . . 14
2.1.4. Simulación de tránsito . . . 16
2.2. Aprendizaje por refuerzo (RL) . . . 22
2.2.1. Procesos de decisión de Markov (MDP) . . . 23
2.2.2. Métodos basados en valor y basados en política . . . 25
2.2.3. Aprendizaje profundo y redes neuronales . . . 29
2.2.4. Aprendizaje por refuerzo profundo (DRL) . . . 32
2.3. Aprendizaje por refuerzo multiagente (MARL) . . . 39
2.3.1. Fundamentos teóricos: juegos estocásticos, equilibrio de Nash y Dec-POMDP . . . 40
2.3.2. Desafíos del aprendizaje multiagente . . . 41
2.3.3. Paradigmas de entrenamiento y ejecución . . . 43
2.3.4. MADRL y líneas de base . . . 44
2.3.5. MAPPO . . . 45
2.3.6. HAPPO . . . 47
2.3.7. Antecedentes recientes del control semafórico multiagente . . . 53

3. Metodología **59**
3.1. Diseño del estudio . . . 59
3.2. Escenarios de simulación . . . 60
3.2.1. Preparación de las redes . . . 61
3.2.2. Redes sintéticas . . . 61
3.2.3. Red de la Ciudad de Mendoza . . . 64

3.3. Sistema de control multiagente . . . 69
3.3.1. Entorno de simulación . . . 69
3.3.2. Observación, acciones y recompensas . . . 70
3.3.3. Agentes y algoritmos de aprendizaje . . . 74
3.4. Entrenamiento y búsqueda de hiperparámetros . . . 75
3.4.1. Condiciones de simulación y semillas . . . 75
3.4.2. Fase 1: búsqueda amplia . . . 76
3.4.3. Fase 2: dominios reducidos . . . 76
3.4.4. Objetivo de búsqueda y evaluación diagnóstica . . . 77
3.4.5. Selección de configuraciones de HPO . . . 78
3.5. Comparación de variantes locales y vecinales de MAPPO . . . 79
3.6. Métricas y análisis de las evaluaciones . . . 81
3.6.1. Medidas de desempeño y vehículos evaluados . . . 81
3.6.2. Comparación por pares y criterio de decisión . . . 83
3.7. Reproducibilidad y alcance . . . 84
3.7.1. Entorno de cómputo y envío de trabajos . . . 84
3.7.2. Alcance de las conclusiones . . . 85

4. Análisis y discusión de resultados **86**
4.1. Resultados del ajuste de hiperparámetros . . . 86
4.1.1. Estado de la consolidación y de las búsquedas . . . 87
4.1.2. Resultados del ajuste y análisis de sensibilidad . . . 87
4.1.3. Selección de configuraciones de HPO . . . 87
4.2. Evaluación de políticas y comprobación de escenarios . . . 87
4.2.1. Criterios de evaluación y métricas de comparación . . . 87
4.2.2. Desempeño en escenarios sintéticos: 3×1, 3×3, 4×4 heterogéneo 87
4.2.3. Comprobación operacional del escenario de la Ciudad de Mendoza . 87
4.3. Discusión e interpretación de resultados . . . 87

5. Conclusiones **88**

A. Arquitectura de software y reproducción experimental **89**
A.1. Diseño orientado a objetos del entorno de simulación . . . 89
A.2. Jerarquía de funciones de recompensa y patrón Strategy . . . 91
A.3. Patrones de diseño de la arquitectura . . . 92
A.4. Parámetros de SUMO y programas semafóricos de referencia . . . 93
A.4.1. Identificadores del escenario urbano de Mendoza . . . 94
A.5. Especificaciones para reproducir los experimentos . . . 95
A.5.1. Semillas de fase 1 en las redes sintéticas . . . 95
A.5.2. Arquitecturas y semillas de la búsqueda informada . . . 96

A.5.3. Dominios completos y parámetros efectivos 96
A.6. Detalles del aprendizaje y de la codificación 99
A.6.1. Configuración fija de la comparación multiescenario 101
A.6.2. Calibraciones y semillas de cooperación 102
A.6.3. Archivos utilizados para las figuras 102

Índice de figuras

1.1. Evolución de la flota vehicular circulante estimada en Argentina entre 1990 y 2025, expresada en millones de vehículos. La línea y los marcadores representan los valores anuales disponibles; el área sombreada facilita la lectura de su magnitud. Elaboración propia a partir de reportes de AFAC. 1

2.1. Movimientos vehiculares, fases y cruces peatonales 7
2.2. Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada. 8
2.3. Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s). 11
2.4. Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de agrupamientos vehiculares. Las curvas representan situaciones conceptuales y no una distribución empírica particular. 13
2.5. Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada. 15
2.6. Correspondencia entre la circulación real y su representación en SUMO . . 17
2.7. Interfaz sumo-gui durante una simulación microscópica 18
2.8. Esquema conceptual de carriles, línea de detención y conexiones de una intersección semaforizada bajo circulación por la derecha. La fase representada habilita movimientos directos opuestos del eje horizontal y retiene los movimientos conflictivos. 20
2.9. Interacción agente-entorno en un proceso de decisión de Markov [SB18]. . . 24
2.10. Esquema de un método basado en valor 26
2.11. Esquema de un método basado en política 27
2.12. Arquitectura actor-crítico aplicada al control semafórico 29
2.13. Operaciones de una unidad neuronal artificial: suma ponderada de las entradas, incorporación del sesgo y aplicación de una función de activación. . 30
2.14. Arquitectura conceptual de una red neuronal *feedforward* con dos capas ocultas. 31

2.15. Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de *bootstrapping* del valor objetivo Diferencia temporal (*Temporal-Difference*) (TD). Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS+15]. 34
2.16. Red de política para un espacio discreto de acciones 36
2.17. Interacción agente-entorno en MARL cooperativo 41
2.18. Evolución de las políticas de dos agentes bajo WoLF-PHC en Piedra, Papel o Tijera 43
2.19. Paradigma de entrenamiento centralizado con ejecución descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales. 44
2.20. Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m}, que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia. 51

3.1. Geometría de las tres redes sintéticas. Los puntos marcan intersecciones semaforizadas; cada panel usa una escala propia. 63
3.2. Red parcial del microcentro de Mendoza. Los extremos grises conectan con la red circundante; el tramo azul marca el ingreso estimado con el conteo de la cámara 169, que registra el flujo hacia el oeste por Av. José Vicente Zapata. 65
3.3. Flujo entre la simulación, el entorno y el aprendizaje. El estado de SUMO y los parámetros de las políticas se almacenan por separado. 69
3.4. La observación conserva la fase activa y codifica la espera de cada carril en niveles discretos. 72
3.5. Información de los algoritmos durante el entrenamiento y la ejecución. El crítico compartido de MAPPO y HAPPO no interviene al ejecutar. 74
3.6. Búsqueda en dos fases: la primera orienta la reducción de dominios de la segunda. 77

A.1. Diagrama de clases del entorno de simulación y de sus relaciones principales. 90
A.2. Especificación UML del módulo de recompensas. 91
A.3. Ejemplo de codificación de observaciones. 100

Índice de tablas

2.1. Variables de tránsito macroscópicas y microscópicas 10
2.2. Archivos de entrada y configuración empleados en una simulación de SUMO. 21
2.3. Comparación sintética de familias algorítmicas PPO en entornos multi-agente cooperativos. 53

3.1. Escenarios y procedimientos del trabajo. 60
3.2. Duraciones de los programas semafóricos de referencia en las redes sintéticas (segundos). 61
3.3. Características de las redes sintéticas y flujo. 63
3.4. Herramientas y responsabilidades en el sistema experimental. 69
3.5. Períodos de calentamiento y control de las campañas. 76
3.6. Variantes examinadas en el cribado exploratorio. 80

4.1. Métricas operacionales del escenario urbano de la Ciudad de Mendoza con los programas semafóricos de la red. 87

A.1. Duraciones nominales de las fases en el archivo de red de Mendoza, en segundos. 94
A.2. Semillas por escenario sintético en la fase 1. 95
A.3. Arquitecturas y semillas asignadas a las doce combinaciones de algoritmo y escenario de las redes sintéticas en fase 2. La semilla de evaluación es común a los algoritmos del mismo escenario; las semillas de TPE no son réplicas de entrenamiento. 96
A.4. Dominios de búsqueda de HAPPO por escenario y arquitectura. 97
A.5. Dominios de búsqueda de IDQN por escenario y arquitectura. 97
A.6. Dominios de búsqueda de IPPO y MAPPO. 98
A.7. Señales de recompensa por procedimiento documentado. 101
A.8. Hiperparámetros fijos de aprendizaje de MAPPO. 101

Siglas

A2C Actor–crítico con ventaja (*Advantage Actor-Critic*).

AFAC Asociación de Fábricas Argentinas de Componentes.

API Interfaz de programación de aplicaciones (*Application Programming Interface*).

ASHA Algoritmo asíncrono de reducción sucesiva a la mitad (*Asynchronous Successive Halving Algorithm*).

BFS Búsqueda en anchura (*Breadth-First Search*).

CCAD Centro de Cómputo de Alto Desempeño.

CCAR Centro de Cómputo de Alto Rendimiento.

CI Tasa de completitud de vehículos insertados (*Completion of Inserted Vehicles*).

CLIP Objetivo sustituto recortado (*Clipped surrogate objective*).

CO Monóxido de carbono.

CO2 Dióxido de carbono.

CONICET Consejo Nacional de Investigaciones Científicas y Técnicas.

CR Tasa de completitud (*Completion Rate*).

CTDE Entrenamiento centralizado con ejecución descentralizada (*Centralized Training with Decentralized Execution*).

CVaR Valor en riesgo condicional (*Conditional Value at Risk*).

DAG Grafo acíclico dirigido (*Directed Acyclic Graph*).

DDPG Gradiente de política determinista profundo (*Deep Deterministic Policy Gradient*).

Dec-POMDP Proceso de decisión de Markov parcialmente observable descentralizado (*Decentralized Partially Observable Markov Decision Process*).

DLR Centro Aeroespacial Alemán (*Deutsches Zentrum für Luft- und Raumfahrt*).

DQN Red Q profunda (*Deep Q-Network*).

DRL Aprendizaje por refuerzo profundo (*Deep Reinforcement Learning*).

E/S Entrada/salida (*Input/Output*).

FCEFN Facultad de Ciencias Exactas, Físicas y Naturales.

FHWA Administración Federal de Carreteras (*Federal Highway Administration*).

FLIR Infrarrojo de barrido frontal (*Forward Looking Infrared*).

GAE Estimación generalizada de ventaja (*Generalized Advantage Estimation*).

GEH Estadístico de comparación de flujos de Geoffrey E. Havers.

GUI Interfaz gráfica de usuario (*Graphical User Interface*).

HAPPO Optimización de política proximal con agentes heterogéneos (*Heterogeneous-Agent Proximal Policy Optimization*).

HARL Aprendizaje por refuerzo con agentes heterogéneos (*Heterogeneous-Agent Reinforcement Learning*).

HATRPO Optimización de política por región de confianza con agentes heterogéneos (*Heterogeneous-Agent Trust Region Policy Optimization*).

HBEFA Manual de factores de emisión para transporte por carretera (*Handbook Emission Factors for Road Transport*).

HC Hidrocarburos no combustionados.

HPC Computación de alto rendimiento (*High-Performance Computing*).

HPO Optimización de hiperparámetros (*Hyperparameter Optimization*).

IA Inteligencia artificial (*Artificial Intelligence*).

IDQN Red Q profunda independiente (*Independent Deep Q-Network*).

IL Aprendizaje independiente (*Independent Learning*).

IPPO Optimización de política proximal independiente (*Independent Proximal Policy Optimization*).

IQM Media intercuartílica (*Interquartile Mean*).

IR Tasa de inserción (*Insertion Rate*).

ITE Instituto de Ingenieros de Transporte (*Institute of Transportation Engineers*).

KL Divergencia de Kullback–Leibler.

MADDPG Gradiente de política determinista profundo multiagente (*Multi-Agent Deep Deterministic Policy Gradient*).

MADRL Aprendizaje por refuerzo profundo multiagente (*Multi-Agent Deep Reinforcement Learning*).

MAPPO Optimización de política proximal multiagente (*Multi-Agent Proximal Policy Optimization*).

MARL Aprendizaje por refuerzo multiagente (*Multi-Agent Reinforcement Learning*).

MDP Proceso de decisión de Markov (*Markov Decision Process*).

ML Aprendizaje automático (*Machine Learning*).

MLP Perceptrón multicapa (*Multi-Layer Perceptron*).

NFS Sistema de archivos de red (*Network File System*).

NOx Óxidos de nitrógeno.

OD Origen-destino (*Origin-Destination*).

OOD Fuera de distribución (*Out-of-Distribution*).

OSM OpenStreetMap.

PHEMlight Modelo de emisiones para vehículos livianos y pesados (*Passenger car and Heavy duty Emission Model light*).

POMDP Proceso de decisión de Markov parcialmente observable (*Partially Observable Markov Decision Process*).

POSIX Interfaz portable de sistemas operativos (*Portable Operating System Interface*).

PPO Optimización de política proximal (*Proximal Policy Optimization*).

QMIX Red de mezcla de valores Q (*Q-Mixing Network*).

ReLU Unidad lineal rectificada (*Rectified Linear Unit*).

RL Aprendizaje por refuerzo (*Reinforcement Learning*).

SGD Descenso de gradiente estocástico (*Stochastic Gradient Descent*).

SL Aprendizaje supervisado (*Supervised Learning*).

SLURM Herramienta simple de Linux para gestión de recursos (*Simple Linux Utility for Resource Management*).

SUMO Simulación de Movilidad Urbana (*Simulation of Urban MObility*).

TAZ Zona de análisis de tránsito (*Traffic Analysis Zone*).

TCP Protocolo de control de transmisión (*Transmission Control Protocol*).

TD Diferencia temporal (*Temporal-Difference*).

TPE Estimador de Parzen estructurado en árbol (*Tree-structured Parzen Estimator*).

TraCI Interfaz de control del tránsito (*Traffic Control Interface*).

TRPO Optimización de política por región de confianza (*Trust Region Policy Optimization*).

UL Aprendizaje no supervisado (*Unsupervised Learning*).

UML Lenguaje unificado de modelado (*Unified Modeling Language*).

UTM Sistema de coordenadas universal transversal de Mercator (*Universal Transverse Mercator*).

VDN Redes de descomposición de valor (*Value-Decomposition Networks*).

VF Función de valor (*Value Function*).

WGS84 Sistema Geodésico Mundial 1984 (*World Geodetic System 1984*).

WoLF-PHC Ascenso de colina en la política con ganar o aprender rápido (*Win or Learn Fast Policy Hill-Climbing*).

XML Lenguaje de marcado extensible (*Extensible Markup Language*).

Glosario

G_t **(retorno)** Suma acumulada, usualmente descontada, de las recompensas futuras a partir del instante t.

$G_{\mathbf{wait}}$ **(coeficiente de Gini de esperas)** Índice adimensional entre 0 y 1 que cuantifica la desigualdad entre las medias temporales de espera por intersección; no describe la distribución entre vehículos.

$M_{i_{1:m}}$ **(factor de reponderación HAPPO)** Factor acumulativo de razones de probabilidad que repondera la ventaja conjunta en la actualización secuencial de HAPPO para reflejar los cambios de política de los agentes predecesores.

$Q(s, a)$ **(valor de acción)** Retorno esperado de ejecutar la acción a en el estado s y continuar posteriormente de acuerdo con una política determinada.

$R_t^{\mathbf{target}}$ **(retorno objetivo)** Estimación de retorno utilizada como referencia para ajustar la predicción de valor del crítico en el instante t.

$S[\pi_\theta]$ **(entropía de la política)** Medida de la dispersión de la distribución de acciones de la política; su inclusión en el objetivo favorece la exploración.

$V(s)$ **(valor de estado)** Retorno esperado al comenzar en el estado s y seguir una política determinada, promediando las acciones que dicha política selecciona.

$W_{\mathbf{net}}$ **(demora acumulada global)** Métrica agregada que suma los segundos de espera de los vehículos detenidos en la red durante la ventana de evaluación.

α **(tasa de aprendizaje)** Paso de actualización que regula la magnitud del ajuste de los parámetros o valores en cada iteración de aprendizaje. En arquitecturas actor–crítico se especializa mediante subíndices, tales como α_θ para la red del actor y α_ϕ (o η) para el crítico.

δ_t **(error de diferencia temporal)** Diferencia entre la recompensa más el valor descontado del estado siguiente y la estimación de valor del estado actual.

ϵ_{clip} **(margen de recorte)** Hiperparámetro de PPO que delimita el intervalo utilizado para recortar la razón entre la probabilidad nueva y la anterior de una acción.

ϵ_{exp} **(exploración)** Probabilidad o coeficiente que controla la selección de acciones exploratorias frente a las acciones favorecidas por la política o la función de valor.

η **(tasa del optimizador)** Tamaño de paso empleado por el optimizador en la actualización de parámetros mediante descenso de gradiente.

γ **(factor de descuento)** Parámetro en el intervalo $[0, 1)$ que pondera la contribución de las recompensas futuras al retorno esperado.

$\hat{A}_t$ **(ventaja estimada)** Estimación del valor de la función de ventaja en el paso temporal t (habitualmente calculada mediante GAE), que cuantifica cuánto supera o desmejora el rendimiento de la acción ejecutada respecto del valor basal esperado del estado.

λ **(suavizado de la ventaja)** Hiperparámetro que pondera errores de diferencia temporal de distintos horizontes al estimar la ventaja y equilibra sesgo y varianza.

$\mathcal{L}(\theta)$ **(función de pérdida)** Función escalar que cuantifica el error del modelo y que se minimiza para ajustar los parámetros θ.

ϕ **(parámetros del crítico)** Vector de parámetros entrenables de la red que estima la función de valor del crítico.

π_θ **(política parametrizada)** Distribución de probabilidad sobre las acciones condicionada por el estado u observación y controlada por el vector de parámetros entrenables θ.

ρ_{coop} **(ponderación cooperativa)** Coeficiente en $[0, 1)$ que pondera la combinación entre la recompensa local de una intersección y la señal promedio de sus vecinos topológicos; los dominios de búsqueda pueden restringirlo a un intervalo menor.

θ **(parámetros de la política)** Vector de parámetros entrenables que determina la distribución de acciones de la política del actor.

a **(aceleración vehicular)** Parámetro de aumento de velocidad del tipo de vehículo en SUMO, expresado en metros por segundo cuadrado. En las ecuaciones de aprendizaje, la letra a puede representar una acción; son usos distintos que se distinguen por el contexto.

a_i, A_t **(acción)** Decisión ejecutada por un agente. En formulaciones temporales, A_t denota la variable aleatoria de acción en el paso t; en entornos multiagente, a_i representa la acción local del agente i y $\mathbf{a} = (a_1, \ldots, a_n)$ la acción conjunta. En el control semafórico corresponde a la fase asignada a la intersección.

c_1 **(coeficiente de valor)** Peso asignado al término de pérdida de la función de valor en el objetivo combinado de PPO.

c_2 **(coeficiente de entropía)** Peso asignado al término de entropía que incentiva la exploración en el objetivo combinado de PPO.

d **(desaceleración vehicular)** Parámetro de frenado del tipo de vehículo en SUMO, expresado como una magnitud positiva en metros por segundo cuadrado.

o_i **(observación local)** Información disponible para el agente i durante la ejecución descentralizada. Se obtiene a partir de los sensores o variables observables de la intersección que controla.

r_t **(recompensa)** Señal escalar recibida en el paso temporal t después de ejecutar una acción. Se utiliza para orientar la actualización de la política o de la función de valor.

s **(estado global)** Representación conjunta de las variables relevantes del entorno en un instante determinado. En el modelo multiagente puede incluir la información de todas las intersecciones y sus corrientes vehiculares.

$v_{\text{máx}}$ **(velocidad máxima vehicular)** Velocidad máxima configurada para un tipo de vehículo, expresada en metros por segundo; el límite de la vía también restringe la velocidad permitida.

Binding Mecanismo que conecta una biblioteca escrita en un lenguaje de programación con otro lenguaje para poder utilizar sus funciones.

Bootstrap Procedimiento de remuestreo con reemplazo utilizado para aproximar la incertidumbre de un estimador a partir de las observaciones disponibles.

Buffer Estructura que almacena temporalmente datos para su posterior procesamiento o remuestreo en algoritmos de aprendizaje.

Epoch **(época)** Recorrido completo de optimización sobre el conjunto de datos recolectado para una iteración; puede reutilizarse durante varios *epochs*.

Logit Puntuación real sin normalizar que una red de política produce para una acción antes de convertirla en probabilidad.

Minibatch **(minilote)** Subconjunto reducido de muestras extraído de un lote utilizado para estimar el gradiente y actualizar los parámetros de un modelo durante el entrenamiento.

Replay buffer Estructura que almacena experiencias previas para remuestrearlas durante el entrenamiento de un método basado en valor.

Rollout Secuencia ordenada de transiciones generada mientras una política interactúa con el entorno.

Softmax Función que transforma un conjunto de puntuaciones reales en probabilidades no negativas cuya suma es uno.

***Teleport* (SUMO)** Mecanismo específico de SUMO que retira temporalmente un vehículo bloqueado y lo reinserta posteriormente para evitar que la simulación quede detenida.

Actor (Aprendizaje por Refuerzo) Componente de la arquitectura del modelo de aprendizaje responsable de seleccionar y ejecutar la acción en el entorno basándose en el estado actual y la política que está aprendiendo.

Agente Entidad computacional que observa el estado de su entorno y ejecuta acciones según una política determinada con el objetivo de maximizar su recompensa a largo plazo.

Ciclo (Ingeniería de Tránsito) Secuencia completa y repetitiva de todas las fases semafóricas programadas en una intersección particular.

Cola vehicular Fila de vehículos que se detienen o avanzan muy lentamente debido a una restricción en la capacidad de la vía, tal como la luz roja de un semáforo.

Crítico (Aprendizaje por Refuerzo) Componente encargado de estimar la función de valor, prediciendo la recompensa futura esperada para evaluar la calidad de las acciones tomadas por el Actor y así guiar su actualización.

Densidad vehicular Número de vehículos presentes en un tramo o longitud específica de un carril en un instante dado.

Entorno El sistema dinámico con el que interactúa el agente. Responde a las acciones del agente produciendo transiciones de estado y emitiendo señales de recompensa.

Episodio Secuencia de interacción que comienza al reiniciar el entorno y termina cuando se alcanza una condición terminal o el horizonte máximo definido.

Estado Representación cuantitativa o cualitativa de la situación actual del entorno en un instante de tiempo específico. Sirve como base para que el agente seleccione su próxima acción.

Fase (Ingeniería de Tránsito) Estado temporal de un controlador semafórico que habilita el derecho de paso a uno o más movimientos o corrientes vehiculares compatibles entre sí para que crucen una intersección de forma segura.

Flujo vehicular Cantidad de vehículos que pasan por una sección transversal específica de un carril o calzada durante un intervalo de tiempo determinado.

Función de Valor Estimación matemática de la cantidad total de recompensa que un agente puede esperar acumular en el futuro, comenzando desde un estado particular y siguiendo una política específica.

Iteración de entrenamiento Ciclo completo en el que se recolectan datos con una política y se realizan una o más actualizaciones de sus parámetros.

Lote (*batch*) Conjunto de muestras o transiciones procesadas conjuntamente durante una operación de entrenamiento..

Memoria temporal de espera Intervalo finito sobre el cual SUMO acumula la espera consultada mediante TraCI; se distingue de la espera total registrada por viaje en TripInfo.

Observación Subconjunto de información del estado completo del entorno que es efectivamente visible o medible por un agente individual, especialmente relevante en escenarios de observabilidad parcial.

Par de políticas Unidad experimental compuesta por dos políticas entrenadas con la misma semilla y evaluadas con el mismo conjunto de semillas de simulación.

Política (Aprendizaje por Refuerzo) Estrategia que define el comportamiento del agente en un momento dado, mapeando los estados observados a probabilidades de selección de las acciones disponibles.

Proceso de Decisión de Markov Marco matemático utilizado para modelar la toma de decisiones en situaciones donde los resultados son en parte aleatorios y en parte bajo el control de un agente tomador de decisiones.

Recompensa Señal escalar devuelta por el entorno que evalúa cuán buena o mala fue la acción ejecutada por el agente en un estado particular. El objetivo del agente es maximizar la suma total de estas señales.

Semilla Número que inicializa un generador de números pseudoaleatorios. Permite repetir su secuencia cuando se conservan el procedimiento y la configuración; usar la

misma semilla no garantiza resultados idénticos si se modifican las decisiones del agente o el entorno.

Serialización Conversión de un objeto o mensaje a una representación que puede almacenarse o transmitirse y luego reconstruirse.

Telemetría Conjunto de mediciones que el simulador registra durante la ejecución, como velocidades, posiciones, detenciones y tiempos de espera.

Transición (Aprendizaje por Refuerzo) Registro de un paso de interacción que contiene el estado u observación actual, la acción ejecutada, la recompensa recibida, el estado siguiente y, cuando corresponde, un indicador de terminación.

Ventaja (Aprendizaje por Refuerzo) Métrica que cuantifica cuánto mejor o peor resulta tomar una acción particular en un estado dado, en comparación con el desempeño promedio esperado según la política actual.

1

Introducción

En las últimas décadas, el crecimiento del parque automotor y la urbanización sostenida han convertido a la congestión vehicular en uno de los principales desafíos para la movilidad urbana. Según los reportes de flota vehicular de Argentina elaborados por la Asociación de Fábricas Argentinas de Componentes (AFAC), y tomando 2012 como línea base, el volumen de vehículos aumentó un 21,7 % en 2017. Esta tendencia continuó durante la última década y alcanzó un incremento acumulado del 44,4 % en 2025 (véase la Figura 1.1) [Aso18, Aso26].

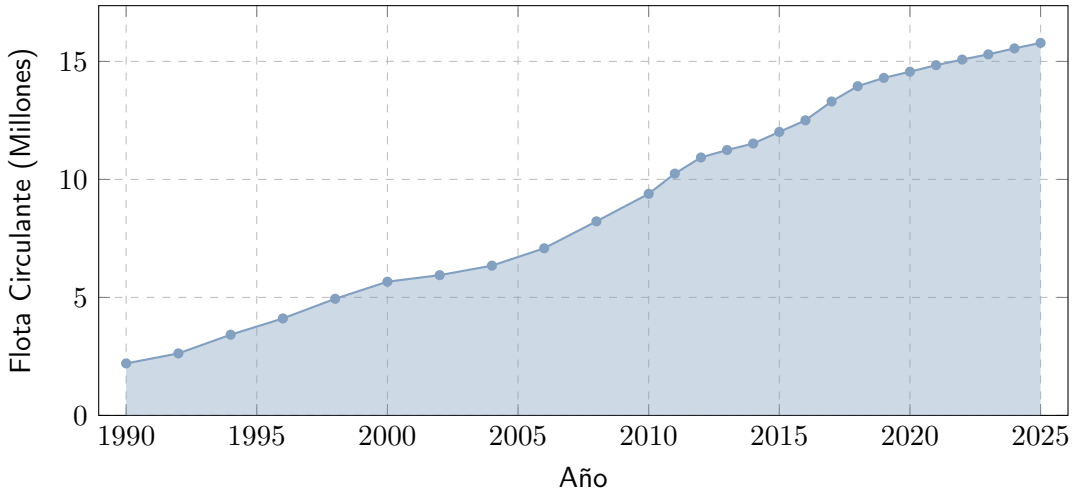


Figura 1.1: Evolución de la flota vehicular circulante estimada en Argentina entre 1990 y 2025, expresada en millones de vehículos. La línea y los marcadores representan los valores anuales disponibles; el área sombreada facilita la lectura de su magnitud. Elaboración propia a partir de reportes de AFAC.

Como refleja la Figura 1.1, la flota vehicular activa en Argentina experimentó una expansión sostenida entre 1990 y 2025, pasando de aproximadamente 2,2 millones a 15,78 millones de unidades operativas [Aso18, Aso26]. En los reportes de AFAC, la flota circulante comprende automóviles y vehículos comerciales livianos y pesados en condiciones de circular, excluyendo acoplados y remolques; representa una estimación del parque activo bajo esa definición, no el total histórico de unidades patentadas ni la cantidad de viajes realizados. Este incremento acumulado ejerce una presión creciente sobre redes viales consolidadas que poseen una capacidad física fija, agudizando la formación de colas y demoras en las intersecciones urbanas críticas.

La congestión deteriora las condiciones de operación de la red e incrementa las demoras y las detenciones de los vehículos [CyMR18]. Este trabajo aborda decisiones de fases semafóricas en tres escenarios sintéticos y diseña su evaluación también para una red del microcentro de la Ciudad de Mendoza, construida con información parcial de tránsito. La comparación local y vecinal de MAPPO se circunscribe a las redes sintéticas; Mendoza se incorpora al diseño de búsqueda y se comprueba por separado con sus programas semafóricos. La geometría y las condiciones de construcción se especifican en la sección 3.2.

Frente a la congestión, una alternativa que no requiere modificaciones importantes de la infraestructura consiste en ajustar la operación de las intersecciones mediante sistemas semafóricos adaptativos. Los enfoques tradicionales pueden presentar limitaciones para responder a variaciones en el flujo vehicular [PW16]. En este contexto, los avances en Inteligencia artificial (*Artificial Intelligence*) (IA) y en particular el Aprendizaje por refuerzo (*Reinforcement Learning*) (RL) permiten modelar los semáforos como agentes que aprenden políticas de control a partir de la interacción con el entorno. Las acciones corresponden a cambios de fase y la recompensa puede definirse, por ejemplo, como el negativo del tiempo de espera acumulado. Diversos estudios han mostrado la viabilidad de este enfoque, desde algoritmos tabulares aplicados a una sola intersección [Wie00] hasta modelos de redes coordinadas basados en redes neuronales profundas y control multiagente [W$^+$19].

Sin embargo, la aplicación simultánea de este enfoque en múltiples intersecciones introduce desafíos adicionales. Un esquema centralizado puede resultar poco escalable, pues el espacio de acciones conjunto crece multiplicativamente con el número de intersecciones [ACS24]. En una formulación de Aprendizaje por refuerzo multiagente (*Multi-Agent Reinforcement Learning*) (MARL) con Aprendizaje independiente (*Independent Learning*) (IL), cada semáforo optimiza su propia intersección sin considerar explícitamente el efecto de sus decisiones sobre el resto de la red [AS22]. Además, como todos los agentes aprenden y modifican sus políticas al mismo tiempo, cada uno observa un entorno no estacionario. Esto dificulta la convergencia de los algoritmos de agente único [FNF$^+$17].

Por esta razón, este trabajo estudia el control multiagente con un objetivo de coor-

dinación vial. Cada agente recibe una recompensa individual. Las búsquedas informadas emplean una señal local, mientras que una línea experimental separada compara MAPPO local con una variante que agrega observaciones y recompensa vecinales. En MAPPO y HAPPO, un crítico centralizado incorpora información conjunta durante el entrenamiento. La observación vecinal, la recompensa mixta y la centralización del crítico son mecanismos distintos cuyo uso depende del protocolo.

El objetivo general es desarrollar y evaluar procedimientos de control semafórico mediante aprendizaje por refuerzo multiagente. Se consideran MAPPO y HAPPO con entrenamiento centralizado y ejecución descentralizada, e IPPO e IDQN con aprendizaje independiente. El diseño de búsqueda comprende estos cuatro algoritmos en las redes sintéticas y en Mendoza; la exploración amplia documentada corresponde a las redes sintéticas y los dominios de la segunda fase se reducen a partir de esa evidencia. En las tres redes sintéticas se compara además una configuración de cooperación vecinal de MAPPO con su variante local mediante pares con semillas compartidas. La comprobación operacional de Mendoza usa los programas semafóricos incluidos en la red, no políticas aprendidas.

La propuesta incorpora una codificación de la espera acumulada por carril mediante una transformación logarítmico-lineal en I niveles. La resolución no uniforme distingue con mayor detalle valores bajos de esa medida y acota los valores representados a partir de un umbral. La elección de cómo representar el tránsito influye en el problema de aprendizaje y cuenta con antecedentes en control semafórico [GR18]. En este trabajo, la codificación se utiliza en todos los agentes.

También se estudia una adaptación de HAPPO para intersecciones con distintas geometrías, fases disponibles y demandas. A diferencia de la actualización de actores utilizada en MAPPO, esta adaptación los actualiza secuencialmente y repondera las ventajas individuales según los cambios de política precedentes [KCW+22]. El crítico centralizado incorpora información conjunta durante el entrenamiento. La comparación con MAPPO, IPPO y IDQN examina el desempeño de procedimientos completos en esos escenarios, sin atribuir de antemano una ventaja a HAPPO ni aislar el efecto de su actualización secuencial.

1.1. Motivación

La motivación de este trabajo se centra en estudiar cómo coordinar agentes que aprenden en una misma red vial. El control adaptativo basado en aprendizaje por refuerzo permite ajustar decisiones a las condiciones observadas [MMLK25]. En una red de múltiples intersecciones, interesa examinar cómo esa adaptación puede incorporar los efectos de las decisiones locales sobre los cruces vecinos.

Las diferencias de demanda, geometría y fases disponibles entre intersecciones motivan, además, el estudio de políticas que puedan adaptarse a esas condiciones. La comparación

de MAPPO y HAPPO con los esquemas independientes IPPO e IDQN permite examinar distintas formas de aprendizaje y coordinación bajo una misma formulación del problema.

1.2. Organización del documento

Para dar respuesta a los objetivos planteados y estructurar el desarrollo de la investigación, el presente documento se organiza en cinco capítulos y un apéndice técnico.

Este capítulo contextualiza la congestión vehicular urbana y presenta el alcance y los objetivos del trabajo. El capítulo 2 desarrolla los fundamentos de ingeniería de tránsito, procesos de decisión de Markov, aprendizaje profundo y aprendizaje por refuerzo multiagente necesarios para seguir el diseño experimental.

El capítulo 3 describe los escenarios, la representación de observaciones, acciones y recompensas, y los protocolos de búsqueda y evaluación. Comprende las tres redes sintéticas y la red urbana de Mendoza, construida con conteos parciales; la comprobación de esta última con los programas semafóricos de la red se distingue de la búsqueda de políticas. El capítulo 4 organiza la evidencia de búsqueda, la comparación entre las variantes local y vecinal de MAPPO y la comprobación operacional de Mendoza, que no evalúa políticas aprendidas.

Finalmente, el capítulo 5 sintetiza las conclusiones principales a la luz de los objetivos propuestos, delimita el alcance y las limitaciones del estudio, y proyecta líneas de trabajo futuro. Como complemento técnico, el apéndice A describe el diseño de software del paquete `marlTLC` mediante diagramas de clases en Lenguaje unificado de modelado (*Unified Modeling Language*) (UML) y reúne los dominios, las semillas y las especificaciones necesarias para reproducir los experimentos.

2

Marco teórico

Este capítulo presenta los fundamentos teóricos del trabajo y vincula la ingeniería de tránsito con el aprendizaje por refuerzo para la toma de decisiones secuenciales en redes viales. Estos conceptos permiten modelar una red semafórica como un entorno estocástico y formular políticas de control adaptativo con agentes homogéneos y heterogéneos.

Primero se introducen los conceptos de ingeniería de tránsito y simulación (sección 2.1). Luego se desarrollan los fundamentos del aprendizaje por refuerzo y su integración con redes neuronales (sección 2.2), seguidos de su extensión al caso multiagente y de los algoritmos MAPPO y HAPPO (sección 2.3). Finalmente, se revisan los antecedentes recientes de control semafórico multiagente según sus esquemas de coordinación, representación del estado y tratamiento de la heterogeneidad (sección 2.3.7).

2.1. Ingeniería de tránsito y simulación

La ingeniería de tránsito es una rama de la ingeniería de transporte, tradicionalmente vinculada con la ingeniería civil. Se ocupa de la planificación, el diseño geométrico y la operación segura y eficiente del tránsito en calles, carreteras y redes viales, de acuerdo con manuales especializados como el *Traffic Engineering Handbook* [PW16] y con la literatura clásica del área [CyMR18]. En este trabajo, sus conceptos se abordan desde la perspectiva del modelado computacional y la gestión adaptativa. Permiten describir el funcionamiento físico de la red, caracterizar la oferta y la demanda de circulación e identificar las variables sobre las que puede intervenir el sistema de control.

Históricamente, la práctica de la ingeniería de tránsito puso un fuerte énfasis en aumentar la capacidad física vial mediante la construcción de nuevas obras o el ensanche de

calzadas [PW16]. Sin embargo, el crecimiento sostenido del parque automotor evidenció que la ampliación de la infraestructura no constituye una solución definitiva al problema de la congestión. En áreas urbanas consolidadas, además, modificar el trazado vial resulta costoso y, en algunos casos, difícil de realizar [CyMR18]. Por ello, la práctica contemporánea complementa la infraestructura física con estrategias de control dinámico del tránsito, priorizando la optimización de los recursos existentes mediante dispositivos de control semafórico.

La operación de una red vial puede analizarse a partir de la relación entre la demanda vehicular y la capacidad disponible. La demanda vehicular (D) corresponde a la cantidad de vehículos que requieren desplazarse por un determinado sistema vial en un lapso dado, mientras que la oferta vial o capacidad (c) representa la cantidad máxima de vehículos que pueden circular por dicho espacio físico bajo determinadas condiciones de operación [CyMR18]. Cuando la demanda se aproxima a la capacidad, el tránsito se vuelve inestable; cuando la supera ($D > c$), se presentan condiciones de sobresaturación o flujo forzado, caracterizadas por detenciones frecuentes y grandes demoras. En este sentido, la congestión no se entiende solo como una percepción general de mal funcionamiento, sino como un fenómeno con manifestaciones físicas medibles: demoras, colas y detenciones [FA11].

Estas magnitudes cumplen una función doble en el modelo computacional: pueden formar parte de las observaciones y de la señal de recompensa de los agentes, y también se utilizan para evaluar el desempeño del agente. Su definición permite vincular la descripción física del tránsito con la formulación posterior del problema de aprendizaje.

2.1.1. Terminología de tránsito

Antes de formular el problema de control, es necesario distinguir algunos términos que se utilizarán a lo largo del capítulo. En particular, se diferencian los movimientos que realizan los vehículos, las fases que habilitan esos movimientos y los intervalos durante los cuales se mantienen las indicaciones semafóricas [Fed08].

En este contexto, se emplearán las siguientes nociones:

- **Intersección (*intersection*), acceso (*approach*) y carril (*lane*):** una intersección es el área física y operacional en la que confluyen dos o más corrientes de tránsito. Cada una de las calzadas de aproximación que ingresan a ella constituye un acceso [PW16], organizándose internamente en uno o más carriles destinados a canalizar los flujos vehiculares [CyMR18].

- **Movimiento vehicular y movimientos en conflicto:** un movimiento vehicular representa la trayectoria específica que describe un vehículo al cruzar la intersección hacia una salida determinada, por ejemplo, un giro o un movimiento directo [W$^+$19]. Dos movimientos pueden generar puntos de conflicto cuando sus trayectorias se intersectan (véase la Figura 2.2). Cuando la geometría y la señalización no permiten

proteger su operación simultánea, esos movimientos deben programarse en intervalos distintos por razones de seguridad [FA11].

- **Fase (*phase*), ciclo (*cycle*) e intervalo (*interval*):** una fase es una configuración de señales que habilita simultáneamente un conjunto de movimientos compatibles [PW16]. El ciclo corresponde a la secuencia completa de fases necesaria para atender los movimientos de la intersección. Un intervalo es el período durante el cual las indicaciones semafóricas permanecen constantes [CyMR18].

La Figura 2.1 ilustra estos conceptos en una intersección de cuatro accesos. Los números del 1 al 8 identifican movimientos vehiculares individuales y las etiquetas $P2$, $P4$, $P6$ y $P8$ identifican cruces peatonales del esquema. Esta numeración sigue de manera esquemática una convención habitual de los manuales de temporización semafórica; no representa una configuración experimental ni una asignación obligatoria de fases para el caso de estudio. Las flechas, los accesos y las etiquetas de calles permiten distinguir trayectorias y sentidos, mientras que los cruces peatonales muestran otros usuarios que también deben considerarse al ordenar los movimientos. En este trabajo, la figura se utiliza para separar la noción geométrica de movimiento de la regla semafórica que habilita un conjunto de movimientos compatibles [Fed08]. Por ejemplo, dos movimientos que no comparten un punto de conflicto pueden incluirse en una misma fase, mientras que los que sí lo comparten deben atenderse en fases compatibles o sucesivas.

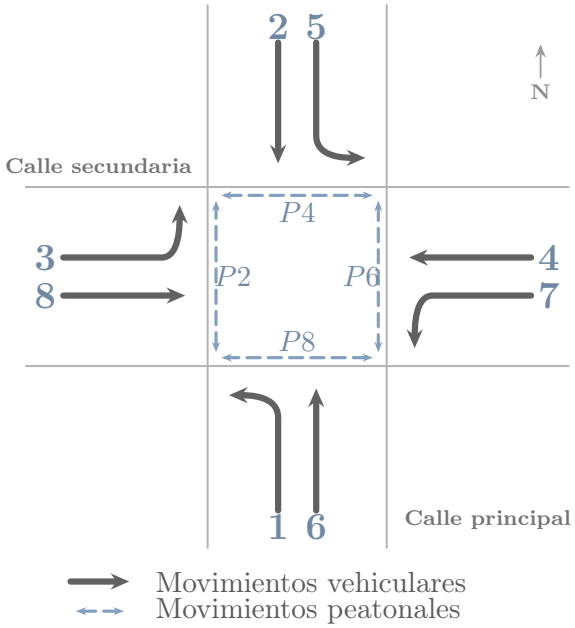


Figura 2.1: Convención esquemática de movimientos vehiculares, fases y cruces peatonales concurrentes en una intersección de cuatro accesos, adaptada conceptualmente del manual de temporización semafórica de Administración Federal de Carreteras (*Federal Highway Administration*) (FHWA) [Fed08].

La Figura 2.2 presenta tres formas esquemáticas de relación entre trayectorias. Hay

cruce cuando dos trayectorias se intersectan en un punto; *convergencia* cuando trayectorias distintas se unen en una misma corriente; y *divergencia* cuando una corriente se separa en trayectorias diferentes. Los puntos coloreados marcan el lugar de interacción que se analiza en cada caso. Cada relación puede originar un punto de conflicto según la geometría y la simultaneidad de los movimientos: por eso, la incompatibilidad no depende solo de que las líneas se relacionen, sino de que su operación conjunta comprometa la seguridad. En el modelo semafórico, estos casos se utilizan para determinar qué movimientos pueden habilitarse en una misma fase y cuáles deben retenerse; la figura muestra relaciones geométricas, no una programación temporal concreta.

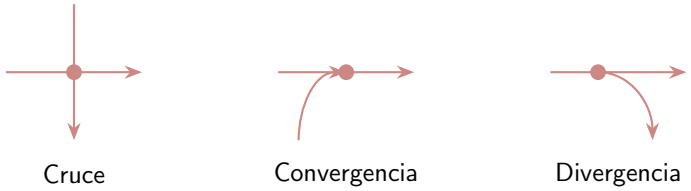


Figura 2.2: Representación esquemática de movimientos vehiculares y puntos de conflicto en una intersección semaforizada.

En arterias urbanas, el tránsito se desarrolla bajo condiciones de flujo interrumpido: los vehículos no avanzan únicamente por su interacción con otros vehículos, sino también por la presencia de intersecciones, señales y dispositivos de control de tránsito [PW16]. Las intersecciones son puntos críticos de la red porque allí confluyen distintos accesos y movimientos vehiculares de cruce, convergencia y divergencia que pueden resultar incompatibles entre sí, como resume la Figura 2.2. Por ello, su operación requiere ordenar estos movimientos y asignar la prioridad de paso de manera segura y eficiente.

El semáforo es uno de los dispositivos de control de tránsito utilizados para regular la circulación en intersecciones. Su función es asignar la prioridad de paso en forma alternada entre corrientes vehiculares incompatibles, evitando que distintos movimientos intenten ocupar simultáneamente el mismo espacio de cruce [FA11]. Desde el punto de vista del modelado algorítmico, esta capacidad de modificar la prioridad de paso convierte al semáforo en el actuador mediante el cual un sistema de control puede intervenir sobre la operación de la intersección y afectar, directa o indirectamente, las demoras y colas observadas en la red.

2.1.2. Fundamentos de la dinámica vehicular

Para modelar y optimizar la operación de una red vial, es necesario caracterizar físicamente el movimiento de los vehículos. La ingeniería de tránsito estudia este fenómeno a partir de las características fundamentales de la corriente de tránsito, que permiten describir las condiciones de operación de una vía o intersección mediante magnitudes observables como el flujo, la velocidad y la densidad [PW16]. Cal y Mayor distingue estas magnitudes entre variables principales, que describen el comportamiento agregado del flu-

jo vehicular, y variables asociadas, que expresan relaciones temporales y espaciales entre vehículos consecutivos [CyMR18].

A nivel macroscópico, la corriente de tránsito se representa mediante variables agregadas como la tasa de flujo, la densidad y los promedios de velocidad. A nivel microscópico, se analizan magnitudes propias de vehículos individuales e interacciones entre pares consecutivos, como la velocidad puntual, el intervalo y el espaciamiento [CyMR18]. Esta separación entre escalas es fundamental para el modelado computacional: las variables macroscópicas resumen el estado operacional de los accesos de la red, mientras que las variables microscópicas describen el comportamiento de cada vehículo dentro del simulador [FA11]. La correspondencia y las relaciones de conversión entre las variables de ambas escalas se resumen en la Tabla 2.1.

En el nivel macroscópico, las variables principales de la corriente de tránsito son [CyMR18]:

- **Flujo o tasa de flujo (q):** representa la cantidad de vehículos que atraviesan una sección transversal de la vía por unidad de tiempo. Si el período de observación es menor a una hora, se proyecta como una tasa horaria equivalente en vehículos por hora, calculada como $q = N/T$, donde N es el número de vehículos observados y T es la duración del intervalo de medición expresado en horas.

- **Densidad o concentración (k):** indica el número de vehículos que ocupan simultáneamente una longitud específica de vía, expresada en vehículos por kilómetro, calculada como $k = N/d$, donde d representa la longitud del tramo de vía analizado. En la práctica, se suele estimar indirectamente a través del porcentaje de ocupación temporal registrado por detectores inductivos [PW16].

- **Velocidad media espacial (v_s):** es la velocidad media de los vehículos que ocupan un tramo de vía en un instante determinado. Corresponde al promedio armónico de las velocidades puntuales de los vehículos individuales y representa la magnitud de velocidad consistente con la dinámica del flujo espacial.

- **Velocidad media temporal (v_t):** representa el promedio aritmético de las velocidades de los vehículos que cruzan una sección transversal fija de la vía durante un período de tiempo determinado.

En el nivel microscópico, las variables asociadas son [CyMR18]:

- **Velocidad puntual (v_i):** es la velocidad instantánea a la que un vehículo individual (i) atraviesa una sección transversal específica de la vía.

- **Intervalo temporal (h):** es el tiempo transcurrido entre el paso de dos vehículos consecutivos por una sección fija de la calzada, medido respecto a puntos homólogos,

por ejemplo, los paragolpes delanteros. El intervalo promedio en segundos (h_{prom}) se relaciona con la tasa de flujo como $h_{\text{prom}} = 3600/q$.

- **Espaciamiento simple (s):** es la distancia física entre puntos homólogos de dos vehículos consecutivos en un instante de tiempo. El espaciamiento promedio en metros (s_{prom}) se relaciona de forma inversa con la densidad de la corriente de tránsito como $s_{\text{prom}} = 1000/k$.

La Figura 2.3 complementa estas definiciones con las magnitudes que se miden en el espacio y en el tiempo. El largo del vehículo l y la separación libre e son distancias, expresadas en metros; el espaciamiento s es la distancia entre puntos homólogos de vehículos consecutivos, también en metros. El avance temporal β es el tiempo asociado al recorrido del largo del vehículo a la velocidad v, la brecha temporal g es el tiempo entre el paso del vehículo precedente y el siguiente respecto de puntos de referencia distintos, y el intervalo temporal h es el tiempo entre pasos de puntos homólogos; β, g y h se expresan en segundos. Las cotas inferiores representan relaciones espaciales y las superiores relaciones temporales; no son dos vehículos distintos ni dos mediciones independientes. En este trabajo, estas magnitudes sirven para vincular la descripción individual de los vehículos con las variables agregadas del flujo.

Tabla 2.1: Comparación y correspondencia de variables de tránsito en escalas macroscópica y microscópica.

Escala	**Variable Principal**	**Relación / Conversión**
Macroscópica	Flujo (q, vehículos por hora)	$q = 3600/h_{\text{prom}}$
	Densidad (k, vehículos por kilómetro)	$k = 1000/s_{\text{prom}}$
	Velocidad media espacial (v_s, kilómetros por hora)	$q = k \cdot v_s$
	Velocidad media temporal (v_t, kilómetros por hora)	$v_t = v_s + \frac{\sigma_s^2}{v_s}$ (Relación de Wardrop)
Microscópica	Velocidad puntual (v_i, kilómetros por hora)	Variable individual de cada vehículo
	Intervalo temporal (h, segundos)	Tiempo entre pasos de vehículos sucesivos
	Espaciamiento simple (s, metros)	Distancia espacial entre paragolpes homólogos

La Figura 2.3 presenta de manera conjunta estas distancias y tiempos entre vehículos, y permite interpretar cómo las magnitudes microscópicas describen la separación y el avance de una corriente.

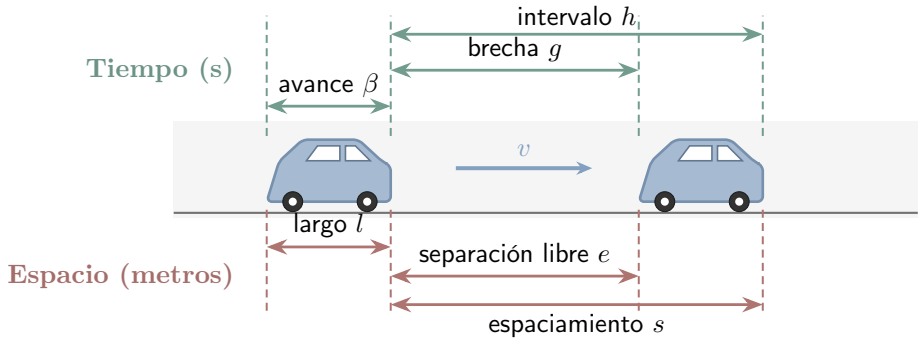


Figura 2.3: Relaciones temporales y espaciales entre vehículos consecutivos de una corriente de tránsito: intervalo (h) y espaciamiento (s).

Estas variables macroscópicas se relacionan de manera directa a través de la ecuación fundamental del flujo de tránsito:

$$q = k \cdot v_s \tag{2.1}$$

donde q es la tasa de flujo, k es la densidad y v_s es la velocidad media espacial. La ecuación (2.1) utiliza la velocidad media espacial, ya que relaciona el flujo y la cantidad de vehículos presentes en un tramo. La velocidad media temporal (v_t) es la media aritmética de las velocidades observadas en un punto durante un intervalo, mientras que v_s, asociada al promedio de las velocidades de una muestra espacial tomada sobre un tramo, se estima en la práctica como media armónica de las velocidades puntuales medidas en una sección durante un intervalo. Cuando existe variabilidad entre las velocidades, ambas magnitudes pueden diferir. La relación entre ellas se expresa mediante la relación de Wardrop [FA11]:

$$v_t = v_s + \frac{\sigma_s^2}{v_s} \tag{2.2}$$

donde σ_s^2 representa la varianza de la distribución de las velocidades de los vehículos en el espacio. La ecuación (2.2) muestra que, dado que la varianza es no negativa ($\sigma_s^2 \geq 0$), se cumple $v_t \geq v_s$, con igualdad únicamente cuando todos los vehículos circulan a la misma velocidad. En una intersección semaforizada, una indicación roja puede reducir localmente la velocidad media y aumentar la acumulación de vehículos en los accesos. Estos efectos se manifiestan en la formación de colas y en el incremento de las demoras [CyMR18].

Métricas de ineficiencia operativa

La interrupción del flujo causada por semáforos o saturación genera efectos adversos que los sistemas de control buscan mitigar. Siguiendo la terminología de la ingeniería de tránsito, las principales manifestaciones medibles de esta ineficiencia son las demoras, las colas y las detenciones [FA11]:

- **Demoras:** representan el tiempo adicional que un vehículo debe invertir para atravesar una intersección respecto del tiempo que le tomaría circular a velocidad de flujo libre. En intersecciones semaforizadas, una parte de esa demora aparece por la espera normal durante la luz roja. Otras demoras se producen cuando las llegadas de vehículos varían, cuando la demanda se acerca o supera la capacidad de la intersección, o cuando quedan colas sin disiparse al finalizar el verde, de modo que algunos vehículos deben esperar más de un ciclo para cruzar.

- **Colas:** corresponden al número de vehículos acumulados detrás de una línea de detención a la espera de traspasarla. Esta medida espacial es especialmente relevante porque, cuando la cola supera la longitud del arco de aproximación, puede bloquear la intersección precedente y propagar la congestión al resto de la red.

- **Detenciones:** cuantifican la cantidad de veces que un vehículo debe reducir su velocidad hasta cero durante el recorrido. Una alta frecuencia de detenciones deteriora la continuidad operativa del flujo y suele asociarse con mayores procesos de aceleración y desaceleración.

Estas magnitudes permiten describir el desempeño operacional de una intersección y de la red. En la formulación computacional, también pueden utilizarse para construir observaciones, señales de recompensa o criterios de evaluación del agente.

Descripción probabilística del flujo y agrupamientos vehiculares

En el modelado de tránsito, las llegadas vehiculares no siempre pueden representarse adecuadamente como una secuencia determinista y constante. Para flujos vehiculares bajos y medios, y cuando las llegadas no están fuertemente condicionadas por semáforos previos, es habitual aproximar el número de vehículos que llega a una sección durante un intervalo de tiempo mediante un proceso de Poisson [CyMR18].

Esta aproximación supone independencia estadística entre llegadas y permite describir la variabilidad observada en la demanda de acceso a una intersección [CyMR18]. La probabilidad discreta $P(X)$ de que lleguen exactamente X vehículos en un intervalo de tiempo se define como:

$$P(X) = \frac{m^X \cdot e^{-m}}{X!} \tag{2.3}$$

donde $X \in \mathbb{N}_0$ es la variable aleatoria discreta que representa el número de arribos vehiculares, $m > 0$ representa el valor esperado o media de llegadas en dicho intervalo, y e es la base del logaritmo natural. La ecuación (2.3) formaliza esta probabilidad discreta. Bajo el mismo supuesto, los intervalos de tiempo continuos (h) medidos entre vehículos sucesivos se describen mediante una distribución de probabilidad exponencial negativa [CyMR18]:

$$P(h \geq t) = e^{-\frac{q}{3600} \cdot t} \tag{2.4}$$

donde h es el intervalo temporal en segundos, q es el flujo en vehículos por hora, y t es el umbral de tiempo analizado. La ecuación (2.4) expresa la probabilidad de que el intervalo supere dicho umbral.

Esta representación probabilística supone independencia entre arribos sucesivos y resulta adecuada para flujos bajos o moderados en condiciones de flujo libre, sin influencia de semáforos cercanos en intersecciones precedentes [CyMR18]. El supuesto pierde validez en condiciones de congestión o en redes urbanas semaforizadas. En estos casos, los semáforos interrumpen el flujo agrupando los vehículos, lo que introduce dependencia espacio-temporal entre arribos consecutivos. La Figura 2.4 compara conceptualmente los intervalos temporales resultantes bajo ambas condiciones. En ella, $h_{\text{mín}}$ representa un umbral mínimo conceptual para el intervalo mostrado en la curva con agrupamientos vehiculares; no es una estimación obtenida de datos de este trabajo. Las curvas y ese umbral tienen, por lo tanto, función ilustrativa y no constituyen una distribución empírica particular.

En este trabajo, la limitación de esos modelos analíticos motiva el uso de simulación microscópica para analizar la operación semafórica bajo condiciones variables. En el simulador *Simulation of Urban Mobility* (SUMO), la formación, dispersión e interacción de los agrupamientos vehiculares emergen de la dinámica simulada de los vehículos y de sus cambios de carril. Los procesos aleatorios del simulador se inicializan con una semilla (*seed*), un número que permite repetir su secuencia pseudoaleatoria bajo la misma configuración y versión. Cambiar la semilla permite estudiar variaciones de la simulación. Repetirla no hace iguales los resultados si también cambia el agente o sus decisiones.

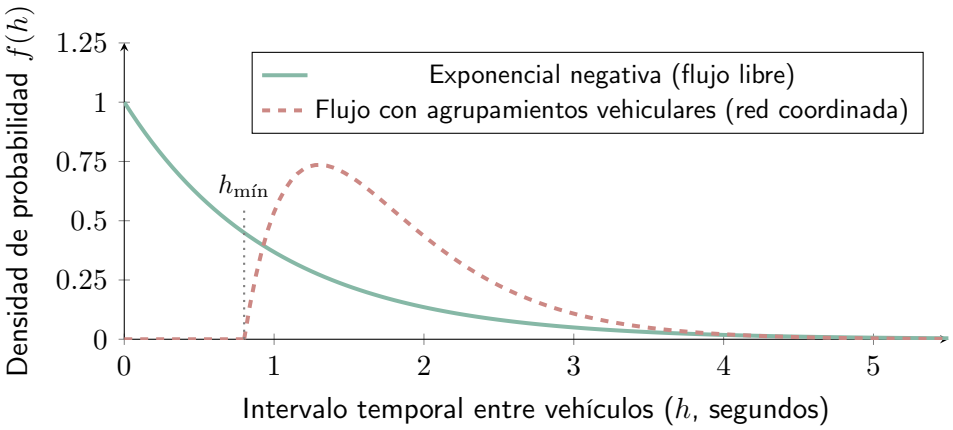


Figura 2.4: Comparación esquemática de distribuciones de intervalos temporales entre vehículos en flujo libre y en una corriente con formación de agrupamientos vehiculares. Las curvas representan situaciones conceptuales y no una distribución empírica particular.

La Figura 2.4 se interpreta comparando la forma de ambas curvas. El eje horizontal representa el intervalo h entre vehículos y el eje vertical la densidad de probabilidad $f(h)$, es decir, la concentración relativa de intervalos alrededor de cada valor. En el modelo

exponencial, la densidad disminuye desde el origen y asigna mayor frecuencia relativa a los intervalos breves. En la representación con agrupamientos vehiculares, la densidad comienza después de $h_{\text{mín}}$ y alcanza un máximo alejado del origen, lo que ilustra que el arribo de un vehículo queda condicionado por la separación y la coordinación con los que lo preceden. La comparación permite reconocer por qué el supuesto de arribos independientes puede resultar insuficiente en una red semaforizada.

2.1.3. Sistemas de control de tránsito

Para ordenar los movimientos en conflicto y regular el paso en las intersecciones urbanas se utilizan sistemas de control de tránsito [PW16]. El semáforo asigna el derecho de paso de manera alternada mediante indicaciones luminosas rojas, amarillas y verdes [CyMR18].

En el modelo computacional, el semáforo actúa como el mecanismo mediante el cual el controlador interviene sobre la red. La selección de fases, la duración del verde efectivo y la coordinación entre intersecciones modifican la prioridad de paso y afectan la evolución de las colas y las demoras.

Sobre esta base terminológica, la programación temporal de un semáforo se describe mediante la duración de las fases, la longitud de ciclo y la definición de los intervalos de transición y despeje. Estos parámetros determinan cuánto tiempo recibe prioridad cada conjunto de movimientos compatibles y condicionan directamente la capacidad de descarga, las demoras y la seguridad operacional de la intersección [PW16].

En relación con los giros izquierdos, los manuales de temporización distinguen entre operación permisiva, protegida y protegida-permisiva. En una fase permisiva, el giro se ejecuta aprovechando brechas en la corriente opuesta; en una fase protegida, el giro recibe una indicación exclusiva y los movimientos conflictivos permanecen detenidos; en una fase protegida-permisiva, ambas condiciones se combinan durante distintos intervalos del ciclo [Fed08]. En este análisis, la distinción resulta útil porque muestra que una fase no es solo un color del semáforo, sino una regla operacional que habilita ciertos movimientos, impone cesión de paso en otros y restringe los movimientos en conflicto.

Para garantizar la seguridad vial, las transiciones entre fases no pueden ejecutarse de manera instantánea, sino que deben respetar intervalos de cambio y despeje ($t_{\text{amarillo}} + t_{\text{rojo_todo}}$) [PW16]. En esta investigación, esos tiempos se consideran restricciones operativas de seguridad previamente definidas en la configuración del simulador y del sistema semafórico. En consecuencia, su determinación no forma parte del proceso de decisión del agente, cuyo control se limita a la selección o extensión de fases dentro de un esquema temporal compatible con dichas restricciones.

Modelado de la capacidad y el verde efectivo

Desde el punto de vista operativo, la capacidad de descarga de un acceso semaforizado depende del tiempo efectivo de servicio que recibe cada fase y de la tasa con la que la

fila puede disiparse cuando el movimiento tiene prioridad [FA11]. En la Figura 2.5, $q_{\text{máx}}$ representa la tasa de descarga de saturación, g el intervalo de verde efectivo, y l_1 y l_2 las pérdidas inicial y final de servicio, respectivamente; el tiempo se expresa en segundos y la tasa de flujo en vehículos por hora. La barra inferior distingue los intervalos de rojo, verde y amarillo, mientras que la curva superior muestra cómo la descarga alcanza y abandona el régimen de saturación. La descarga observada no depende solo de la duración nominal del verde, sino también de pérdidas de arranque, transiciones y otras restricciones temporales del control. La figura es conceptual: estos parámetros no se estiman en este trabajo. En su lugar, los efectos se representan mediante la dinámica microscópica del simulador y la configuración temporal del sistema semafórico, ya que esas interacciones determinan las colas, demoras y detenciones que el controlador busca mejorar.

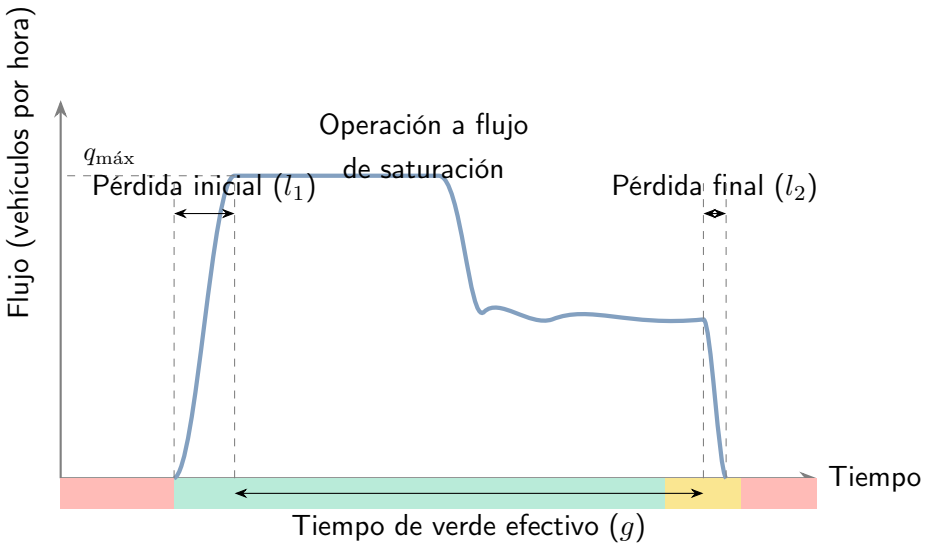


Figura 2.5: Distribución temporal de la tasa de descarga y pérdidas asociadas durante un intervalo de verde en una aproximación semaforizada.

La Figura 2.5 debe leerse como una relación temporal: las pérdidas l_1 y l_2 reducen el intervalo en que la fase puede descargar vehículos a la tasa $q_{\text{máx}}$. Por tanto, el verde efectivo g no equivale necesariamente al verde nominal programado en el controlador.

Clasificación de los controladores de semáforos

De acuerdo con su mecanismo operativo y su grado de respuesta a la demanda vehicular, los controladores de semáforos se clasifican clásicamente en control de tiempo fijo y control accionado por el tránsito [PW16]. En el primero, los planes de señales se repiten de manera preprogramada a partir de datos históricos y no cambian en respuesta a la demanda instantánea. En el segundo, la intersección utiliza detectores en los accesos para variar dinámicamente el inicio o la duración de las fases según la demanda observada en tiempo real.

Incluso en los esquemas accionados, la operación permanece sujeta a parámetros límite

de seguridad y funcionamiento, como el verde mínimo ($g_{\text{mín}}$) y el verde máximo ($g_{\text{máx}}$) [PW16]. Estos parámetros restringen cuánto puede sostenerse o finalizarse una fase y, en este trabajo, forman parte de las condiciones operativas dentro de las cuales el controlador toma decisiones.

2.1.4. Simulación de tránsito

Debido a la densidad y complejidad de las redes urbanas, la sincronización y optimización de los tiempos semafóricos requiere representar la evolución del tránsito bajo condiciones variables. Los modelos analíticos y agregados resultan útiles bajo determinados supuestos, pero pueden ser limitados para describir la formación de colas y las interacciones locales durante la operación dinámica de una red [PW16]. De acuerdo con la taxonomía clásica [FA11], los modelos de tránsito pueden organizarse en tres niveles de resolución. Los modelos macroscópicos describen el tránsito de manera agregada mediante variables como flujo, densidad y velocidad; los mesoscópicos representan entidades intermedias, como agrupamientos vehiculares; y los microscópicos modelan individualmente los vehículos, sus rutas y su evolución dentro de la red. Para esta investigación resulta especialmente relevante este último nivel, pues permite examinar cómo las decisiones semafóricas inciden en la formación de colas, las detenciones y las interacciones locales entre vehículos.

La simulación de tránsito constituye una representación abstracta del fenómeno real. Su objetivo no es reproducir visualmente una escena particular, sino modelar los procesos que determinan la evolución de la circulación mediante reglas de comportamiento y variables observables. La calidad de los resultados depende de la representación de la red, la demanda, la flota, los modelos de comportamiento y las reglas de control. Por ello, la simulación debe entenderse como un entorno experimental sujeto a supuestos y parámetros, cuyos resultados requieren una evaluación operacional acorde con el objetivo del estudio. La correspondencia entre el fenómeno y su representación se resume en la Figura 2.6: el tránsito real constituye la referencia y el simulador representa sus vehículos, red y reglas dinámicas para permitir experimentos controlados.



Figura 2.6: Correspondencia entre la circulación real y su representación microscópica en SUMO. Fuente: DLR – Institute of Transportation Systems, *SUMO User Documentation* [DLR26].

La Figura 2.6 muestra, de forma sintética, la correspondencia entre una escena vial y su representación microscópica en SUMO. Su función es ilustrar cómo los vehículos y la infraestructura se transforman en entidades computacionales; no constituye una validación del simulador ni una representación del escenario utilizado en este trabajo.

SUMO pertenece a la categoría microscópica descrita por Fernández [FA11]. Es una *suite* libre y de código abierto iniciada en el Instituto de Sistemas de Transporte del Centro Aeroespacial Alemán (*Deutsches Zentrum für Luft- und Raumfahrt*) (DLR), que permite modelar sistemas de tránsito intermodales, incluidos vehículos, transporte público y peatones. También ofrece herramientas para importar redes, calcular rutas, visualizar y evaluar simulaciones, además de interfaces para controlar su ejecución [DLR26].

En SUMO, la evolución vehicular se actualiza en pasos temporales discretos sobre una representación espacial continua. El entorno permite modelar calles con múltiples carriles, cambios de carril, distintos tipos de vehículos, rutas individuales, semáforos y salidas de simulación a nivel de vehículo, carril, arco o detector [DLR26].

La interfaz gráfica *sumo-gui* permite inspeccionar visualmente la red, los vehículos y algunos valores de la simulación durante la ejecución. Esta visualización cumple una función de apoyo para la inspección del escenario y la depuración del modelo; las métricas utilizadas para la evaluación se obtienen de las salidas de simulación y de las interfaces de control [DLR26]. La interfaz y su función de inspección se muestran en la Figura 2.7; la Interfaz gráfica de usuario (*Graphical User Interface*) (GUI) no constituye la fuente de las métricas ni reemplaza su cálculo operacional.

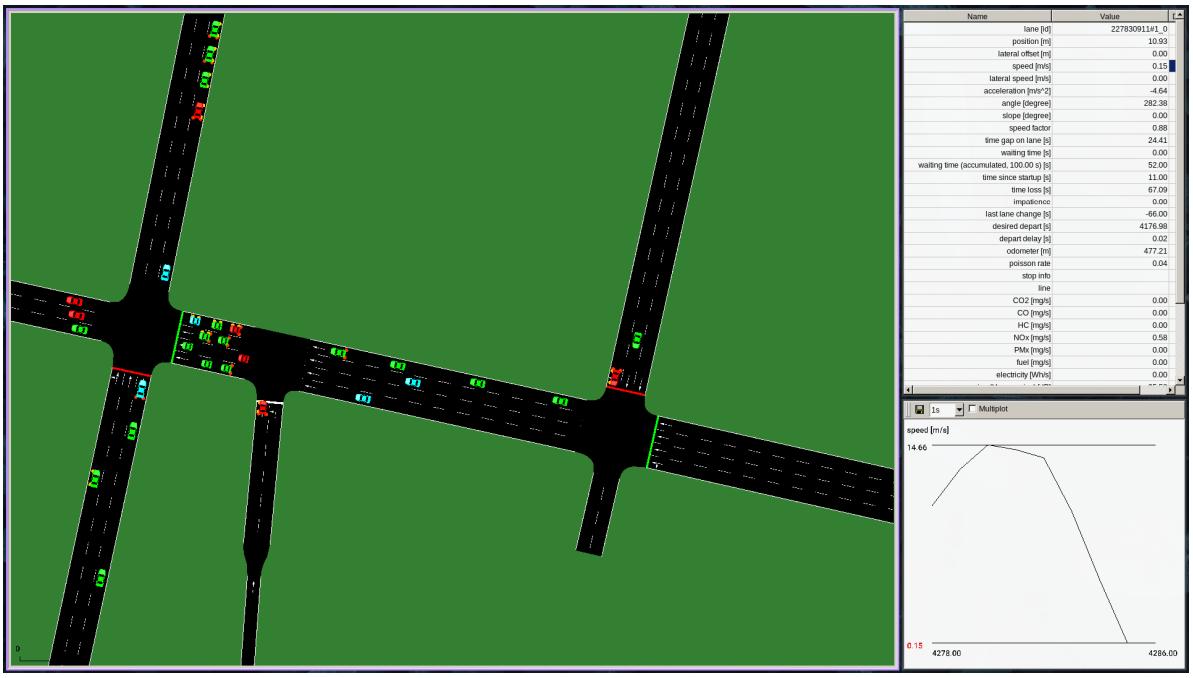


Figura 2.7: Interfaz *sumo-gui* durante una simulación microscópica. El panel izquierdo presenta una vista general de la red; el derecho muestra información sobre un vehículo particular; el recuadro inferior grafica su velocidad y el superior muestra los valores de sus variables relacionadas. Fuente: DLR – Institute of Transportation Systems, *SUMO User Documentation* [DLR26].

La Figura 2.7 reúne cuatro zonas de inspección. A la izquierda se observa la red completa, útil para localizar la parte del escenario que se está examinando. A la derecha se muestra una vista ampliada de la calzada y de un vehículo seleccionado, mientras que el gráfico superpuesto en la parte inferior representa la evolución temporal de su velocidad, expresada en metros por segundo. La franja inferior contiene el registro de mensajes y advertencias producidos durante la ejecución, y la barra superior agrupa los controles de avance, pausa, tiempo y escala de visualización. En conjunto, estos elementos permiten verificar visualmente la presencia de vehículos, su movimiento y posibles inconsistencias de configuración. En este trabajo la interfaz se usa con fines de inspección y depuración; la evaluación cuantitativa se realiza con las salidas de simulación y las interfaces de control [DLR26].

Esta granularidad vuelve a la simulación microscópica especialmente adecuada para estudiar control semafórico mediante aprendizaje por refuerzo. Las decisiones sobre fases y tiempos de servicio pueden evaluarse mediante sus consecuencias sobre la dinámica simulada: colas, tiempos de espera, detenciones, ocupación de carriles, completitud de viajes y propagación de congestión. En la formulación computacional, las variables de tránsito constituyen observaciones, las fases admisibles constituyen acciones y la evolución simulada constituye la transición del entorno. La definición operacional de estas variables y su procedimiento de cálculo se presentan en la metodología.

Representación microscópica de intersecciones en SUMO

La naturaleza microscópica de la simulación no se reduce a representar visualmente vehículos. Cada unidad modifica su velocidad y posición de acuerdo con modelos de seguimiento vehicular y cambio de carril, los parámetros de su tipo de vehículo y las restricciones de la red. De este modo, variables como la aceleración, la separación entre vehículos, la velocidad deseada y la disponibilidad de carriles intervienen en la formación y dispersión de colas. Los valores concretos utilizados para la flota y para estos parámetros pertenecen a la configuración experimental y se detallan en la metodología [DLR26].

En este contexto, el simulador de tránsito constituye el entorno operativo sobre el cual se formaliza el problema de control semafórico. La Figura 2.8 esquematiza la representación microscópica de una intersección con circulación por la derecha [DLR26], en la que se distinguen carriles, líneas de detención y conexiones:

- **Carriles entrantes ($L_{\mathbf{in}}$) y salientes ($L_{\mathbf{out}}$):** los carriles entrantes conducen el flujo vehicular hacia la intersección y los salientes reciben la descarga tras cruzar el nodo, indicados mediante flechas que señalan el sentido de circulación.
- **Línea de detención (*stop line*):** líneas blancas continuas próximas al final de cada carril entrante, delante de las cuales se detienen los vehículos cuando la fase no habilita su movimiento.
- **Conexiones (*links*):** relaciones que conectan un carril entrante con uno saliente a través del área de cruce. En SUMO, cada enlace controlado se asocia con un índice y con un estado de señal (G para verde prioritario, g para verde con cesión de paso, y para amarillo y r para rojo). En la fase conceptual ilustrada, las flechas verdes corresponden a los movimientos directos opuestos habilitados del eje horizontal, mientras que las flechas rojas señalan las conexiones retenidas.

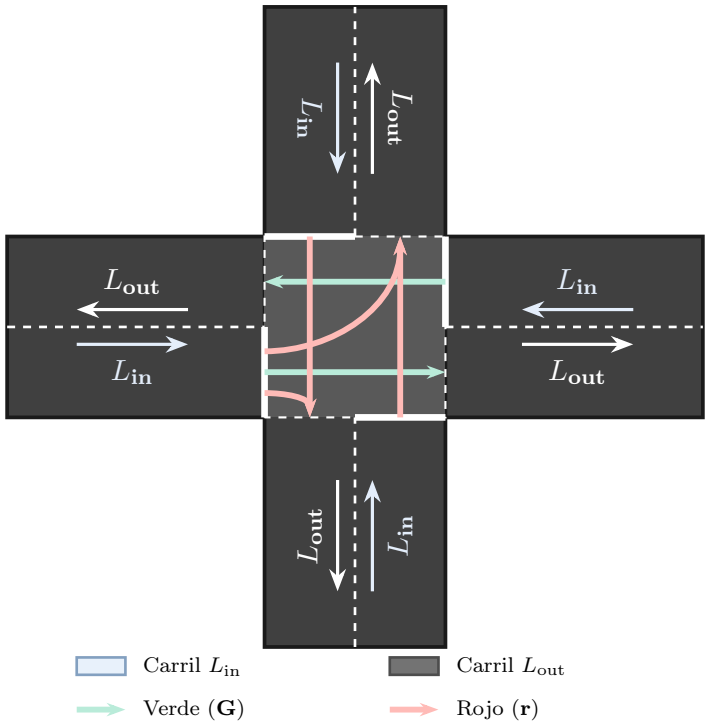


Figura 2.8: Esquema conceptual de carriles, línea de detención y conexiones de una intersección semaforizada bajo circulación por la derecha. La fase representada habilita movimientos directos opuestos del eje horizontal y retiene los movimientos conflictivos.

Por consiguiente, una fase no modifica la geometría ni el sentido de los carriles, sino que determina qué conexiones pueden utilizarse durante cada intervalo de control. Esta representación resulta directamente compatible con la formulación algorítmica del problema, donde una acción válida debe seleccionar una fase compatible con la geometría de la intersección y respetar las transiciones y restricciones de seguridad definidas por el entorno.

Componentes y configuración de SUMO

La *suite* SUMO reúne aplicaciones para construir redes, generar demandas, ejecutar simulaciones, visualizar sus resultados y controlar la ejecución mediante interfaces externas [DLR26]:

- **netconvert:** aplicación que importa redes desde distintas fuentes y compila su representación en el formato nativo de SUMO. También permite reconstruir o ajustar elementos de la red, como las conexiones y los programas semafóricos.
- **netedit:** editor gráfico para crear y modificar de forma interactiva nodos, calles, carriles, conexiones, restricciones de giro y elementos de control semafórico.
- **osmWebWizard:** herramienta que automatiza la generación de un escenario a partir de un área seleccionada de OpenStreetMap (OSM). Se considera una alter-

nativa de importación y no reemplaza el procesamiento específico realizado con `netconvert`.

- **Interfaz de control del tránsito (*Traffic Control Interface*) (TraCI):** interfaz que permite consultar el estado de una simulación en ejecución y modificarlo durante el avance paso a paso, incluyendo variables de vehículos, carriles, detectores y semáforos [DLR26].

- **libsumo:** implementación embebida que ofrece la interfaz de programación de SUMO sin la comunicación mediante conexiones de red (*sockets*) de la modalidad TraCI estándar. Esta alternativa resulta útil para integrar el simulador con controladores externos y reducir la sobrecarga de comunicación.

Desde la perspectiva del modelado computacional, una simulación de SUMO se construye a partir de archivos Lenguaje de marcado extensible (*Extensible Markup Language*) (XML) de red, demanda, elementos adicionales y configuración [DLR26]. Los archivos Zona de análisis de tránsito (*Traffic Analysis Zone*) (TAZ) pueden utilizarse durante la estructuración espacial y la generación de demanda, mientras que las salidas de la simulación se solicitan mediante opciones de configuración específicas. Sus funciones generales se resumen en la Tabla 2.2:

Tabla 2.2: Archivos de entrada y configuración empleados en una simulación de SUMO.

Archivo	**Extensión**	**Función en el Modelo Computacional**
Red Vial	`.net.xml`	Topología física, carriles, conexiones y lógicas semafóricas.
Rutas y Demanda	`.rou.xml`	Tipos de vehículos, viajes, flujos e itinerarios.
Zonas TAZ	`.taz.xml`	Zonas para estructurar espacialmente la demanda.
Elementos Adicionales	`.add.xml`	Detectores, tipos de vehículo y programas auxiliares.
Configuración	`.sumocfg`	Archivos de entrada, opciones de ejecución y salidas.

Métricas de evaluación operacional

La simulación produce medidas para evaluar el control semafórico. En este trabajo se consideran las siguientes categorías de indicadores:

- **Demora y tiempo de espera:** cuantifican el tiempo adicional o el tiempo detenido que experimentan los vehículos durante su recorrido. Pueden analizarse de forma agregada en la red y desagregarse por intersección, acceso o agente, siempre que se mantenga una definición operacional consistente [CyMR18, DLR26].

- **Completitud y flujo servido:** la cantidad de vehículos que completa su itinerario y su relación con la demanda generada permiten distinguir una reducción genuina de la demora de una disminución del flujo atendido.
- **Equidad espacial:** la distribución de las demoras entre intersecciones permite identificar si una política mejora el desempeño global a costa de concentrar la espera en accesos o agentes particulares.
- **Indicadores de validez de la simulación:** el mecanismo *teleport* de SUMO retira temporalmente vehículos bloqueados y los reinserta para evitar que la simulación se detenga. Sus activaciones, los vehículos pendientes y otras incidencias permiten detectar bloqueos o condiciones que comprometen la comparabilidad de una ejecución.

Las definiciones operacionales, fórmulas, unidades, ventanas de evaluación y procedimientos de agregación empleados en este trabajo se presentan en la metodología. Las recompensas y las pérdidas de optimización corresponden al entrenamiento de los algoritmos de aprendizaje y se desarrollan en las secciones dedicadas a RL y MARL.

En este marco, SUMO permite observar el estado de la red y modificar las fases semafóricas durante la ejecución mediante interfaces de control. Esta capacidad convierte el control semafórico en un problema de decisión secuencial, lo que motiva la introducción del aprendizaje por refuerzo en la siguiente sección [DLR26].

2.2. Aprendizaje por refuerzo (RL)

El Aprendizaje automático (*Machine Learning*) (ML) suele dividirse en tres paradigmas. En el Aprendizaje supervisado (*Supervised Learning*) (SL), el modelo se entrena con ejemplos etiquetados que indican la salida esperada. En el Aprendizaje no supervisado (*Unsupervised Learning*) (UL), se busca descubrir estructura en datos no etiquetados, por ejemplo mediante agrupamiento o reducción de dimensionalidad. El Aprendizaje por Refuerzo (*Reinforcement Learning*) (RL) aborda un problema distinto: una entidad aprende mediante interacción con un entorno qué acciones elegir para maximizar una recompensa acumulada [SB18].

Esta diferencia es central. En RL no existe, en general, un supervisor que indique cuál era la acción correcta en cada situación. El agente aprende por experiencia, ensayando acciones, observando sus consecuencias y ajustando su comportamiento a partir de las recompensas obtenidas. Además, las consecuencias relevantes de una acción no siempre se manifiestan de forma inmediata: una decisión puede producir una recompensa baja en el corto plazo, pero conducir a estados favorables en el futuro. Por este motivo, el aprendizaje por refuerzo se ocupa explícitamente del problema de la recompensa diferida y de la asignación de crédito a decisiones pasadas [SB18].

La formulación básica distingue entre el agente (*agent*), que aprende y toma decisiones, y el entorno (*environment*), que representa todo aquello externo al agente con lo cual este interactúa. En cada paso, el agente observa una situación, selecciona una acción y recibe una nueva observación junto con una recompensa: este ciclo constituye la interacción agente–entorno. El comportamiento está determinado por una política (*policy*), denotada por π, que especifica cómo elegir acciones a partir de los estados u observaciones disponibles. La señal de recompensa (*reward signal*) define el objetivo inmediato, mientras que la función de valor estima la conveniencia de estados o acciones en términos de recompensa futura. Sutton y Barto también distinguen el modelo del entorno, entendido como una descripción de sus transiciones y recompensas; cuando no se conoce o no se utiliza explícitamente, se habla de métodos libres de modelo (*model-free*).

En este trabajo, se utiliza controlador para el sistema general que regula las señales y agente para la entidad que aprende y selecciona acciones. Cada agente regula una intersección y la red vial simulada en SUMO constituye su entorno. La selección de una fase constituye la acción, mientras que la evolución posterior del tránsito produce nuevas observaciones y una señal de recompensa que orienta el aprendizaje [DLR26].

Para fijar ideas, considérese un par de intersecciones vecinas. El agente observa localmente las colas y demoras de su intersección, selecciona una fase y modifica la corriente de vehículos que llegará a la siguiente. Esa consecuencia aguas abajo puede cambiar sus arribos y colas, y se refleja en una recompensa asociada al desempeño de la red. El retorno permite acumular esos efectos a lo largo de varias decisiones, en lugar de evaluar una fase solo por su consecuencia inmediata.

Un aspecto característico de RL es el dilema entre exploración y explotación. El agente debe explotar acciones que, según su experiencia, producen buenos resultados, pero también debe explorar alternativas menos conocidas que podrían conducir a políticas superiores. Si explora demasiado, desperdicia oportunidades de obtener recompensa; si explota demasiado pronto, puede converger a comportamientos subóptimos. Este equilibrio es especialmente relevante en problemas de control, donde las acciones modifican la experiencia futura disponible para el aprendizaje [SB18].

2.2.1. Procesos de decisión de Markov (MDP)

El marco matemático estándar para estudiar problemas de aprendizaje por refuerzo de un solo agente es el Proceso de decisión de Markov (*Markov Decision Process*) (MDP). En un MDP finito, el agente y el entorno interactúan en pasos temporales discretos $t = 0, 1, 2, \ldots$: en cada instante, el agente recibe $S_t \in \mathcal{S}$, selecciona $A_t \in \mathcal{A}(S_t)$ y el entorno emite $R_{t+1} \in \mathcal{R}$ y transita a S_{t+1} [LML$^+$25]. La notación se ilustra en la Figura 2.9.

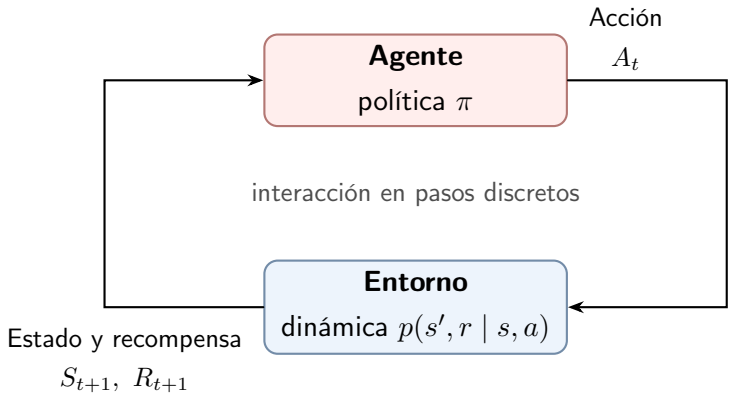


Figura 2.9: Interacción agente-entorno en un proceso de decisión de Markov [SB18].

De forma compacta, un MDP finito queda caracterizado por la tupla $\langle \mathcal{S}, \mathcal{A}, P, R, \gamma \rangle$, donde $\mathcal{S}$ es el espacio de estados, $\mathcal{A}$ el espacio de acciones, P la dinámica de transición probabilística, R la función de recompensa y $\gamma \in [0, 1)$ el factor de descuento.

La dinámica del entorno queda definida por la función $p(s', r \mid s, a)$, que especifica la probabilidad condicional conjunta de observar el próximo estado s' y la recompensa r al ejecutar la acción a en el estado s:

$$p(s', r \mid s, a) \doteq \Pr\{S_{t+1} = s', R_{t+1} = r \mid S_t = s, A_t = a\} \tag{2.5}$$

En esta formulación, $S_t \in \mathcal{S}$ y $A_t \in \mathcal{A}$ denotan el estado y la acción en el instante t, mientras que $S_{t+1} \in \mathcal{S}$ y $R_{t+1} \in \mathbb{R}$ representan el estado sucesor y la recompensa escalar devueltos por el entorno. Si bien conceptualmente el estado comprende toda la información necesaria para determinar la dinámica, en el control semafórico el agente suele disponer de observaciones locales de tránsito que constituyen una representación parcial de dicho estado.

El proceso satisface la propiedad de Markov si la distribución condicional del próximo estado y recompensa depende exclusivamente del estado y la acción presentes, resultando condicionalmente independiente del historial de transiciones previas [SB18].

Una suposición habitual del MDP es que el agente observa un estado suficientemente informativo del entorno. Sin embargo, en muchas aplicaciones reales el agente solo recibe observaciones parciales o ruidosas. En estos casos, el marco se generaliza a un Proceso de decisión de Markov parcialmente observable (*Partially Observable Markov Decision Process*) (POMDP), donde el historial de observaciones puede ser necesario para inferir el estado subyacente [ACS24].

Para formalizar la meta de maximizar la recompensa a largo plazo se define el retorno descontado G_t, que incorpora un factor $\gamma \in [0, 1)$ para ponderar la importancia de las

recompensas futuras [SB18]:

$$G_t \doteq R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \tag{2.6}$$

En esta formulación, el retorno G_t acumula las recompensas futuras ponderadas por el factor de descuento $\gamma \in [0, 1)$. Un valor de γ cercano a cero privilegia la optimización a muy corto plazo, mientras que valores próximos a uno confieren mayor peso a las consecuencias remotas [ACS24]. En el control semafórico, esta acumulación temporal resulta indispensable para que el agente aprenda a prevenir condiciones de bloqueo o saturación en ciclos futuros, en lugar de limitarse a disipar la cola inmediata a expensas de la red.

2.2.2. Métodos basados en valor y basados en política

Los algoritmos de RL pueden agruparse, a grandes rasgos, en métodos basados en valor y métodos basados en política. Esta distinción no agota la taxonomía del campo, pero permite introducir las familias de algoritmos más relevantes para este trabajo.

Esta clasificación distingue dos formas de obtener una regla de decisión. Los métodos basados en política aprenden directamente una función que relaciona cada estado con una acción o con una distribución de probabilidad sobre las acciones. En cambio, los métodos basados en valor aprenden indirectamente la regla de decisión: estiman la conveniencia de los estados o de los pares estado–acción y, a partir de esas estimaciones, determinan qué alternativa conviene ejecutar [SB18]. En el control semafórico, las alternativas corresponden a las distintas fases admisibles de una intersección.

La Figura 2.10 representa esta segunda alternativa. A partir del estado observado, la función de valor estima la conveniencia esperada de las acciones disponibles y la decisión se obtiene comparando esas estimaciones. El esquema corresponde directamente al caso de una función de valor de acción $Q(s, a)$; si se utiliza una función de valor de estado $V(s)$, la elección de la acción requiere valorar los estados sucesores que produciría cada alternativa. Por ello, $V(s)$ y $Q(s, a)$ cumplen funciones relacionadas, pero no intercambiables en la evaluación de los estados y en la selección de acciones.

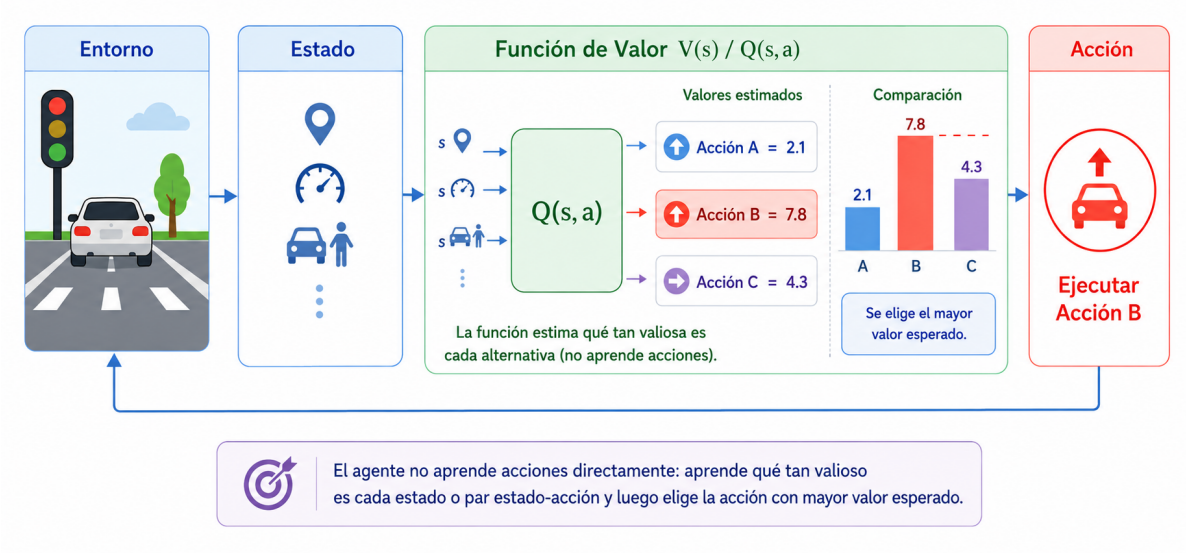


Figura 2.10: Esquema de un método basado en valor. A partir del estado observado s, la función de valor estima el retorno esperado de los pares estado–acción $Q(s, a)$. Las estimaciones para las acciones disponibles se comparan y se selecciona la alternativa con mayor valor esperado para ejecutarla en el entorno. La exploración puede incorporarse mediante estrategias como la selección ε-greedy [SB18].

En los métodos basados en valor, el objetivo es determinar la conveniencia de encontrarse en un estado o de ejecutar una acción concreta. Formalmente, esto se expresa mediante la función de valor de estado $v_\pi(s) \doteq \mathbb{E}_\pi[G_t \mid S_t = s]$ y la función de valor de acción $q_\pi(s, a) \doteq \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$, las cuales satisfacen las relaciones recursivas de las ecuaciones de Bellman [SB18]. En la notación empleada en la Figura 2.10, estas funciones se escriben habitualmente como $V(s)$ y $Q(s, a)$ cuando no es necesario hacer explícita la política respecto de la cual se calcula el valor.

Cuando el propósito es identificar la política óptima en lugar de evaluar una fija, se emplean las ecuaciones de optimalidad de Bellman:

$$q_*(s, a) = \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') \mid S_t = s, A_t = a\right]. \tag{2.7}$$

En la ecuación (2.7), la esperanza $\mathbb{E}$ promedia las transiciones estocásticas del entorno y la maximización sobre las fases admisibles a' formaliza el principio de optimalidad. En el ámbito semafórico, esta relación expresa el compromiso entre la recompensa inmediata obtenida por la fase activa y la calidad del estado vehicular resultante para las decisiones subsiguientes [SB18].

En ausencia de un modelo analítico de transición, los métodos de TD aproximan estas funciones a partir de la experiencia mediante *bootstrapping*: utilizan una estimación del valor del estado siguiente para actualizar el valor actual. Un ejemplo es Q-learning, cuya regla de actualización converge hacia la función óptima $q_*(s, a)$ en el caso tabular, bajo el supuesto de que todos los pares estado–acción continúen explorándose y que la tasa de

aprendizaje decrezca adecuadamente a lo largo del tiempo [SB18]:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right] \tag{2.8}$$

donde $\alpha \in (0, 1]$ es la tasa de aprendizaje (*learning rate*) que pondera la magnitud del ajuste, R_{t+1} es la recompensa inmediata observada, y el término $\max_a Q(S_{t+1}, a)$ aproxima el retorno óptimo alcanzable desde el estado sucesor S_{t+1}.

Q-learning es un algoritmo *off-policy*: la política que genera los datos de interacción puede diferir de la política objetivo cuya función de valor se optimiza. Por el contrario, los métodos de gradiente de política operan generalmente de forma *on-policy*, utilizando transiciones producidas por la política en curso de optimización. Esta diferencia resulta decisiva al extender los algoritmos al entorno multiagente, donde las políticas de las intersecciones vecinas evolucionan simultáneamente.

En los métodos basados en política, la regla de decisión se parametriza directamente como $\pi_\theta(a \mid s) = \Pr\{A_t = a \mid S_t = s, \theta\}$, donde $\theta \in \mathbb{R}^d$ representa el vector de parámetros ajustables de la política. La política puede ser determinista, si asigna siempre la misma acción a un estado, o estocástica, si produce una distribución de probabilidad sobre las acciones posibles [SB18]. La Figura 2.11 ilustra este último caso: para el estado observado, la política asigna probabilidades a cada fase admisible y la selección se efectúa mediante muestreo o según la alternativa más probable, de acuerdo con la regla de ejecución adoptada.

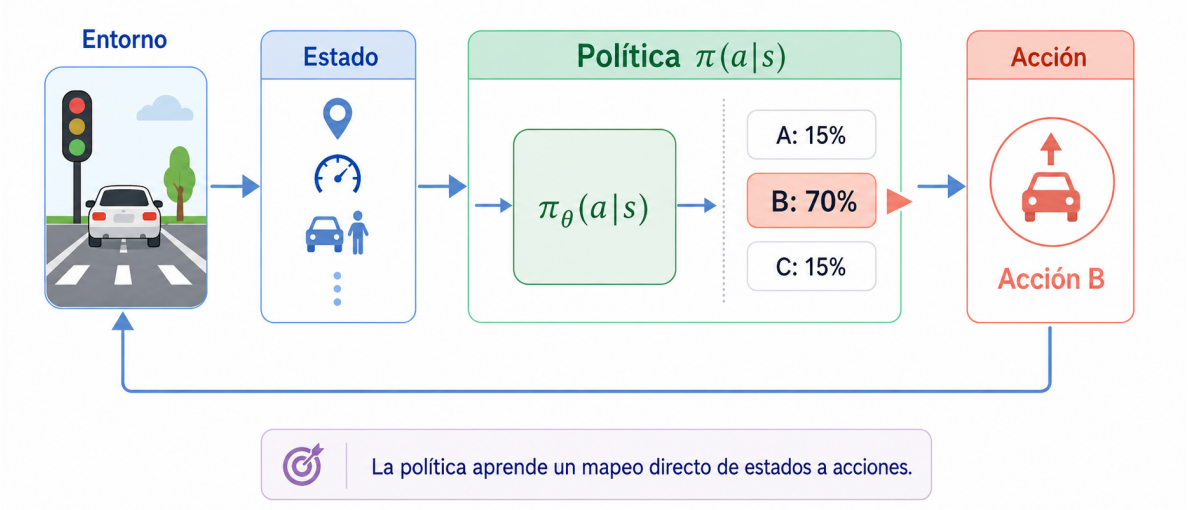


Figura 2.11: Esquema de un método basado en política. A partir del estado observado s, la política parametrizada $\pi_\theta(a \mid s)$ genera directamente una distribución de probabilidad sobre las acciones disponibles. La regla de ejecución puede seleccionar la alternativa de mayor probabilidad o muestrear una acción a partir de dicha distribución [SB18].

La optimización persigue maximizar el rendimiento esperado $J(\theta)$ mediante ascenso

de gradiente:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta_t) \tag{2.9}$$

donde $\nabla J(\theta_t)$ es el gradiente del rendimiento respecto de los parámetros y $\alpha > 0$ determina la magnitud del paso de actualización. El Teorema del Gradiente de Política (*Policy Gradient Theorem*) proporciona el sustento teórico para estimar analíticamente $\nabla J(\theta_t)$ a partir de trayectorias muestreadas, ajustando la política para elevar la probabilidad de aquellas fases que conducen a menores demoras en la red [SB18].

Los métodos actor-crítico combinan una política parametrizada con una estimación de valor y actualizan ambos componentes de manera concurrente [KT03, SB18]. El actor representa la política $\pi_\theta(a \mid s)$ y determina la distribución utilizada para seleccionar la acción. El crítico no controla el entorno: estima, por ejemplo, el valor de estado $V_\phi(s)$ y produce una señal que indica si el resultado observado fue mejor o peor que lo esperado bajo la política vigente. Los parámetros θ y ϕ pueden optimizarse de forma independiente o compartir una representación común de las observaciones de tránsito.

En una formulación actor-crítico de un paso, después de ejecutar A_t en S_t, el entorno devuelve la recompensa R_{t+1}, el estado S_{t+1} y el indicador terminal d_{t+1}. El crítico calcula el error de diferencia temporal mediante:

$$\delta_t = R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t), \tag{2.10}$$

En la ecuación (2.10), el factor $(1 - d_{t+1})$ anula la contribución del estado siguiente ante transiciones terminales. Cuando V_ϕ aproxima adecuadamente el valor real V^π, δ_t actúa como un estimador de un paso de la ventaja: un valor positivo refleja que la fase ejecutada reportó un desempeño superior al esperado en ese estado de tránsito, mientras que un valor negativo indica lo contrario. El crítico ajusta sus parámetros ϕ minimizando el error temporal, mientras que el actor utiliza esta misma señal δ_t para modular la actualización de la política:

$$\theta \leftarrow \theta + \alpha_\theta \, \delta_t \, \nabla_\theta \log \pi_\theta(A_t \mid S_t). \tag{2.11}$$

En la regla (2.11), $\alpha_\theta > 0$ es la tasa de aprendizaje del actor y $\nabla_\theta \log \pi_\theta(A_t \mid S_t)$ indica cómo cambiar sus parámetros para aumentar la probabilidad de la acción elegida. Un error δ_t positivo favorece esa acción, mientras que uno negativo reduce su probabilidad. Los métodos posteriores emplean estimadores de ventaja de varios pasos, como Estimación generalizada de ventaja (*Generalized Advantage Estimation*) (GAE), y procesan conjuntos de transiciones llamados lote (*batch*). Esta regla permite comprender cómo el crítico orienta la actualización del actor.

La Figura 2.12 sintetiza el flujo operativo entre actor y entorno, así como el circuito de evaluación mediante el cual el crítico genera la señal de retroalimentación compartida.

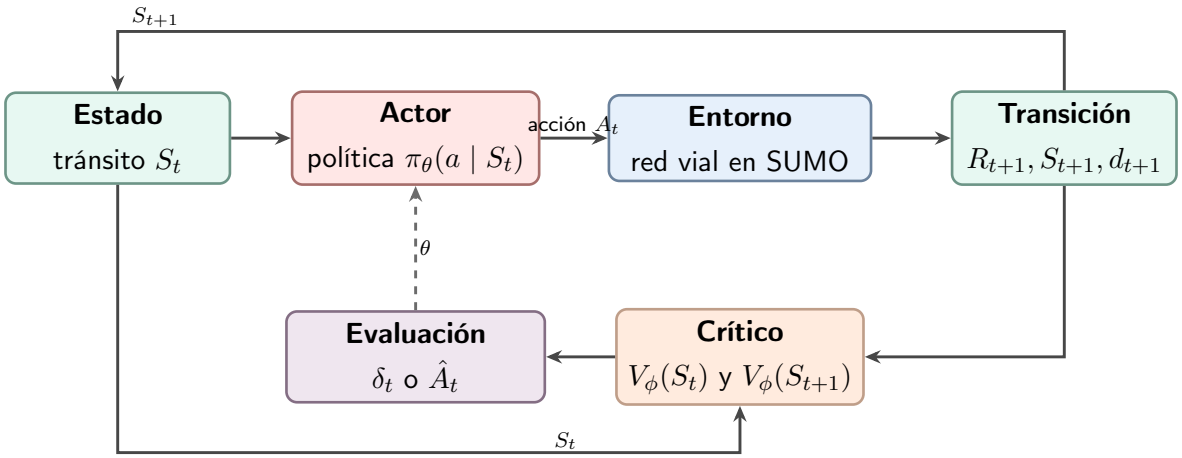


Figura 2.12: Arquitectura actor-crítico aplicada al control semafórico en la formulación de un MDP. El actor selecciona una acción y el crítico evalúa la transición para construir la señal que actualiza ambos componentes. Elaboración propia a partir de [KT03, SB18].

En síntesis, el estado resume las condiciones de tránsito disponibles para el agente, la acción corresponde a la selección de una fase, la recompensa refleja la consecuencia operativa inmediata y el valor estima su impacto a largo plazo. Cuando el espacio de estados es continuo o de gran escala, como ocurre en las redes viales urbanas, las funciones de valor y las políticas requieren aproximadores de alta capacidad representacional. A continuación se introducen las redes neuronales artificiales como el modelo adoptado para este fin, preservando intactos los fundamentos de RL expuestos hasta aquí.

2.2.3. Aprendizaje profundo y redes neuronales

Para estructurar este desarrollo, se expone en primer término la unidad neuronal básica y su composición en redes multicapa; luego se aborda su optimización mediante retropropagación y descenso de gradiente; finalmente, se analiza su integración en RL a través de DQN y PPO. Cabe destacar que la red neuronal actúa como un aproximador de funciones de alta capacidad, mientras que la dirección del aprendizaje proviene exclusivamente de las señales de recompensa generadas por la interacción con el entorno [SB18].

El aprendizaje profundo (*Deep Learning*) se enfoca en el entrenamiento de arquitecturas con múltiples capas de parámetros ajustables capaces de extraer representaciones jerárquicas y relaciones no lineales directamente de los datos [BB24, GBC16]. En este trabajo, esta capacidad se aprovecha para aproximar funciones de valor y políticas de control semafórico donde una formulación tabular resulta impracticable debido a la continuidad y dimensionalidad del estado del tránsito.

Aunque la denominación "neuronal" proviene de una analogía simplificada con modelos biológicos iniciales, una red artificial constituye un modelo matemático determinista cuyos parámetros se optimizan a partir de datos y gradientes de pérdida [GBC16].

La unidad neuronal y redes *feedforward*

En problemas de control con espacios de estados continuos o de gran escala, los métodos tabulares resultan impracticables [SB18]. La aproximación de funciones resuelve esta limitación mediante representaciones parametrizadas, lo que permite generalizar lo aprendido a situaciones similares no observadas. Para este fin se emplean redes neuronales artificiales [BB24].

La unidad neuronal artificial constituye el bloque computacional básico de estas arquitecturas [BB24, GBC16]. Esta procesa un vector de entradas ponderándolo mediante un vector de pesos, añade un término de sesgo y aplica una función de activación no lineal para producir una respuesta escalar. En el ámbito semafórico, las entradas corresponden típicamente a variables observables del tránsito (como tiempos de espera acumulados o longitudes de cola por carril), mientras que los pesos modulan la contribución relativa de cada acceso hacia la decisión final.

La Figura 2.13 ilustra este procesamiento: la transformación afín $z = \mathbf{w}^\top \mathbf{x} + b$ integra las entradas $\mathbf{x} = (x_1, \ldots, x_n)$ con los pesos $\mathbf{w}$ y el sesgo b, antes de que la activación $y = g(z)$ genere la salida. Sin la introducción de estas no linealidades $g(\cdot)$, una red multicapa se reduciría a una transformación afín simple, imposibilitando la captura de patrones de congestión complejos.

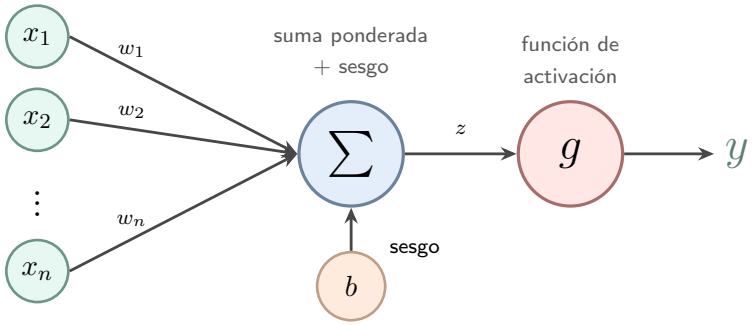


Figura 2.13: Operaciones de una unidad neuronal artificial: suma ponderada de las entradas, incorporación del sesgo y aplicación de una función de activación.

Como ejemplo conceptual, la red puede recibir las métricas de demora de una intersección y emitir estimaciones de calidad o probabilidades sobre las fases admisibles, orientando la asignación del verde hacia las aproximaciones más críticas.

En una red multicapa, la operación para una unidad u dentro de la capa k se formaliza mediante la ecuación (2.12) [BB24, GBC16]:

$$a_{k,u} = \mathbf{w}_{k,u}^\top \mathbf{x}_{k-1} + b_{k,u}, \qquad y_{k,u} = g_k(a_{k,u}) \tag{2.12}$$

donde $\mathbf{x}_{k-1}$ es el vector de activaciones proveniente de la capa anterior, $\mathbf{w}_{k,u}$ es el vector de pesos, $b_{k,u}$ es el sesgo y g_k es la función de activación. Entre las funciones más empleadas en capas intermedias destacan:

- **Unidad lineal rectificada (*Rectified Linear Unit*) (ReLU):** $g(z) = \max\{0, z\}$, que propaga linealmente las activaciones positivas y bloquea las negativas, con pendiente uno para valores positivos durante la optimización.
- **Leaky ReLU:** $g(z) = \max\{0, z\} + \alpha \min\{0, z\}$, con $\alpha \ll 1$, que conserva una pendiente pequeña en el semieje negativo para mitigar la inactivación de unidades.
- **Tanh:** $g(z) = \tanh(z)$, que ofrece una respuesta no lineal diferenciable acotada en $(-1, 1)$.

Las redes *feedforward* encadenan estas transformaciones en capas sucesivas sin conexiones recurrentes sin retroalimentación temporal. Para una arquitectura con L capas parametrizadas, la propagación completa se describe como:

$$\mathbf{h}_k = g_k\left(\mathbf{W}_k^\top \mathbf{h}_{k-1} + \mathbf{b}_k\right), \qquad k = 1, \ldots, L, \qquad \mathbf{h}_0 = \mathbf{x}, \qquad \mathbf{y} = \mathbf{h}_L \tag{2.13}$$

donde $\mathbf{W}_k$ y $\mathbf{b}_k$ denotan la matriz de pesos y el vector de sesgos de la capa k, respectivamente, y g_k se aplica componente a componente. En el agente semafórico, el vector de entrada $\mathbf{h}_0 = \mathbf{x}$ contiene las esperas acumuladas por carril y la salida $\mathbf{y} = \mathbf{h}_L$ produce las valoraciones Q o probabilidades de selección de fase.

La Figura 2.14 esquematiza esta composición con dos capas ocultas conectadas en cascada hacia la salida.

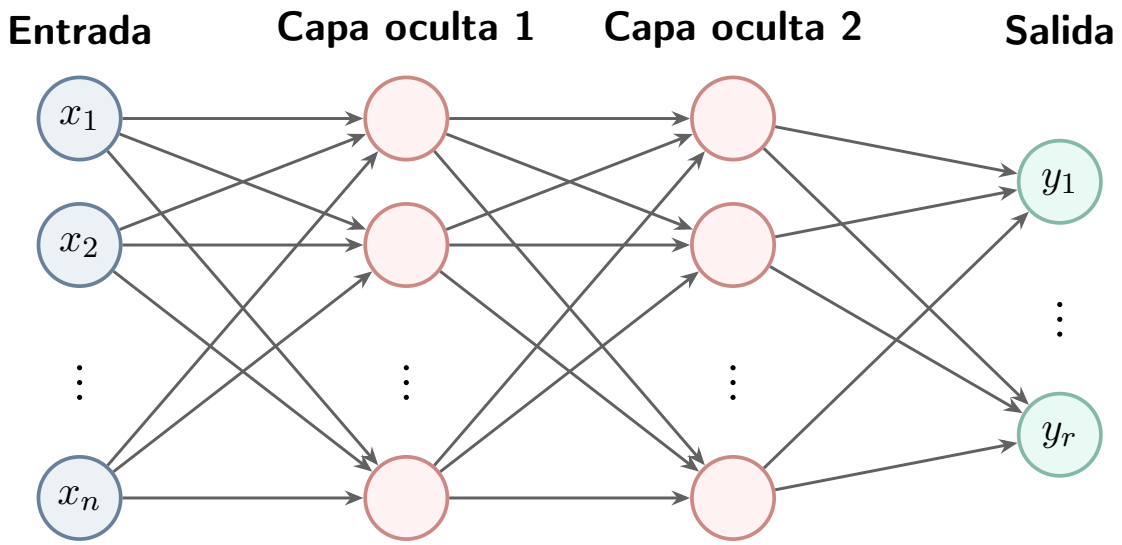


Figura 2.14: Arquitectura conceptual de una red neuronal *feedforward* con dos capas ocultas.

El Teorema de Aproximación Universal es un resultado de existencia: bajo condiciones apropiadas, una red de dos capas parametrizadas, con una capa oculta suficientemente ancha, puede aproximar una función continua sobre un dominio compacto con una precisión arbitraria. El teorema no proporciona por sí mismo un procedimiento para encontrar

los parámetros, no garantiza un entrenamiento eficiente, una buena generalización ni un buen desempeño práctico sobre datos no observados. La profundidad puede ofrecer representaciones jerárquicas y una mayor eficiencia expresiva para ciertas familias de funciones, pero agregar capas o neuronas no garantiza por sí solo un menor error de prueba [BB24].

El proceso de optimización

Durante el entrenamiento, la red procesa las observaciones mediante propagación hacia adelante y contrasta su respuesta frente a un objetivo mediante una función de pérdida escalar diferenciable $\mathcal{L}(\theta)$. El algoritmo de retropropagación (*backpropagation*) calcula analíticamente las derivadas parciales de dicha pérdida respecto de cada peso y sesgo aplicando de forma sistemática la regla de la cadena [BB24].

Con estos gradientes, los parámetros se actualizan iterativamente en sentido opuesto a la pendiente del error:

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L}(\theta_t), \tag{2.14}$$

donde θ reúne los parámetros entrenables de la red, $\nabla_\theta \mathcal{L}(\theta_t)$ es el gradiente de la función de pérdida y $\eta > 0$ es la tasa de aprendizaje. Calcular el gradiente con todos los datos puede resultar costoso. Por ello, se utilizan subconjuntos de muestras, denominados *minibatches*, para realizar cada actualización. Adam adapta el tamaño del paso para cada parámetro a partir de promedios móviles del gradiente y de su cuadrado [BB24].

A diferencia del aprendizaje supervisado convencional —donde la pérdida compara la salida con una etiqueta externa fija—, en RL el objetivo se deriva de las recompensas y transiciones producidas por la interacción con el simulador, y cambia a medida que la política mejora su comportamiento.

2.2.4. Aprendizaje por refuerzo profundo (DRL)

El DRL combina algoritmos de RL con aproximadores neuronales. En el control semafórico, las observaciones del tránsito, como colas, ocupación o velocidades, pueden alimentar la red; sus salidas pueden representar valores de acción o parámetros de una política; y las recompensas asociadas con demoras, detenciones y colas orientan la función de pérdida del algoritmo de aprendizaje. Esta integración resulta útil cuando el espacio de estados es continuo o de alta dimensionalidad, aunque exige mecanismos de estabilización y una evaluación cuidadosa de la eficiencia de las muestras [ACS24].

Factores de inestabilidad del aprendizaje

La integración de aproximadores no lineales con aprendizaje por refuerzo puede inducir inestabilidad numérica o divergencia en las estimaciones si no se controlan las condiciones de actualización. Sutton y Barto [SB18] describen este fenómeno mediante la denominada tríada mortal (*deadly triad*), originada por la concurrencia de tres elementos:

- **Aproximación de funciones:** el uso de un modelo parametrizado, como una red

neuronal, para estimar valores en lugar de almacenarlos individualmente en una tabla.

- ***Bootstrapping*:** la actualización de estimaciones de valor apoyándose en otras estimaciones aprendidas previamente (métodos TD), sin esperar a disponer del retorno completo.

- **Entrenamiento *off-policy*:** el aprendizaje sustentado en una distribución de datos generada por una política de comportamiento distinta de la política objetivo.

Aunque prescindir del *bootstrapping* o de la aproximación de funciones comprometería la eficiencia muestral o la escalabilidad ante redes viales complejas, los algoritmos profundos modernos introducen mecanismos arquitectónicos específicos para amortiguar esta inestabilidad [SB18].

Deep Q-Network (DQN): aproximación neuronal de valores de acción

En los métodos basados en valor, DQN sustituye la tabla de estados y acciones por una red neuronal $Q(s, a; \theta)$ parametrizada por θ [MKS+15]. Ante una observación de tránsito, la red genera una estimación del valor Q para cada fase admisible. La función de acción-valor óptima $q_*(s, a)$ representa el máximo retorno esperado al seleccionar la mejor secuencia de decisiones futuras, objetivo que DQN persigue aproximar a partir de las recompensas del entorno.

Para mitigar la inestabilidad de la tríada mortal —en particular el problema del valor objetivo cambiante, donde actualizar el valor de un estado altera impredeciblemente las estimaciones de estados adyacentes—, DQN incorpora dos componentes estructurales [MKS+15]:

- **Redes objetivo (*Target Networks*):** una red secundaria idéntica cuyos parámetros θ^- se congelan temporalmente. Esta red se utiliza exclusivamente para calcular el término de *bootstrapping* del valor objetivo TD, desacoplando la estimación en curso del objetivo de aprendizaje y amortiguando oscilaciones.

- **Memoria de repetición de experiencias (*replay buffer*):** almacena transiciones históricas $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$ en un *buffer* $\mathcal{D}$. La optimización extrae *minibatches* aleatorios de esta memoria, lo cual mitiga la correlación temporal propia del flujo continuo de vehículos en el simulador y aproxima los datos a la hipótesis de muestras independientes.

Durante la interacción con SUMO, la red en línea evalúa la observación de tránsito y una regla ϵ_{exp}-greedy selecciona la fase a ejecutar; la transición resultante se almacena en $\mathcal{D}$. Durante el entrenamiento, la red en línea actualiza sus pesos θ a partir de los *minibatches* muestreados del *buffer* contra el valor objetivo fijado por la red objetivo, cuyos parámetros θ^- se sincronizan solo de forma periódica (Figura 2.15).

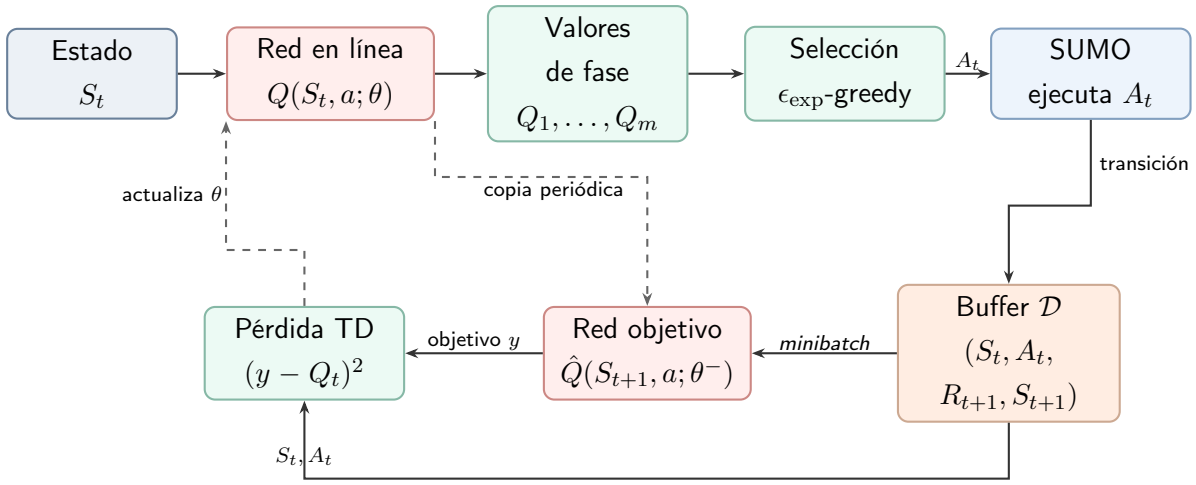


Figura 2.15: Flujo conceptual de DQN con repetición de experiencias y red objetivo. La red en línea selecciona acciones y recibe las actualizaciones; la red objetivo proporciona el término de *bootstrapping* del valor objetivo TD. Elaboración propia a partir del algoritmo descrito por Mnih et al. [MKS+15].

La función de pérdida minimizada por DQN, representada en el bloque inferior de la Figura 2.15, es la de la ecuación (2.15):

$$\mathcal{L}(\theta) = \mathbb{E}_{(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1}) \sim \mathcal{D}} \left[\left(\underbrace{r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)}_{\text{Valor objetivo de TD}} - \underbrace{Q(s_j, a_j; \theta)}_{\text{Predicción actual}} \right)^2 \right] \tag{2.15}$$

En la ecuación (2.15), la esperanza se evalúa empíricamente promediando sobre un *mini-batch* de transiciones $(s_j, a_j, r_{j+1}, s_{j+1}, d_{j+1})$ extraídas de $\mathcal{D}$. La función de pérdida minimiza la discrepancia cuadrática entre la estimación actual $Q(s_j, a_j; \theta)$ y el valor objetivo temporal compuesto por la recompensa inmediata observada y la mayor estimación del valor futuro sobre las fases admisibles, calculada por la red objetivo $\hat{Q}(s_{j+1}, a'; \theta^-)$. El factor $(1 - d_{j+1})$ anula la contribución del estado siguiente cuando la transición es terminal $(d_{j+1} = 1)$, mientras que a' recorre las fases semafóricas admisibles.

Los cuatro bloques numerados como algoritmos en este capítulo son pseudocódigos: descripciones ordenadas e independientes de un lenguaje de programación, no fragmentos ejecutables. En ellos, una transición registra lo ocurrido en un paso; un episodio agrupa pasos desde el reinicio hasta la terminación; y un *rollout* es la secuencia de transiciones recolectada con una política. Una iteración de entrenamiento comprende la recolección de datos y su optimización, mientras que una *epoch* es una reutilización completa de esos datos dentro de la misma iteración. Cada actualización puede calcularse sobre un *mini-batch*, es decir, un subconjunto de las muestras. Las líneas "Entrada" y "Salida" indican, respectivamente, qué información necesita el procedimiento y qué componentes entrena-

dos produce; las líneas indentadas pertenecen al ciclo inmediatamente superior. En lo sucesivo se utiliza lote para *batch* y se conservan los términos *minibatch*, *buffer* y *epoch*, definidos aquí y en el glosario.

El procedimiento de DQN combina la recolección de experiencias, la selección de fases con política ϵ_{exp}-greedy y la optimización de la red neuronal a partir de *minibatches* muestreados del *replay buffer*. El algoritmo 1 especifica este procedimiento:

Algoritmo 1 Deep Q-Network con repetición de experiencias

Entrada: entorno, fases admisibles, capacidad N, horizonte T en pasos de decisión, γ y ϵ_{exp}

Salida: parámetros entrenados θ de la red de acción-valor

Inicializar el *buffer* de experiencias $\mathcal{D}$ con capacidad N
Inicializar la red de acción-valor $Q(s, a; \theta)$ con parámetros aleatorios
Inicializar la red objetivo $\hat{Q}(s, a; \theta^-)$ con $\theta^- \leftarrow \theta$
para episodio $= 1, 2, \ldots$ **hacer**
    Inicializar el estado s_t
    para $t = 1, 2, \ldots, T$ **hacer**
        Seleccionar a_t mediante una política ϵ_{exp}-greedy basada en $Q(s_t, a; \theta)$
        Ejecutar a_t y observar la recompensa r_{t+1}, el estado s_{t+1} y si el episodio terminó
        Almacenar $(s_t, a_t, r_{t+1}, s_{t+1}, d_{t+1})$ en $\mathcal{D}$
        si $\mathcal{D}$ contiene al menos un *minibatch* completo **entonces**
            Extraer un *minibatch* de transiciones de $\mathcal{D}$
            Calcular $y_j = r_{j+1} + \gamma(1 - d_{j+1}) \max_{a'} \hat{Q}(s_{j+1}, a'; \theta^-)$
            Actualizar θ minimizando $(y_j - Q(s_j, a_j; \theta))^2$
        fin si
        Actualizar periódicamente la red objetivo: $\theta^- \leftarrow \theta$
        $s_t \leftarrow s_{t+1}$
        si $d_{t+1} = 1$ **entonces**
            Terminar el ciclo interno y comenzar un nuevo episodio
        fin si
    fin para
fin para

En esta formulación, el agente desacopla la recolección de experiencias en el simulador de la optimización paramétrica de la red. La memoria $\mathcal{D}$ reduce la correlación temporal de las observaciones de tráfico, mientras que la red objetivo estabiliza el valor objetivo de Bellman mediante la sincronización periódica de θ^-. La especificación detallada de los hiperparámetros operacionales (capacidad N, horizonte T, tasa de aprendizaje y cronograma de exploración) se desarrolla en el capítulo 3.

Aproximación de políticas, función de ventaja y *Policy Gradient*

Como alternativa a derivar la política de forma indirecta a partir de una función de valor (como en DQN), DRL permite parametrizar directamente la política del agente mediante una red neuronal $\pi(a|s;\theta)$ [ACS24]. Bajo la formulación MDP, en un problema de acciones discretas la red recibe el estado s, calcula un vector de puntajes continuos $z \in \mathbb{R}^m$ y los proyecta a una distribución de probabilidad mediante la función *softmax* [SB18, BB24]:

$$\pi_\theta(a_i \mid s) = \frac{e^{z_i}}{\sum_{j=1}^{m} e^{z_j}} \tag{2.16}$$

la cual garantiza valores estrictamente positivos que suman la unidad. En este ejemplo, las acciones corresponden a las fases semafóricas admisibles: las variables del estado de tránsito ingresan a la red y sus salidas representan probabilidades sobre dicho conjunto de acciones. Las políticas basadas en observaciones locales se formalizan posteriormente mediante Dec-POMDP. La acción puede muestrearse de esa distribución durante el entrenamiento, preservando la exploración estocástica.

La Figura 2.16 muestra este recorrido para un agente semafórico. A diferencia de DQN, las salidas no son estimaciones de valor: son probabilidades que suman uno. La señal de ventaja no entra a la red para seleccionar la acción; se utiliza durante el entrenamiento para modificar los parámetros en la dirección indicada por el gradiente de la log-probabilidad de la acción observada.

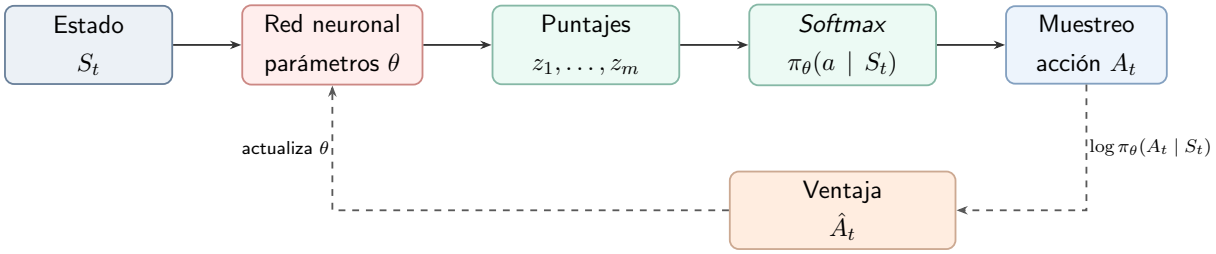


Figura 2.16: Red de política para un espacio discreto de acciones en la formulación de un MDP. El estado determina una distribución sobre las acciones disponibles; durante el aprendizaje, la ventaja pondera el gradiente de la log-probabilidad de la acción seleccionada [SB18, KT03].

La optimización de políticas se fundamenta en aislar la contribución específica de una acción respecto del rendimiento promedio del estado. Formalmente, la función de ventaja (*Advantage Function*) se define como la diferencia entre el valor de la acción y el valor medio del estado [SB18]:

$$A^\pi(s,a) \doteq Q^\pi(s,a) - V^\pi(s) \tag{2.17}$$

donde $Q^\pi(s,a)$ es el retorno esperado al seleccionar la acción a en el estado s bajo la política π, y $V^\pi(s) = \mathbb{E}_{a\sim\pi}[Q^\pi(s,a)]$ representa el valor esperado medio en dicho estado. Un valor $A^\pi(s,a) > 0$ señala que la fase semafórica ejecutada supera el desempeño promedio

de la intersección ante esas condiciones de cola, mientras que $A^\pi(s,a) < 0$ indica una decisión comparativamente deficiente.

Para estimar la ventaja combinando errores temporales de distintos horizontes, Schulman et al. [SML+16] desarrollaron el estimador GAE (*Generalized Advantage Estimation*):

$$\hat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} \doteq \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l}^V \tag{2.18}$$

donde $\delta_t^V = R_{t+1} + \gamma(1 - d_{t+1})V_\phi(S_{t+1}) - V_\phi(S_t)$ es el error temporal del crítico parametrizado por ϕ, d_{t+1} es el indicador terminal, $\gamma \in [0, 1)$ es el factor de descuento y $\lambda \in [0, 1]$ es el parámetro de suavizado exponencial. El hiperparámetro λ interpola estimaciones de ventaja de distintos horizontes. Para $\lambda = 0$, se obtiene el residuo TD de un paso δ_t^V, que suele reducir la varianza a costa de depender de la precisión del crítico y, por tanto, puede introducir sesgo. Para $\lambda = 1$, con el retorno completo, se recupera el estimador Monte Carlo descontado menos el valor del estado; este suele presentar mayor varianza. En trayectorias truncadas, el *bootstrap* y la aproximación del crítico pueden modificar este compromiso.

El Teorema del Gradiente de Política (*Policy Gradient Theorem*) [SB18] fundamenta analíticamente la optimización directa de los parámetros de la política maximizando el retorno esperado $J(\theta)$. Empleando la función de ventaja como línea de base para mitigar la varianza, el gradiente se aproxima mediante la esperanza muestral sobre un lote de transiciones recolectadas:

$$\nabla_\theta J(\theta) \approx \hat{\mathbb{E}}_t \left[\nabla_\theta \log \pi_\theta(A_t \mid S_t) \hat{A}_t \right] \tag{2.19}$$

donde $\pi_\theta(A_t \mid S_t)$ denota la probabilidad asignada a la fase A_t bajo los parámetros θ, y $\hat{A}_t$ es la ventaja estimada mediante GAE. Como esquematiza la Figura 2.16, la dirección del vector gradiente escala proporcionalmente a $\hat{A}_t$, incrementando la probabilidad de ocurrencia de aquellas fases que superan el desempeño esperado y desincentivando aquellas que provocan demoras excesivas.

Proximal Policy Optimization (PPO) y control de las actualizaciones

Una actualización demasiado grande de los parámetros puede reducir el rendimiento de la política. Los métodos que se describen a continuación buscan limitar ese cambio para conservar un aprendizaje estable.

Para acotar el tamaño de paso en el espacio de distribuciones de probabilidad, Kakade y Langford [KL02] y Schulman et al. (Optimización de política por región de confianza (*Trust Region Policy Optimization*) (TRPO) [SLA+15]) formularon métodos de región de confianza. En el análisis teórico de TRPO, bajo los supuestos del modelo y una actualización que respete la región de confianza basada en la divergencia de Kullback–Leibler (KL,

una medida probabilística que cuantifica la discrepancia estadística entre la nueva política y la previa [GBC16, SLA+15]), la maximización exacta de la cota sustituta produce una política con retorno esperado no decreciente:

$$J(\pi_{k+1}) \geq J(\pi_k) \tag{2.20}$$

El algoritmo práctico de TRPO aproxima este procedimiento mediante ventajas estimadas, una divergencia KL promedio y optimización numérica. Sin embargo, resolver la optimización cuadrática con restricciones de TRPO en redes neuronales profundas resulta costoso en términos de cómputo. Como alternativa de primer orden, PPO [SWD+17] reemplaza la restricción formal por un objetivo sustituto recortado (CLIP) que busca limitar actualizaciones excesivas, sin transferir automáticamente la garantía teórica de TRPO.

Definiendo la razón de probabilidad entre la política actual y la previa:

$$r_t(\theta) \doteq \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{\text{old}}}(a_t \mid s_t)} \tag{2.21}$$

donde $\pi_{\theta_{\text{old}}}$ representa la política que recolectó el lote de transiciones, PPO maximiza el objetivo recortado:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\text{mín} \left(r_t(\theta)\hat{A}_t,\ \text{clip}(r_t(\theta), 1-\epsilon_{\text{clip}}, 1+\epsilon_{\text{clip}})\hat{A}_t \right) \right] \tag{2.22}$$

En la ecuación (2.22), $\epsilon_{\text{clip}} > 0$ fija el margen de recorte. La función $\text{clip}(x, l, u)$ conserva x si está entre l y u, devuelve l si queda por debajo y u si lo supera. El operador mín elige el menor de los dos términos. Así, una ventaja positiva deja de incentivar aumentos de la razón por encima de $1+\epsilon_{\text{clip}}$, y una ventaja negativa deja de incentivar reducciones por debajo de $1-\epsilon_{\text{clip}}$. Este mecanismo limita el incentivo del objetivo; no impone una restricción estricta al cambio de la política.

En la arquitectura actor-crítico práctica, este objetivo de política se integra de forma conjunta con la regresión del crítico y un término de regularización entrópica:

$$J^{\text{PPO}}(\theta, \phi) = L^{\text{CLIP}}(\theta) - c_1\hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right] + c_2\hat{\mathbb{E}}_t \left[S[\pi_\theta](s_t) \right] \tag{2.23}$$

donde ϕ son los parámetros del crítico, R_t^{target} es el retorno objetivo estimado (calculado como $\hat{A}_t + V_\phi(s_t)$), $S[\pi_\theta](s_t) \doteq -\sum_a \pi_\theta(a \mid s_t) \log \pi_\theta(a \mid s_t)$ es la entropía de Shannon de la política en s_t, y $c_1, c_2 > 0$ son coeficientes de ponderación [SWD+17]. La inclusión del término entrópico favorece la exploración activa al penalizar distribuciones excesivamente concentradas, favoreciendo que se consideren distintas fases durante el aprendizaje.

PPO opera de forma predominantemente *on-policy*: recolecta un lote de trayectorias con la política vigente $\pi_{\theta_{\text{old}}}$, ejecuta un número acotado de *epochs* de optimización sobre *minibatches*, y luego descarta el lote. De esta forma, las siguientes actualizaciones utilizan

experiencia obtenida con una política más reciente. El algoritmo 2 describe este ciclo de entrenamiento:

Algoritmo 2 Proximal Policy Optimization (PPO)

Entrada: entorno, horizonte de recolección, *epochs* K, γ, λ_{GAE} y ϵ_{clip}
Salida: parámetros entrenados θ del actor y ϕ del crítico

Inicializar los parámetros del actor θ y del crítico ϕ
Inicializar la política de recolección con $\theta_{\text{old}} \leftarrow \theta$
para iteración $= 1, 2, \ldots$ **hacer**
    Recolectar trayectorias con la política $\pi_{\theta_{\text{old}}}$
    Calcular los retornos objetivo y las ventajas estimadas $\hat{A}_t$
    para *epoch* $= 1$ hasta K **hacer**
        Muestrear *minibatches* del conjunto de trayectorias recolectado
        Calcular $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$
        Actualizar θ maximizando $L^{\text{CLIP}}(\theta)$
        Actualizar ϕ minimizando el error cuadrático del valor
        Incorporar, si corresponde, un término de entropía para favorecer la exploración
    fin para
    $\theta_{\text{old}} \leftarrow \theta$
fin para

En PPO, θ_{old} representa la política recolectora y permanece fijo al calcular las razones de probabilidad. El lote se reutiliza durante un número acotado de *epochs*; el recorte reduce el incentivo para que las razones superen el intervalo $[1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}]$, pero no impone una restricción estricta ni garantiza una mejora monótona [SWD+17]. Después de la optimización, θ_{old} se actualiza con θ y se recolectan nuevas experiencias. Durante la ejecución en la red semafórica, el crítico se desacopla y el agente utiliza únicamente su actor local π_θ.

2.3. Aprendizaje por refuerzo multiagente (MARL)

El MARL generaliza el marco de RL hacia sistemas donde múltiples agentes autónomos interactúan y aprenden en un entorno compartido [ACS24]. En el control de redes semafóricas, la hipótesis de agente aislado deja de sostenerse: la fase habilitada en un cruce regula el flujo vehicular emitido hacia los accesos de las intersecciones adyacentes, acoplando espacialmente las dinámicas de congestión de toda la red. Cada agente influye directamente en las tasas de arribo y longitudes de cola de sus vecinos, lo que demanda esquemas de coordinación capaces de optimizar el flujo global.

Liang et al. [LML+25] categorizan los problemas multiagente según la estructura de sus

recompensas en completamente competitivos, mixtos o completamente cooperativos. Este trabajo persigue un objetivo de coordinación vial, pero sus implementaciones entregan recompensas individuales. Las búsquedas informadas emplean recompensa local; la comparación vecinal de MAPPO agrega un término de vecinos en una de sus variantes (sección 3.3.2). La finalidad cooperativa no hace idénticos los retornos individuales ni convierte automáticamente la implementación en un juego de recompensa escalar compartida.

2.3.1. Fundamentos teóricos: juegos estocásticos, equilibrio de Nash y Dec-POMDP

La extensión matemática del MDP ante múltiples actores decisores se formaliza mediante los Juegos Estocásticos o Juegos Markovianos (*Stochastic Games*), introducidos originalmente por Shapley [Sha53] y adaptados al aprendizaje por refuerzo contemporáneo por Albrecht et al. [ACS24].

Un Juego Estocástico de n agentes se define mediante la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^{n}, P, \{R_i\}_{i=1}^{n}, \gamma \rangle \tag{2.24}$$

donde $\mathcal{N} = \{1, \ldots, n\}$ es el conjunto de agentes (intersecciones), $\mathcal{S}$ representa el espacio de estados globales de la red vial, $\mathcal{A}_i$ es el conjunto discreto de fases semafóricas disponibles para el agente i, y $\mathbf{a} = (a_1, \ldots, a_n) \in \boldsymbol{\mathcal{A}} \equiv \mathcal{A}_1 \times \cdots \times \mathcal{A}_n$ denota la acción conjunta. La función de transición $P(s' \mid s, \mathbf{a})$ modela la probabilidad de evolucionar hacia el estado global s' dada la acción conjunta ejecutada en el simulador SUMO, y $R_i(s, \mathbf{a})$ es la función de recompensa asignada al agente i.

En el análisis de estabilidad de estos sistemas, el equilibrio de Nash describe una configuración donde ningún agente puede incrementar unilateralmente su retorno esperado desviándose de su política actual [ACS24]. Formalmente, una política conjunta $\boldsymbol{\pi}^* = (\pi_1^*, \ldots, \pi_n^*)$ constituye un equilibrio de Nash si:

$$J(\pi_i, \boldsymbol{\pi}_{-i}^*) \leq J(\pi_i^*, \boldsymbol{\pi}_{-i}^*), \quad \forall \pi_i \in \Pi_i, \quad \forall i \in \mathcal{N} \tag{2.25}$$

donde $J(\cdot)$ denota el retorno esperado del agente i bajo el perfil de políticas indicado, $\boldsymbol{\pi}_{-i}^*$ representa el perfil de políticas conjuntas de todos los agentes excepto el agente i, y Π_i es el espacio de políticas admisibles. Aunque en sistemas estrictamente cooperativos este equilibrio caracteriza puntos de estabilidad mutua, no garantiza por sí mismo la optimalidad global de Pareto.

En redes urbanas reales, los agentes carecen de información instantánea y global sobre todo el trazado vial. Cada intersección opera sujeta a observabilidad parcial, percibiendo únicamente el estado de sus detectores locales. Una formulación teórica para observabilidad parcial y retorno compartido es el Proceso de decisión de Markov parcialmente observable descentralizado (*Decentralized Partially Observable Markov Decision Process*)

(Dec-POMDP) (*Decentralized Partially Observable Markov Decision Process*) [AOS20], definido por la tupla:

$$\langle \mathcal{N}, \mathcal{S}, \{\mathcal{A}_i\}_{i=1}^{n}, P, R, \{\Omega_i\}_{i=1}^{n}, O, \gamma \rangle \tag{2.26}$$

donde Ω_i es el conjunto de observaciones locales accesibles al agente i, y $O(o_1, \ldots, o_n \mid s', \mathbf{a}) = \Pr\{\mathbf{o} = (o_1, \ldots, o_n) \mid s', \mathbf{a}\}$ es la función de emisión estocástica de observaciones [AOS20]. El estado global s es una variable del modelo que no tiene por qué ser accesible a cada agente; en general, una política descentralizada puede depender del historial local de observaciones y acciones τ_i y expresarse como $\pi_i(a_i \mid \tau_i)$. En la tupla, R es una recompensa escalar compartida. La implementación experimental entrega señales individuales R_i y se describe como un juego parcialmente observable, sin identificarla automáticamente con ese caso cooperativo de recompensa compartida (sección 3.3.2). Cada agente adopta una política reactiva basada en la observación actual $\pi_i(a_i \mid o_i)$, sin identificar o_i con s (Figura 2.17).

La Figura 2.17 esquematiza este acoplamiento: cada agente procesa su observación local o_i y emite una fase a_i. La concurrencia de estas decisiones conforma la acción conjunta $\mathbf{a}$, la cual perturba el estado global del simulador de tráfico y devuelve a cada agente su nueva observación y su retroalimentación individual r_i.

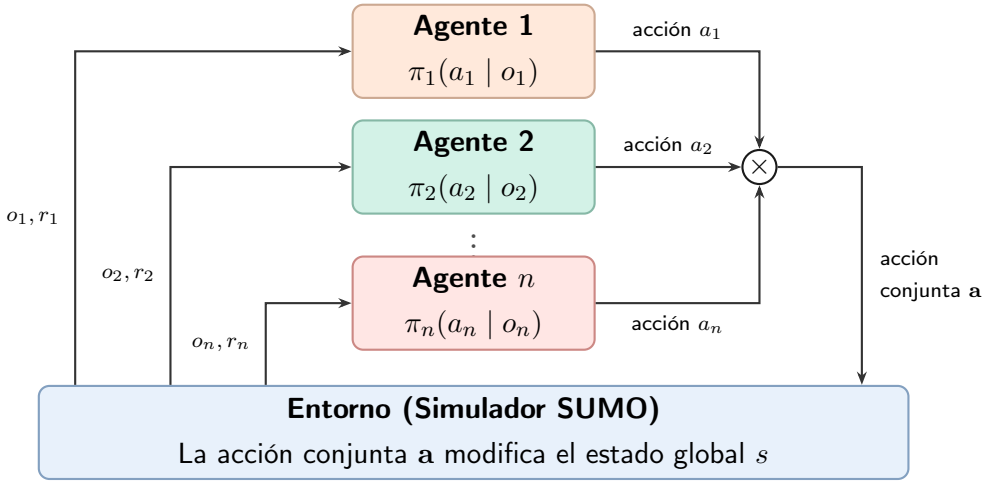


Figura 2.17: Interacción agente-entorno en un esquema MARL cooperativo. Cada semáforo emite una acción local (a_i) formando una acción conjunta ($\mathbf{a}$) que modifica el estado global del entorno de tráfico.

2.3.2. Desafíos del aprendizaje multiagente

La transición desde el aprendizaje individual hacia el dominio multiagente introduce desafíos teóricos y computacionales específicos [ACS24]:

- **Explosión combinatoria del espacio de acciones conjuntas:** si cada agente i dispone de un conjunto de acciones locales $\mathcal{A}_i$, la cardinalidad del espacio de acciones

conjuntas es:

$$|\boldsymbol{\mathcal{A}}| = \prod_{i=1}^{n} |\mathcal{A}_i| \,. \tag{2.27}$$

Cuando todos los agentes poseen A acciones, esta cardinalidad es A^n y, por tanto, crece exponencialmente con el número de agentes. Para una red de escala urbana, evaluar o enumerar exhaustivamente $\boldsymbol{\mathcal{A}}$ deja de ser una alternativa viable, lo que motiva el diseño de enfoques escalables, como políticas de decisión localmente descentralizadas.

- **No estacionariedad y el problema del valor objetivo cambiante:** desde la perspectiva local del agente i, la función de transición ambiental percibida:

$$\mathcal{P}_i(s' \mid s, a_i) = \sum_{\mathbf{a}_{-i}} P(s' \mid s, a_i, \mathbf{a}_{-i}) \boldsymbol{\pi}_{-i}(\mathbf{a}_{-i} \mid s) \tag{2.28}$$

varía de forma continua a medida que los restantes agentes adaptan sus respectivas políticas $\boldsymbol{\pi}_{-i}$. Como formalizaron Bowling y Veloso [BV02], este cambio produce el problema del valor objetivo cambiante (*moving target*): una política óptima frente al comportamiento vigente de los vecinos puede dejar de ser adecuada en cuanto estos ajustan sus parámetros. Aunque el sistema global puede seguir describiéndose mediante un juego markoviano, la dinámica observada por un agente local se vuelve no estacionaria cuando las políticas de los demás agentes cambian y no forman parte de su información. En consecuencia, las condiciones de convergencia de los métodos de agente único basados en MDPs estacionarios dejan de ser suficientes [FNF+17, LML+25].

En juegos competitivos o de suma cero como Piedra, Papel o Tijera, el aprendizaje independiente con tasas fijas genera trayectorias oscilatorias alrededor del equilibrio. Algoritmos especializados como Ascenso de colina en la política con ganar o aprender rápido (*Win or Learn Fast Policy Hill-Climbing*) (WoLF-PHC) (*Win or Learn Fast Policy Hill-Climbing*) modulan adaptativamente su tasa de aprendizaje, amortiguando las espirales hacia el equilibrio de Nash ($\pi^*(a) = 1/3$), como ilustra la Figura 2.18. En el control semafórico, la no estacionariedad se manifiesta cuando un cruce extiende su verde para permitir el paso de un grupo de vehículos y modifica las llegadas a los cruces siguientes; si los agentes aprenden sin coordinación, corren el riesgo de generar variaciones de las colas que dificulten el servicio en otros cruces.

- **Observabilidad parcial:** bajo el formalismo Dec-POMDP, cada semáforo basa su inferencia exclusivamente en detectores locales, desconociendo el estado global de la red vial y las decisiones simultáneas de cruces lejanos, lo cual dificulta la estimación del valor de sus decisiones.

- **Asignación de crédito multiagente (*Credit Assignment*):** cuando múltiples agentes actúan en paralelo, determinar la contribución marginal de cada decisión local sobre el desempeño del sistema resulta complejo [ACS24]. Una acción local desfavorable puede quedar enmascarada si el resto de los semáforos operó de forma eficiente, o viceversa. El diseño de recompensas individuales compuestas (sección 3.3.2) incorpora señales de la intersección y sus vecinos, pero no identifica la contribución causal de cada acción.

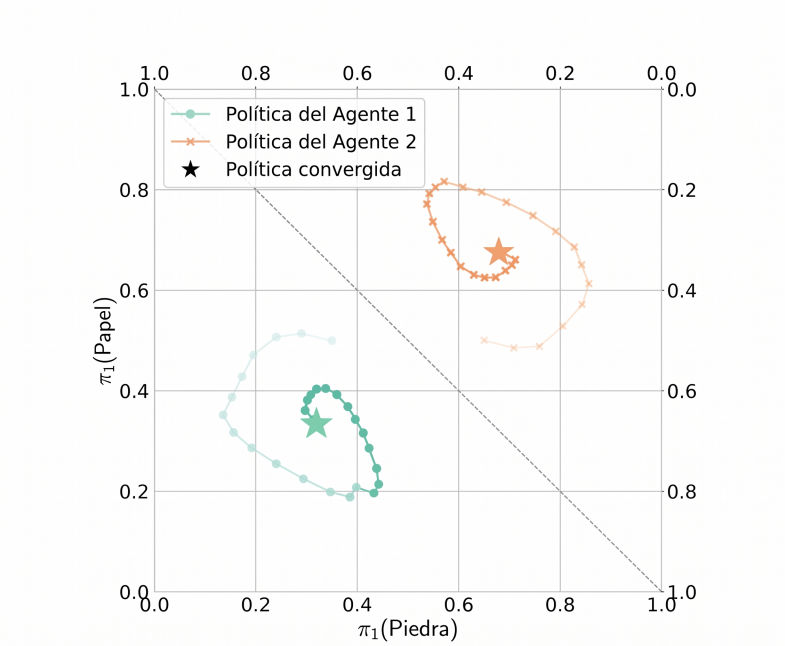


Figura 2.18: Evolución de las políticas de dos agentes con el algoritmo WoLF-PHC en el juego de Piedra, Papel o Tijera. La diagonal discontinua divide el gráfico en dos símplex de probabilidad, uno para cada agente. Cada punto en el símplex representa una distribución sobre las tres acciones disponibles; las trayectorias muestran el avance de las políticas durante el entrenamiento y los marcadores centrales destacan la política convergida en el equilibrio de Nash ($\pi^*(a) = 1/3$ para cada acción). Adaptado de [BV02, ACS24].

2.3.3. Paradigmas de entrenamiento y ejecución

Para mitigar la no estacionariedad respetando la observabilidad parcial de los agentes durante la ejecución, la literatura distingue dos paradigmas fundamentales según la información disponible durante la optimización [ZLYZ23]:

- **Aprendizaje Independiente (IL, *Independent Learning*):** cada agente aprende de forma totalmente descentralizada tratando a los demás agentes como componentes estocásticos del entorno. Aunque computacionalmente simple y altamente escalable, es susceptible a la inestabilidad por no estacionariedad.

- **Entrenamiento centralizado con ejecución descentralizada (*Centralized Training with Decentralized Execution*, CTDE):** durante la etapa de entre-

namiento en el simulador, se aprovecha información global privilegiada (estados conjuntos o concatenaciones de observaciones) para estimar valores y orientar el aprendizaje de las políticas locales; durante la etapa de ejecución, el componente centralizado se desacopla y cada agente opera con su política local. La Figura 2.19 ilustra esta separación entre etapas.

2.3.4. Aprendizaje por refuerzo profundo multiagente (*Multi-Agent Deep Reinforcement Learning*) (MADRL) y líneas de base

El uso de redes neuronales para aproximar políticas y funciones de valor de varios agentes forma parte del MADRL [LWT+17]. Para evaluar empíricamente el valor agregado del entrenamiento centralizado (MAPPO y HAPPO), esta investigación establece dos líneas de base de aprendizaje independiente (IL):

- **IDQN (Línea de base basada en valor):** extensión de DQN donde cada intersección mantiene su propia red $Q_i(o_i, a_i)$ [TMK+17]. Al operar *off-policy* con un *replay buffer*, almacena transiciones generadas bajo políticas anteriores, por lo que las experiencias pueden reflejar comportamientos distintos de los actuales [FNF+17].
- **IPPO (Línea de base basada en política):** instancia un optimizador PPO independiente por agente [dWGM+20]. Dado su régimen *on-policy*, descarta el lote tras cada iteración, mitigando la acumulación de datos obsoletos. Aunque no elimina el acoplamiento no estacionario, su mecanismo de recorte permite utilizarlo como referencia frente a los métodos coordinados.

La Figura 2.19 resume esta partición operativa entre ambas etapas.

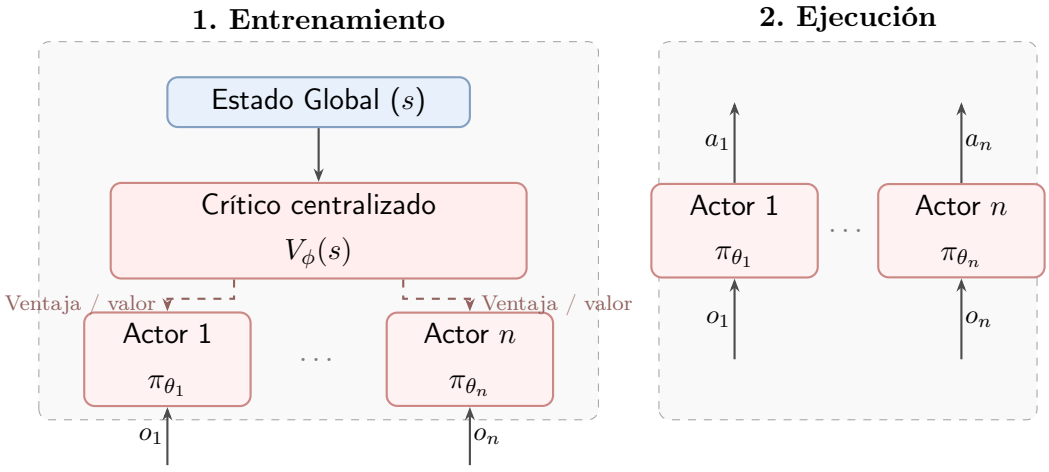


Figura 2.19: Paradigma de entrenamiento centralizado con ejecución descentralizada (CTDE). Durante el entrenamiento, el crítico utiliza información global privilegiada para estimar valores o ventajas que orientan la actualización de los actores locales; durante la ejecución, los actores solo requieren observaciones locales.

2.3.5. Multi-Agent Proximal Policy Optimization (MAPPO)

Para extender las ventajas de estabilidad de PPO al contexto multiagente bajo el paradigma CTDE, Yu et al. [YVV+22] estudiaron una configuración de Multi-Agent Proximal Policy Optimization (MAPPO) basada en PPO y un crítico centralizado. En lugar de rediseñar el estimador base, MAPPO preserva la simplicidad computacional de PPO mediante la factorización explícita entre políticas descentralizadas y evaluación centralizada:

- **Actores descentralizados ($\pi_{\theta_i}(a_i \mid o_i)$):** cada agente i procesa exclusivamente su vector de observación local o_i para seleccionar su fase semafórica a_i. En intersecciones con geometrías idénticas puede compartirse parámetros entre actores, aunque esto no constituye una restricción conceptual.
- **Crítico centralizado ($V_\phi(s)$):** durante el entrenamiento, el crítico puede utilizar información global que no está disponible para los actores. Una representación global adecuada puede facilitar la estimación del valor. Una estimación más precisa también puede reducir la variabilidad de las actualizaciones, aunque esto depende de la configuración [YVV+22].

El objetivo de optimización de los actores promedia los términos individuales de recorte PPO sobre los n agentes:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^{n} \text{mín} \left(r_t^i(\theta) \hat{A}_t(s_t, \mathbf{a}_t),\ \text{clip}(r_t^i(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_t(s_t, \mathbf{a}_t) \right) \right] \tag{2.29}$$

donde $r_t^i(\theta) = \frac{\pi_{\theta_i}(a_t^i|o_t^i)}{\pi_{\theta_{i,\text{old}}}(a_t^i|o_t^i)}$ denota la razón de probabilidad local de cada agente y ϵ_{clip} es el margen de recorte. Paralelamente, el crítico centralizado se actualiza minimizando el error cuadrático frente al retorno objetivo R_t^{target}, estimado empíricamente sobre el lote de transiciones mediante la suma de la ventaja GAE y el valor de estado previo $\hat{A}_t(s_t, \mathbf{a}_t) + V_\phi(s_t)$ [YVV+22, SML+16]:

$$\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]. \tag{2.30}$$

Factores críticos de diseño y optimización en MAPPO

Yu et al. [YVV+22] analizaron empíricamente diversos factores de implementación e hiperparámetros que afectan el rendimiento de MAPPO:

1. **Normalización de retornos:** en los experimentos de Yu et al., normalizar los objetivos de valor puede favorecer el ajuste del crítico cuando cambia la escala de las recompensas.
2. **Estructuración del estado global del crítico:** en ese estudio, la entrada s se evaluó con distintas representaciones: concatenación directa de observaciones (CL :

$s = (o_1, \ldots, o_n)$), estado global extraído directamente del simulador (EP), o estados aumentados específicos por agente ($AS_i = EP \cup o_i$), combinados opcionalmente con filtrado de redundancias (FP).

3. **Reutilización de datos:** la cantidad de *epochs* (K) y la división en *minibatches* regulan la reutilización del lote. Yu et al. observaron que una reutilización excesiva puede deteriorar el rendimiento, especialmente en tareas difíciles.

El algoritmo 3 especifica el ciclo computacional de MAPPO bajo el paradigma CTDE.

Algoritmo 3 Algoritmo Multi-Agent Proximal Policy Optimization (MAPPO)

Entrada: entorno multiagente, n agentes, horizonte de recolección, *epochs* K, γ, λ_{GAE} y ϵ_{clip}

Salida: parámetros entrenados de los actores θ y del crítico centralizado ϕ

Inicializar los parámetros de la política (actor) θ y del valor (crítico) ϕ
Inicializar el *buffer* de datos $\mathcal{D} \leftarrow \emptyset$
para iteración $= 1, 2, \ldots$ **hacer**
    Recolectar trayectorias *on-policy* para todos los agentes utilizando la política conjunta $\boldsymbol{\pi}_\theta$
    Calcular las ventajas conjuntas $\hat{A}_t(s_t, \mathbf{a}_t)$ mediante GAE a partir del crítico centralizado $V_\phi(s_t)$
    Calcular los retornos objetivos R_t^{target} normalizados para el crítico
    Almacenar las transiciones $(s_t, \mathbf{o}_t, \mathbf{a}_t, \hat{A}_t, R_t^{\text{target}})$ en $\mathcal{D}$
    para *epoch* $= 1$ hasta K **hacer**
        Muestrear *minibatches* de datos desde $\mathcal{D}$
        Actualizar la política maximizando el objetivo de recorte PPO por agente: $L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\frac{1}{n} \sum_{i=1}^{n} \text{mín} \left(r_t^i(\theta) \hat{A}_t,\ \text{clip}(r_t^i(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) \hat{A}_t \right) \right]$
        Actualizar el crítico ϕ minimizando el error cuadrático medio del valor: $\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t \left[\left(V_\phi(s_t) - R_t^{\text{target}} \right)^2 \right]$
    fin para
    Vaciar el *buffer* de datos $\mathcal{D} \leftarrow \emptyset$
fin para

Durante el entrenamiento, el crítico centralizado $V_\phi(s)$ procesa el estado global de la red vial para calcular ventajas conjuntas mediante GAE, guiando la optimización paralela de los actores locales π_{θ_i} con el objetivo recortado. Al finalizar cada iteración, el *buffer* $\mathcal{D}$ se descarta por completo para preservar el régimen *on-policy* del método. En la etapa de ejecución, el crítico se desacopla y cada agente actúa con autonomía computacional a partir de su vector local o_t^i. Los hiperparámetros de entrenamiento y la arquitectura de las redes se detallan en el capítulo 3.

La formulación anterior presenta la variante teórica con una ventaja conjunta. En la implementación de este trabajo, el crítico centralizado produce una salida por agente y los objetivos de GAE se calculan por agente a partir de la recompensa individual compuesta descrita en la sección 3.3.2; por tanto, la ecuación resume el paradigma CTDE y no debe leerse como una identidad exacta del código ejecutado.

2.3.6. Heterogeneous-Agent Proximal Policy Optimization (HAPPO)

En sistemas de control complejos, los agentes pueden presentar asimetrías físicas y operacionales: diferencias en la configuración de actuadores, distintos espacios de acciones o demandas regionales contrapuestas. En entornos homogéneos, compartir parámetros puede mejorar la eficiencia de aprendizaje; en entornos heterogéneos, esa práctica puede limitar la especialización. Además, las actualizaciones independientes no poseen una garantía general de convergencia debido al aprendizaje simultáneo no coordinado [KCW+22].

Kuba et al. y Zhong et al. propusieron algoritmos de aprendizaje por refuerzo multiagente para políticas heterogéneas, sin exigir que todos los agentes compartan parámetros. Esta línea se denomina aprendizaje por refuerzo con agentes heterogéneos (*Heterogeneous-Agent Reinforcement Learning*, HARL). Cada agente i puede mantener su propia arquitectura y parámetros θ^i, adaptados a su espacio de acciones local $\mathcal{A}_i$. En esta línea, Optimización de política por región de confianza con agentes heterogéneos (*Heterogeneous-Agent Trust Region Policy Optimization*) (HATRPO) y HAPPO emplean una actualización secuencial de los agentes. Los resultados teóricos de mejora monótona y convergencia se refieren al procedimiento de región de confianza bajo supuestos explícitos; la variante práctica HAPPO sustituye la restricción KL por un objetivo recortado y debe interpretarse como una aproximación [KCW+22, ZKF+24].

Compartición de parámetros y políticas heterogéneas

Una distinción central entre MAPPO y HAPPO radica en la homogeneidad de las políticas y en el orden de actualización:

- **Compartición de parámetros:** en MAPPO se utiliza habitualmente cuando los agentes poseen espacios de observación y acción compatibles. Todos los agentes pueden compartir los mismos pesos $\theta_1 = \cdots = \theta_n = \theta$, aunque la formulación de MAPPO también puede implementarse sin esta restricción.

- **Políticas heterogéneas y actualización secuencial:** HAPPO permite que cada agente mantenga una política independiente $\pi_{\theta^i}(a_i \mid o_i)$ y actualiza los actores siguiendo una permutación. El lote de trayectorias de la iteración actual se reutiliza de forma limitada; esto no equivale a un *buffer* histórico de *experience sharing* como el empleado por algoritmos específicos de compartición de experiencia.

Lema de descomposición de la ventaja multiagente

La actualización simultánea de todos los agentes puede generar inestabilidad porque el cambio concurrente de las políticas altera tanto las distribuciones de acción como la distribución conjunta de las trayectorias. HAPPO aborda este problema mediante una actualización secuencial: en cada iteración se optimiza un agente por vez, condicionando cada paso a los cambios ya efectuados por los precedentes. Para sustentar este esquema, la ventaja conjunta de la red debe poder descomponerse en contribuciones marginales ordenadas [KCW+22].

Esta descomposición se formaliza en el lema de descomposición de la ventaja multiagente propuesto por Kuba et al. [KCW+22]. En cualquier juego markoviano cooperativo, para una política conjunta $\boldsymbol{\pi}$ y un orden arbitrario de los agentes $i_{1:n} = (i_1, \ldots, i_n)$, la ventaja conjunta de la red se descompone exactamente como la suma de las ventajas marginales de cada agente, condicionadas por las decisiones de los que lo preceden en la secuencia:

$$A^{\boldsymbol{\pi}}(s, \mathbf{a}) = \sum_{m=1}^{n} A^{\boldsymbol{\pi}}_{i_m}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \tag{2.31}$$

donde la ventaja marginal del agente i_m se define como:

$$A^{\boldsymbol{\pi}}_{i_m}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) \doteq Q^{\boldsymbol{\pi}}_{i_{1:m}}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m}) - Q^{\boldsymbol{\pi}}_{i_{1:m-1}}(s, \mathbf{a}_{i_{1:m-1}}). \tag{2.32}$$

En la ecuación (2.32), $Q^{\boldsymbol{\pi}}_{i_{1:m}}(s, \mathbf{a}_{i_{1:m-1}}, a_{i_m})$ evalúa el retorno esperado fijando las acciones de los primeros m agentes de la secuencia, mientras que las decisiones de los agentes posteriores ($\mathbf{a}_{i_{m+1:n}}$) se promedian según sus probabilidades bajo la política vigente.

La deducción matemática original de este lema asume un juego markoviano estrictamente cooperativo con un retorno escalar compartido $R(s, \mathbf{a})$ para toda la red. En sistemas espaciales distribuidos de gran escala, como el control semafórico urbano, asignar una recompensa global idéntica a todas las intersecciones dificulta la asignación de crédito, extremo que Chu et al. respaldan con evidencia empírica en redes extensas [CWCL19], y encarece la estimación de las ventajas [ACS24]. En este trabajo se emplean señales locales y, en protocolos específicos, de vecindario inmediato. Esta decisión de implementación no conserva por sí sola los supuestos del lema original.

Bajo retornos desacoplados, la identidad aditiva de la ecuación (2.31) no se traduce en una equivalencia algebraica directa entre un único valor global y los retornos individuales; no obstante, el principio de optimización secuencial por bloques (tipo Gauss-Seidel) conserva su utilidad práctica para desacoplar el aprendizaje concurrente [ACS24, WZK+23].

La relevancia teórica de este lema radica en que es una identidad analítica exacta para cualquier juego cooperativo sin imponer restricciones de aditividad o de monotonía sobre la función de valor en la red de mezclado (a diferencia de las condiciones de factorización IGM de arquitecturas como Redes de descomposición de valor (*Value-Decomposition*

Networks) (VDN) [SLG+18] o Red de mezcla de valores Q (*Q-Mixing Network*) (QMIX) [RSDW+18]).

Esquema de actualización secuencial y factor de razón compuesto M

Para explotar analíticamente esta descomposición teniendo en cuenta los cambios de los agentes ya actualizados, HAPPO actualiza las políticas de los agentes de manera secuencial dentro de cada iteración k. En cada ciclo se sortea una permutación de los agentes $i_{1:n} \in \mathrm{Sym}(n)$ (donde $\mathrm{Sym}(n)$ representa el conjunto de todas las permutaciones posibles de los n agentes) para que ningún agente ocupe siempre la misma posición en la secuencia [KCW+22, WZK+23].

Cuando corresponde el turno del agente i_m, los agentes previos en la secuencia ($i_{1:m-1}$) ya han actualizado sus parámetros a $\theta_{k+1}^{i_j}$, mientras que los agentes posteriores ($i_{m+1:n}$) conservan aún sus parámetros previos $\theta_k^{i_j}$. Para guiar al agente i_m teniendo en cuenta la alteración de la distribución conjunta producida por los agentes ya actualizados, Kuba et al. [KCW+22] demostraron que no se requiere entrenar un crítico separado por cada subgrupo; la ventaja marginal puede computarse reponderando la ventaja conjunta $A^{\boldsymbol{\pi}_k}(s, \mathbf{a})$ mediante la productoria acumulada de las razones de probabilidad de los agentes ya actualizados:

$$M_{i_{1:m}}(s_t, \mathbf{a}_t) \doteq \left(\prod_{j=1}^{m-1} \frac{\pi_{\theta_{k+1}^{i_j}}^{i_j}(a_t^{i_j} \mid o_t^{i_j})}{\pi_{\theta_k^{i_j}}^{i_j}(a_t^{i_j} \mid o_t^{i_j})} \right) A^{\boldsymbol{\pi}_k}(s_t, \mathbf{a}_t) \tag{2.33}$$

donde $\pi_{\theta_{k+1}^{i_j}}^{i_j}$ es la política recién optimizada del agente i_j y $\pi_{\theta_k^{i_j}}^{i_j}$ es su versión previa. Para el primer agente ($m = 1$), la productoria es vacía y $M_{i_1}(s_t, \mathbf{a}_t) \equiv A^{\boldsymbol{\pi}_k}(s_t, \mathbf{a}_t)$. En la formulación algorítmica práctica, el término acumulado $M_{i_{1:m}}$ se mantiene en una variable de memoria M_{i_m} por cada agente, la cual se actualiza recursivamente conforme avanzan las optimizaciones precedentes. El factor $M_{i_{1:m}}$ actúa como un estimador de importancia que ajusta la ventaja conjunta al nuevo perfil de comportamiento sin requerir simulaciones adicionales en el entorno [KCW+22].

Así, la política del agente i_m se optimiza maximizando el objetivo sustituto recortado ponderado por $M_{i_{1:m}}$:

$$\begin{aligned} L^{\mathrm{HAPPO}}(\theta^{i_m}) = \hat{\mathbb{E}}_t \Big[\min \Big(& r_t^{i_m}(\theta^{i_m}) M_{i_{1:m}}(s_t, \mathbf{a}_t), \\ & \mathrm{clip}\big(r_t^{i_m}(\theta^{i_m}), 1 - \epsilon_{\mathrm{clip}}, 1 + \epsilon_{\mathrm{clip}}\big) M_{i_{1:m}}(s_t, \mathbf{a}_t) \Big) \Big] \end{aligned} \tag{2.34}$$

donde $r_t^{i_m}(\theta^{i_m}) = \frac{\pi_{\theta^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} | o_t^{i_m})}$ representa la razón de probabilidad individual del agente.

Concluida la optimización de θ^{i_m}, se computa la razón post-optimización:

$$\rho_{\text{post}}^{i_m} \doteq \frac{\pi_{\theta_{k+1}^{i_m}}^{i_m}(a_t^{i_m} \mid o_t^{i_m})}{\pi_{\theta_k^{i_m}}^{i_m}(a_t^{i_m} \mid o_t^{i_m})} \tag{2.35}$$

y se propaga recursivamente hacia los agentes pendientes mediante:

$$M_{i_{1:m+1}}(s_t, \mathbf{a}_t) \leftarrow \rho_{\text{post}}^{i_m} \cdot M_{i_{1:m}}(s_t, \mathbf{a}_t). \tag{2.36}$$

Garantía de mejora monótona

El principio de mejora monótona en aprendizaje por refuerzo formaliza que la secuencia de retornos esperados de la política conjunta no disminuye entre ciclos sucesivos de entrenamiento, comportándose como una función monótonamente no decreciente respecto de la iteración k:

$$J(\boldsymbol{\pi}_{k+1}) \geq J(\boldsymbol{\pi}_k). \tag{2.37}$$

Para políticas heterogéneas, Kuba et al. [KCW+22] demostraron analíticamente esta propiedad para el algoritmo HATRPO (*Heterogeneous-Agent Trust Region Policy Optimization*) bajo los supuestos del juego markoviano cooperativo con recompensa compartida y una actualización restringida por la divergencia KL. Resolver las restricciones de HATRPO en redes neuronales profundas requiere un costo de cómputo elevado. Por ello, HAPPO sustituye la cota formal de divergencia KL por el objetivo sustituto recortado CLIP de primer orden de la ecuación (2.34).

Esta sustitución constituye una aproximación pragmática: al igual que ocurre con PPO frente a TRPO en el caso de agente único, la garantía matemática demostrada para HATRPO no se transfiere automáticamente a la variante con recorte HAPPO, aunque los trabajos citados informan resultados empíricos favorables [KCW+22, ZKF+24]. Además, en redes urbanas con señales de recompensa local-vecinales, el procedimiento no garantiza una convergencia monótona analítica sobre una función objetivo global única; su implementación concreta debe evaluarse empíricamente, sin atribuirle de antemano una reducción de no estacionariedad o de conflictos entre actualizaciones. La Figura 2.20 resume este esquema de propagación secuencial.

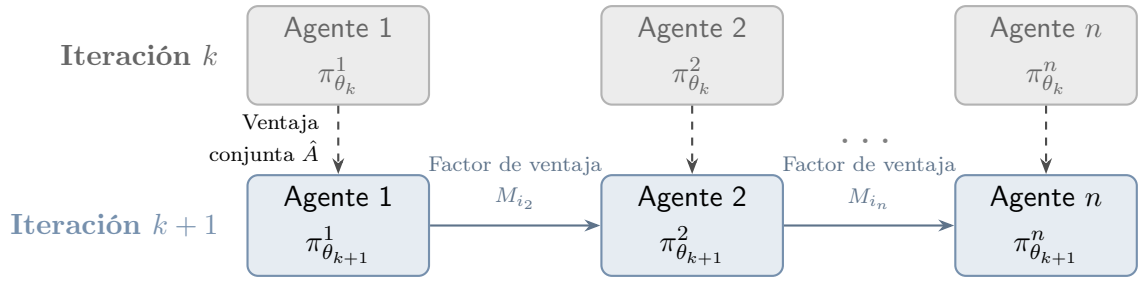


Figura 2.20: Esquema de actualización secuencial de HAPPO. A diferencia de PPO independiente, la actualización de cada agente está condicionada por el factor de ventaja reponderada M_{i_m}, que incorpora los cambios efectuados por los agentes que lo precedieron en la secuencia.

El algoritmo 4 presenta la secuencia completa de optimización de HAPPO.

Algoritmo 4 Algoritmo Heterogeneous-Agent Proximal Policy Optimization (HAPPO)

Entrada: entorno multiagente, conjunto $\mathcal{N}$ de n agentes, horizonte de recolección, γ, λ_{GAE} y ϵ_{clip}

Salida: políticas heterogéneas entrenadas $\{\theta^i\}_{i=1}^n$ y crítico centralizado ϕ

Inicializar los parámetros de las políticas individuales $\{\theta^i, \forall i \in \mathcal{N}\}$ y del crítico centralizado $V_\phi(s)$
para iteración $= 1, 2, \ldots$ **hacer**
  Recolectar un conjunto de trayectorias $\mathcal{D}$ utilizando la política conjunta $\boldsymbol{\pi}_\theta$
  Calcular el estimador de ventaja conjunta $\hat{A}(s, \mathbf{a})$ con GAE a partir del crítico $V_\phi(s)$
  Calcular los retornos objetivo R_t^{target} utilizados para ajustar el crítico
  Sortear una permutación de los agentes $i_{1:n} \in \text{Sym}(n)$
  Inicializar los factores $M_{i_m}(s, \mathbf{a}) \leftarrow \hat{A}(s, \mathbf{a})$ para todo $m \in \{1, \ldots, n\}$
  para $m = 1 \ldots n$ **hacer**
    Obtener el agente actual i_m y su factor modificador $M_{i_m}(s, \mathbf{a})$
    Actualizar θ^{i_m} a $\theta^{i_m}_{k+1}$ maximizando el objetivo recortado de la ecuación (2.34)
    Recalcular la razón post-optimización: $\rho^{i_m}_{\text{post}} \leftarrow \frac{\pi^{i_m}_{\theta^{i_m}_{k+1}}(a_t^{i_m}|o_t^{i_m})}{\pi^{i_m}_{\theta^{i_m}_{k}}(a_t^{i_m}|o_t^{i_m})}$
    para $j = m+1 \ldots n$ **hacer**
      Actualizar el factor modificador para agentes posteriores: $M_{i_j}(s_t, \mathbf{a}_t) \leftarrow \rho^{i_m}_{\text{post}} \cdot M_{i_j}(s_t, \mathbf{a}_t)$
    fin para
  fin para
  Actualizar el crítico centralizado ϕ mediante descenso de gradiente: $\mathcal{L}^{\text{VF}}(\phi) = \hat{\mathbb{E}}_t\left[\left(V_\phi(s_t) - R_t^{\text{target}}\right)^2\right]$
fin para

El procedimiento de HAPPO organiza secuencialmente la actualización de los actores locales mediante la permutación aleatoria $i_{1:n}$ y la propagación recursiva de los factores modificadores M_{i_m}. Tras optimizar sucesivamente a los n agentes sobre el lote de transiciones $\mathcal{D}$, el crítico centralizado V_ϕ ajusta sus parámetros mediante descenso de gradiente sobre los retornos acumulados. Durante la etapa de ejecución descentralizada, cada política local selecciona a_i a partir de o_i, sin requerir evaluar el crítico centralizado ni comunicación entre agentes para seleccionar su acción [KCW+22].

En la adaptación para control semafórico de este trabajo, la secuencia conserva esta estructura de permutación y actualización sucesiva, pero inicializa los factores M_{i_m} con las ventajas individuales $\hat{A}_i(s, \mathbf{a})$ calculadas mediante el crítico centralizado multi-salida a partir de las señales de recompensa compuestas (sección 3.3.2), propagándolos mediante la razón $\rho^{i_m}_{\text{post}}$ tras optimizar a cada actor. Esta adaptación utiliza ventajas individuales, mientras que la demostración teórica parte de una recompensa global compartida. Su efecto sobre el aprendizaje debe comprobarse mediante los experimentos; esta adaptación no hereda por sí sola la garantía teórica de mejora.

Para finalizar el estudio comparativo de los algoritmos derivados de PPO en escenarios MARL, la Tabla 2.3 sintetiza sus principales propiedades operativas, arquitectónicas y teóricas.

Tabla 2.3: Comparación sintética de familias algorítmicas PPO en entornos multiagente cooperativos.

Eje de análisis	**PPO de agente único**	**IPPO independiente**	**MAPPO / CTDE**	**HAPPO / HARL**
Paradigma	agente único	IL (descentralizado)	CTDE (centralizado/desc.)	CTDE; familia HARL
Actores	único $\pi_\theta(a \mid s)$	n políticas locales	actores locales; puede compartir parámetros	n políticas heterogéneas
Arquitectura del crítico	único $V_\phi(s)$	n críticos locales $V_{\phi_i}(o_i)$	un crítico centralizado $V_\phi(s)$	un crítico centralizado $V_\phi(s)$
Optimización	PPO estándar	PPO local simultáneo	PPO local con crítico global	actualización secuencial $i_{1:n}$
Señal de ventaja	ventaja local $\hat{A}(s, a)$	ventaja local $\hat{A}_i(o_i, a_i)$	ventaja conjunta $\hat{A}(s, \mathbf{a})$	factor compuesto $M_{i_{1:m}}(s, \mathbf{a})$
Mejora monótona	No estricta para PPO	Sin garantía general	Sin garantía estricta	Teórica bajo supuestos; HAPPO-clip es aproximado
Tipo de agente	No aplica	Depende de la implementación	Pensado para agentes homogéneos; la compartición de parámetros es opcional	Pensado para agentes heterogéneos

2.3.7. Antecedentes recientes del control semafórico multiagente

Para orientar la comparación, los antecedentes seleccionados se organizan según tres ejes: coordinación y entrenamiento, representación de las observaciones, y tratamiento de la heterogeneidad o la equidad entre intersecciones. Se priorizan investigaciones directamente relacionadas con la formulación de este trabajo.

El control coordinado de múltiples intersecciones mediante aprendizaje por refuerzo constituye un área de investigación activa. Entre los enfoques iniciales se encuentran aquellos que asignaban un agente a cada semáforo y utilizaban información local sobre colas, fases o vehículos detenidos. Sin embargo, la optimización aislada de cada intersección no contempla que una acción modifica el flujo recibido por los cruces vecinos. Por este

motivo, la literatura evolucionó hacia formulaciones cooperativas que incorporan mecanismos de coordinación espacial, representaciones estructuradas del estado y esquemas de entrenamiento centralizado [ACS24, LML+25].

Coordinación y entrenamiento

En la consolidación del control semafórico coordinado mediante aprendizaje profundo, varios trabajos abordaron de manera temprana la asignación de crédito y la no estacionariedad en redes viales:

- **Control actor-crítico con descuento espacial (MA2C):** Chu et al. [CWCL19] propusieron una arquitectura de ventaja actor-crítico multiagente donde cada intersección incorpora a su función de valor y recompensa la información de sus vecinos inmediatos atenuada por un factor de descuento espacial. Este trabajo fundamentó conceptualmente y evidenció de manera empírica que una recompensa de equipo global en redes extensas induce severos problemas de asignación de crédito, mientras que las recompensas regionalmente estructuradas permiten coordinar corredores sin requerir estados globales inmanejables.

- **Coordinación basada en presión vehicular (PressLight):** Wei et al. [WCZ+19] fundamentaron su función de recompensa en el principio de *Max-Pressure* de la ingeniería de transporte, que en la teoría de control de redes garantiza estabilidad de cola bajo supuestos específicos de un esquema de servicio discreto. PressLight incorpora ese principio como recompensa de aprendizaje por refuerzo al maximizar la diferencia de vehículos detenidos entre corrientes entrantes y salientes. Esa incorporación no hereda la garantía formal de estabilidad y coordina de manera implícita las intersecciones adyacentes sin necesidad de comunicación explícita.

- **Atención en grafos para cooperación a nivel de red (CoLight):** Wei et al. [WXZ+19] introdujeron redes de atención en grafos para permitir que los agentes semafóricos intercambien dinámicamente representaciones de sus estados con cruces vecinos, adaptando la influencia de cada intersección adyacente según las condiciones variables de congestión.

- **Límites del aprendizaje independiente en redes viales:** Ault y Sharon [AS22] analizaron las limitaciones de los enfoques independientes frente a formulaciones cooperativas en redes de múltiples intersecciones, evidenciando un deterioro progresivo en el desempeño y en la fluidez del tránsito a medida que la demanda se incrementa debido a la falta de coordinación y a la no estacionariedad del entorno.

A partir de estas bases fundacionales, diversas investigaciones recientes han explorado mecanismos alternativos de optimización y distribución de cómputo. Wu et al. [WWS+22] estudian el control de redes de gran escala mediante agentes distribuidos de aprendizaje

por refuerzo profundo y una arquitectura de cómputo distribuido. Proponen dos variantes, Nash-A2C y Nash-A3C, que incorporan un equilibrio de Nash en el aprendizaje por refuerzo y se despliegan en una arquitectura distribuida para simulación y control. En sus experimentos reportan reducciones del tiempo de congestión y de la demora total de red frente a controladores de referencia. Este antecedente es pertinente como referencia de escalabilidad, pero no es equivalente a la formulación de este trabajo: aquí la ejecución también es local por intersección, mientras que la coordinación se estudia mediante la señal individual compuesta local-vecinal y el entrenamiento centralizado, sin introducir una capa de cómputo fog ni un equilibrio de Nash.

En una formulación cooperativa basada en DQN, Haddad et al. [HHA22] incorporan a la función de aprendizaje de cada intersección información sobre los estados, acciones y recompensas de sus vecinos. El estudio informa mejoras en tiempo de espera, longitud de cola y emisiones de dióxido de carbono frente a controladores de referencia. Esta evidencia respalda la necesidad de considerar el efecto de las decisiones vecinas, pero su mecanismo de coordinación es una transferencia explícita de información hacia la pérdida de DQN. En este estudio, IPPO e IDQN implementan el paradigma de aprendizaje independiente puro y reciben una señal de recompensa estrictamente local ($R_i = L_i$, $\rho_{\text{coop}} = 0$); MAPPO y HAPPO usan también recompensa local en la búsqueda informada, con crítico centralizado durante el entrenamiento. La mezcla vecinal se evalúa en una línea separada de MAPPO y no se atribuye a todos los algoritmos. Estas alternativas no son equivalentes al mecanismo propuesto por Haddad et al., sino procedimientos de coordinación distintos que se comparan bajo una misma representación y escenario.

Fang et al. [FYX+23] proponen un método de coordinación de red que formula el control semafórico como un problema multiobjetivo: combina eficiencia, seguridad y coordinación espacial entre intersecciones mediante un esquema de entrenamiento centralizado con ejecución descentralizada. Además de reportar resultados en redes sintéticas y arterias reales, el trabajo destaca que las dependencias espacio-temporales entre agentes deben representarse explícitamente. Su propuesta ofrece un contraste con la señal individual compuesta y la representación interpretable utilizadas en este trabajo, que no incorporan una optimización multiobjetivo ni una representación latente basada en atención.

Representación del estado

En cuanto a la representación agregada y el aprendizaje basado en valor, Kolat et al. [KKBA23] proponen un enfoque DQN multiagente con parámetros compartidos. Los agentes observan cantidades escaladas de vehículos detenidos y reciben una recompensa que penaliza el desequilibrio entre los accesos. Los resultados muestran mejoras en el tiempo de viaje y el consumo de combustible respecto de un controlador de ciclo fijo. Sirve como referencia para contrastar una representación agregada del estado y una coordinación inducida por la recompensa con la evaluación de IDQN y de métodos basados

en políticas realizada en esta investigación.

Dentro de la línea centrada en la representación del estado, Yang [Yan23] propone HG-M2I, una arquitectura que combina información espacial y temporal mediante un grafo jerárquico. El modelo integra estados de los agentes y de sus vecinos en representaciones latentes utilizadas por las políticas. Posteriormente, Chen et al. [CYL+24] presentan CoevoMARL, que utiliza atención en grafos y una unidad recurrente para aprender cómo varía en el tiempo la dependencia entre intersecciones. Mientras HG-M2I organiza la información según una jerarquía espacial, CoevoMARL adapta las relaciones entre agentes a la evolución del tránsito. Ambos trabajos muestran la importancia de construir observaciones informativas, aunque recurren a representaciones latentes aprendidas, a diferencia de la transformación explícita e interpretable que se evalúa aquí.

En el caso de una representación basada en celdas, Deng et al. [DLSZ26] introducen una partición de longitud variable. Los accesos se dividen mediante una función que combina componentes logarítmicos y lineales, y cada celda almacena la cantidad de vehículos, la velocidad media y la ocupación espacial. Esta partición asigna mayor detalle a las zonas próximas a la intersección y comprime las regiones más alejadas. Entre los trabajos seleccionados, es el antecedente más cercano al uso de una transformación logarítmica en la representación semafórica; sin embargo, el logaritmo determina la longitud espacial de las celdas y no codifica el tiempo de espera acumulado por carril.

Heterogeneidad y equidad

Ren et al. [RDZ+24] abordan la coordinación bajo demandas heterogéneas mediante un esquema de dos capas. Cada agente combina su ventaja local con la de las intersecciones vecinas a través de un factor de cooperación adaptable, que luego se ajusta según el rendimiento global de la red. El método considera la demora y las emisiones vehiculares, y coordina los objetivos locales y globales mediante ese factor adaptable. Este antecedente permite contrastar la coordinación adaptativa de las señales de aprendizaje con la heterogeneidad de políticas considerada por HAPPO, aunque no emplea su procedimiento de actualización secuencial.

En la coordinación mediante entrenamiento centralizado, Liu et al. [LZF+25] formulan el control semafórico como un juego parcialmente observable con restricciones y proponen VF-MAPPO. En este método, la equidad se expresa mediante una cota sobre el tiempo máximo de espera de los vehículos, mientras que un crítico centralizado con atención espaciotemporal estima tanto la eficiencia de la red como el cumplimiento de la restricción. Esta formulación resulta especialmente pertinente para esta investigación porque vincula con una variable temporal de operación el paradigma de entrenamiento centralizado con ejecución descentralizada. No obstante, el tiempo de espera se incorpora como restricción de optimización y no mediante una codificación no lineal de la observación.

En el tratamiento de agentes heterogéneos, otro enfoque es HAPS-PPO, presentado

por Lu et al. [LFYZ25]. El método completa con relleno las observaciones de distinta dimensión y agrupa a los agentes según sus espacios de acción, asignando cabezales de política específicos a cada grupo. Esto permite coordinar intersecciones con geometrías y conjuntos de fases diferentes.

Posicionamiento del proyecto

La selección analizada sugiere que el desempeño del control semafórico multiagente depende de tres decisiones relacionadas: el algoritmo utilizado para aprender y coordinar las políticas, la representación de las condiciones de operación y el tratamiento de intersecciones con demandas, geometrías o espacios de acción diferentes. Los enfoques basados en DQN proporcionan una referencia para el aprendizaje por valor. Los modelos de grafos construyen representaciones latentes de las relaciones espaciales, mientras que las variantes de MAPPO incorporan información global durante el entrenamiento. Los esquemas de cooperación adaptable o con múltiples cabezales atienden distintas formas de heterogeneidad. La codificación por carril y la discretización de variables de tránsito también cuentan con antecedentes en control semafórico [GR18]; aquí se emplean para representar la espera acumulada por carril mediante la transformación especificada en la metodología.

Entre los antecedentes considerados, HeteroLight aborda la heterogeneidad mediante una arquitectura de parámetros compartidos con módulos que extraen características generales y específicas de cada intersección [ZLFS24], mientras que MATLight se apoya en un marco de aprendizaje especular para agentes heterogéneos (*heterogeneous-agent mirror learning*) y emplea un Transformer para procesar secuencias de estados [ZWL24]. Son mecanismos distintos de la adaptación HAPPO implementada aquí: no se los presenta como equivalentes ni como evidencia de que HAPPO sea superior. Entre las fuentes consultadas no se localizó una evaluación conjunta de IDQN, IPPO, MAPPO y HAPPO bajo una misma formulación, ni una aplicación directa del HAPPO original al control de fases semafóricas urbanas. Esta observación delimita la búsqueda realizada y no permite concluir que tales estudios no existan.

En particular, este trabajo explora una codificación híbrida logarítmica-lineal del tiempo de espera acumulado por carril. Al asignar valores continuos a un conjunto finito de intervalos, la cuantización agrupa esperas dentro de una misma categoría y limita los valores representables; la saturación también reúne los valores que alcanzan la cota. En aprendizaje por refuerzo, una abstracción de estados puede simplificar el problema y conservar comportamiento cercano al óptimo cuando los estados agrupados son suficientemente similares, bajo las condiciones teóricas correspondientes [AHL16]. Ese resultado no constituye una garantía para esta codificación: aquí se transforman observaciones locales en un problema parcialmente observable y no se demuestra que los valores agrupados sean equivalentes para la política óptima. La resolución no uniforme del codificador asig-

na más niveles a esperas bajas, lo que permite distinguirlas con mayor detalle; es una hipótesis de diseño, pues la cuantización también puede descartar distinciones relevantes, en particular cerca de los umbrales entre intervalos.

Su efecto se examina como parte del sistema completo, sin un experimento que compare el mismo sistema con y sin esa transformación. Esta propuesta se diferencia de las observaciones agregadas utilizadas por los enfoques DQN, de las proyecciones latentes basadas en grafos y de la partición espacial de Deng et al. [DLSZ26], que divide los accesos y no codifica la espera acumulada por carril.

A su vez, la evaluación bajo IDQN, IPPO, MAPPO y HAPPO examina el desempeño de cuatro procedimientos completos de aprendizaje y coordinación, cada uno con su configuración ajustada, sobre una misma representación de observación. En MAPPO se utiliza un crítico centralizado compartido de múltiples salidas, condicionado por la concatenación de observaciones locales, con valores y ventajas calculados por agente. En HAPPO, las ventajas individuales estimadas mediante GAE se reponderan en las actualizaciones secuenciales de los actores; esta adaptación no aplica directamente el lema original de descomposición de ventajas. Los críticos estiman valores durante el entrenamiento en ambos procedimientos, pero no se atribuye al crítico la diferencia entre ellos ni se postula una actualización secuencial del crítico.

Dado que la comparación se basa en procedimientos completos, no es posible aislar el efecto directo de un solo elemento, como la actualización secuencial o el crítico centralizado. Determinar dicho impacto requeriría una retirada de componentes, evaluando el sistema sin esa pieza específica mientras los demás factores permanecen constantes. De este modo, la solución puede contrastarse con los antecedentes recientes en términos de representación, paradigma de entrenamiento y capacidad para tratar agentes heterogéneos. El capítulo 3 describe cómo se concreta esta propuesta en el diseño experimental.

3

Metodología

Este capítulo describe el diseño y los procedimientos para evaluar agentes que seleccionan fases semafóricas a partir de observaciones locales del tránsito. Presenta tres redes sintéticas y, por separado, la construcción del escenario de Mendoza y la comprobación de su operación con los programas semafóricos de la red; esta comprobación no evalúa políticas aprendidas.

El capítulo describe la búsqueda de configuraciones de entrenamiento y la comparación de dos variantes de MAPPO. También explica cómo el entorno transforma las mediciones en observaciones, ejecuta las fases elegidas y calcula recompensas; presenta las medidas de evaluación y remite los detalles de implementación y reproducción al apéndice A.

3.1. Diseño del estudio

El estudio incluye la búsqueda de configuraciones de entrenamiento para MAPPO, HAPPO, IPPO e IDQN y, como experimento independiente, una comparación de dos condiciones de MAPPO. En la fase 1 se exploró el peso de una señal de recompensa que incorpora información de intersecciones vecinas; en las configuraciones sintéticas seleccionadas de fase 2, ese peso se fijó en cero y se utilizó la señal local. La comparación independiente sí evaluó una condición que añadía a la observación información de intersecciones conectadas y combinaba su señal de recompensa con la local. Esta condición se examinó como alternativa y no se adoptó para el controlador final. En las pruebas con aprendizaje, las recompensas guían el entrenamiento y las métricas de tránsito describen el desempeño de las políticas durante la evaluación.

El diseño contempla la búsqueda de hiperparámetros para MAPPO, HAPPO, IPPO e

IDQN en las tres redes sintéticas y en la red urbana de Mendoza. En las redes sintéticas, los documentos disponibles describen dos fases de búsqueda: una exploración amplia (fase 1) y una búsqueda posterior con dominios reducidos (fase 2). Mendoza forma parte del alcance previsto, pero no se cuenta con un protocolo urbano que permita presentar una campaña de búsqueda como ejecutada.

El procedimiento A/C contrasta dos configuraciones de MAPPO en las tres redes sintéticas: A usa información local y C añade observaciones y recompensa vecinales. Los hiperparámetros de aprendizaje se mantienen fijos. Como ambas condiciones difieren a la vez en observación y recompensa, el contraste estima el efecto de esa configuración conjunta, no el aporte aislado de cada componente.

La sección 3.2 especifica la geometría, la demanda y las condiciones de referencia de las redes usadas en cada procedimiento.

La tabla 3.1 delimita el alcance de cada procedimiento.

Tabla 3.1: Escenarios y procedimientos del trabajo.

Componente	Alcance
Escenarios sintéticos	Tres redes utilizadas en la búsqueda de hiperparámetros y en la comparación de variantes de MAPPO.
Escenario urbano	Red de la Ciudad de Mendoza construida con conteos parciales; búsqueda de hiperparámetros incluida en el alcance previsto, sin campaña urbana documentada como ejecutada; comprobación operacional con sus programas semafóricos.
Búsqueda de hiperparámetros	MAPPO, HAPPO, IPPO e IDQN; fases 1 y 2 documentadas en las redes sintéticas. Mendoza forma parte del alcance previsto.
Comparación de variantes	Pruebas preliminares de observación y recompensa, seguidas de una comparación por pares entre las variantes local y vecinal de MAPPO.

3.2. Escenarios de simulación

La comparación A/C se realiza en tres redes sintéticas de SUMO [DLR26]. El alcance previsto de la búsqueda de hiperparámetros comprende esas redes y la red urbana de Mendoza, construida y comprobada por separado; las campañas documentadas en este capítulo corresponden a las redes sintéticas. Las redes se prepararon con `netedit`, `netconvert` y programas de Python.

Los nombres 3×1, 3×3 y $4 \times 4_H$ identifican, respectivamente, un corredor con tres intersecciones, una cuadrícula de nueve y una red de dieciséis intersecciones con geometría heterogénea. C1, C4 y D1 son identificadores de intersecciones de la última red.

3.2.1. Preparación de las redes

Para cada red se definieron las calzadas, los carriles por aproximación y las longitudes de almacenamiento, mediante `netedit` o información cartográfica de OSM. Después se revisaron los sentidos de circulación, las conexiones de giro, los carriles y las maniobras permitidas, contrastándolos con imágenes satelitales y vistas de calle. Se conservaron los programas semafóricos de los archivos de red. En las redes sintéticas, los programas son de tiempo fijo y sus duraciones se resumen en la tabla 3.2; los de Mendoza se describen en el apéndice A.4. Son referencias de la simulación, no mediciones de planes municipales.

Las secuencias de la tabla 3.2 siguen el orden de ejecución e incluyen las transiciones; su suma corresponde a la duración del ciclo.

Tabla 3.2: Duraciones de los programas semafóricos de referencia en las redes sintéticas (segundos).

Red e intersecciones	Secuencia de duraciones	Ciclo
3×1: todas (3)	42, 3, 42, 3	90
3×3: todas (9)	42, 3, 42, 3	90
$4 \times 4_H$: 13, excepto C1, C4 y D1	42, 3, 42, 3	90
$4 \times 4_H$: C1	38, 3, 37, 3, 6, 3	90
$4 \times 4_H$: C4 y D1	42, 3, 41, 3, 1	90

En la secuencia común, el verde dura 42 segundos y el amarillo, 3. C1, C4 y D1 tienen secuencias específicas por sus movimientos permitidos; C4 y D1 incluyen un segundo con todas las señales en rojo. Algunas transiciones mantienen el verde de movimientos que continúan habilitados. El control por agentes utiliza los tiempos de la sección 3.4.1.

3.2.2. Redes sintéticas

Todas las redes sintéticas comparten las siguientes reglas operacionales:

- **Límite de velocidad:** 13,89 metros por segundo (50 kilómetros por hora) en todas las arterias urbanas. La velocidad máxima de un tipo vehicular no elimina este límite vial.

- **Flota vehicular heterogénea:** se definieron tres tipos de vehículos en el archivo `urban_vehicle_mix.add.xml`, donde a_{veh} es la aceleración, d_{veh} la desaceleración y $v_{\text{máx}}$ la velocidad máxima. Los valores de a_{veh} y d_{veh} se expresan en metros por segundo cuadrado, y $v_{\text{máx}}$ en metros por segundo [DLR26]:

  1. `SLOWVEHTYPE` (25 %): perfil de menor aceleración, con $a_{\text{veh}} = 1{,}9$ y $d_{\text{veh}} = 4{,}0$ metros por segundo cuadrado, y velocidad máxima de $v_{\text{máx}} = 8{,}33$ metros por segundo (30 kilómetros por hora).

2. `MEDVEHTYPE` (60 %): perfil de referencia, con $a_{\text{veh}} = 2{,}6$ y $d_{\text{veh}} = 4{,}5$ metros por segundo cuadrado; $v_{\text{máx}} = 13{,}89$ metros por segundo (50 kilómetros por hora).
3. `FASTVEHTYPE` (15 %): perfil de mayor aceleración, con $a_{\text{veh}} = 3{,}1$ y $d_{\text{veh}} = 5{,}0$ metros por segundo cuadrado; $v_{\text{máx}} = 19{,}44$ metros por segundo (70 kilómetros por hora).

Los tres tipos comparten una longitud de 4,5 metros y una separación mínima entre vehículos de 2,5 metros, configurada mediante `minGap`. Se utiliza el modelo de seguimiento vehicular de Krauss, que determina cómo un vehículo adapta su avance al que lo precede. Su parámetro adimensional σ representa la imperfección del conductor y controla la variabilidad estocástica de la conducción; se fija en 0,5 [Kra98, DLR26].

- **Generación de demanda por zonas:** las rutas se generan mediante `random_routes_taz.py` a partir de matrices de zonas de análisis de tránsito (TAZ, del inglés *Traffic Analysis Zone*). Los tres escenarios sintéticos almacenan flujos con el atributo `vehsPerHour`; el valor solicitado se expresa en vehículos por hora y SUMO inserta los vehículos conforme a ese flujo. No se atribuye a estas redes un proceso de llegadas exponencial.
- **Ajuste de la demanda:** los flujos utilizados en las tres redes se resumen en la tabla 3.3. El registro disponible informa que, en 3×3, cerca del 70 % de los vehículos cargados completó su viaje; no permite atribuir el mismo comportamiento a 3×1 ni a $4 \times 4_H$. La comprobación de 3×3 incluyó 600 segundos de calentamiento —período inicial en que la simulación se ejecuta antes de medir la operación— y 3600 segundos de control, durante los cuales se registró el comportamiento de la red.

La figura 3.1 permite comparar la forma y la disposición de las intersecciones en las tres redes.

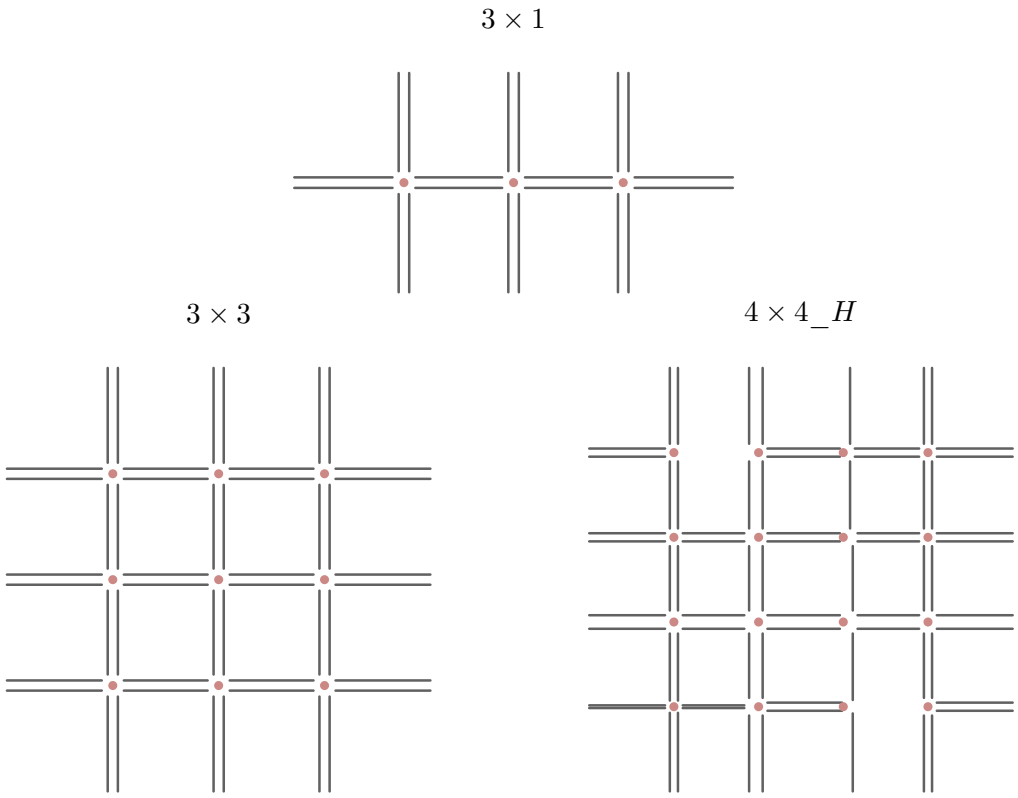


Figura 3.1: Geometría de las tres redes sintéticas. Los puntos marcan intersecciones semaforizadas; cada panel usa una escala propia.

La tabla 3.3 resume las características de las redes y el flujo definido en sus archivos de rutas.

Tabla 3.3: Características de las redes sintéticas y flujo.

Escenario	**Agentes**	**Tramos**	**Carriles**	**Flujo (vehículos por hora)**
3×1	3	20	40	3.260
3×3	9	48	96	7.087
$4 \times 4_H$	16	71	166	7.202

Las tres topologías sintéticas cubren geometrías y distribuciones de demanda distintas. En los pares que siguen, $o \to d$ indica una zona de origen y una de destino, y el porcentaje asigna a ese par una fracción del flujo solicitado.

- **Corredor arterial** 3×1 **(3.260 vehículos por hora solicitados):** tres intersecciones dispuestas en línea. El perfil de demanda asigna explícitamente 40 % al par $3 \to 4$ y 35 % al par $4 \to 3$; la parte restante se distribuye de forma reproducible entre los pares válidos restantes. Su disposición permite examinar la propagación longitudinal de colas y la coordinación de fases sobre las corrientes predominantes.
- **Red simétrica** 3×3 **(7.087 vehículos por hora solicitados):** nueve intersecciones dispuestas en una cuadrícula con accesos periféricos. El perfil de demanda

asigna 20 %, 17,5 %, 17,5 % y 20 % a los pares $5 \to 6$, $6 \to 5$, $10 \to 1$ y $1 \to 10$, respectivamente; el 25 % restante se distribuye entre los pares válidos.

- **Cuadrícula $4\times4_H$ (7.202 vehículos por hora solicitados):** dieciséis intersecciones con geometría heterogénea. El perfil de demanda asigna 20 %, 17,5 %, 17,5 % y 20 % a los pares $5 \to 9$, $6 \to 5$, $13 \to 1$ y $1 \to 13$, respectivamente; el 25 % restante se distribuye entre los pares válidos.

3.2.3. Red de la Ciudad de Mendoza

El escenario modela parte del microcentro de la Ciudad de Mendoza. Su construcción combina un trazado cartográfico, conteos parciales y supuestos para representar la demanda.

Datos de tránsito y alcance. El modelo es una aproximación: el conteo de un acceso informa sobre el volumen y su variación horaria en ese punto, pero no determina por sí solo los orígenes y destinos de todos los viajes. Por ello, la distribución espacial y el volumen total de la red incorporan supuestos explícitos [CyMR18, PW16].

Los registros disponibles contienen conteos y clasificaciones de cámaras térmicas ya instaladas en el entorno de Costanera y Acceso Este. Para este trabajo no se instalaron cámaras ni se realizaron mediciones nuevas. Los registros se organizan en intervalos de 15 minutos.

Se examinaron registros de cuatro dispositivos. Tres observan corrientes de tránsito fuera del área modelada; por ello, se utilizaron únicamente los de la cámara que registra el ingreso hacia el oeste por la Avenida José Vicente Zapata, sobre el canal Cacique Guaymallén.

Los registros de conteo de las cámaras termográficas abarcan aproximadamente de octubre de 2025 a abril de 2026. Para construir el perfil se usaron los datos de la cámara 169; los otros tres dispositivos registran corrientes que no representan ese ingreso.

El procesamiento descartó fechas no interpretables, identificadores `ZoneID` fuera del rango 1–19 y códigos `ClassID` distintos de 1, 2 y 3. `ZoneID` identifica una región virtual de conteo configurada en la cámara; `ClassID` identifica una clase por longitud: hasta 5 metros, de 5 a 10 metros y más de 10 metros, respectivamente. También se descartaron conteos negativos. Los tiempos se redondearon al intervalo de 15 minutos más cercano y los conteos se sumaron por cámara, intervalo, zona y clase. Se seleccionaron 33 días con todos los intervalos esperados, ocurridos de lunes a viernes, entre el 5 de diciembre de 2025 y el 9 de abril de 2026.

En esos 33 días, el promedio del conteo diario fue $\mu_{\text{obs}} = 25.283,85$ vehículos por día y su desviación estándar muestral —medida de variación entre los días seleccionados— fue $\sigma_{\text{obs}} = 2.316,54$ vehículos por día.

Representación y revisión de la red. El trazado geométrico base se obtuvo mediante la exportación de datos de OSM y su conversión al formato de red de SUMO (`.net.xml`). El dominio modelado delimita el microcentro de la Ciudad de Mendoza entre Av. José Vicente Zapata / Colón al sur, Av. General San Martín como eje central norte-sur, Av. Manuel Belgrano al oeste y Av. Las Heras al norte. Para ubicar los elementos del mapa se utiliza la proyección Sistema de coordenadas universal transversal de Mercator (*Universal Transverse Mercator*) (UTM), zona 19S (zona 19 del hemisferio sur), con el sistema de referencia Sistema Geodésico Mundial 1984 (*World Geodetic System 1984*) (WGS84).

La figura 3.2 muestra la red y ubica el tramo elegido para representar el ingreso registrado por la cámara 169.

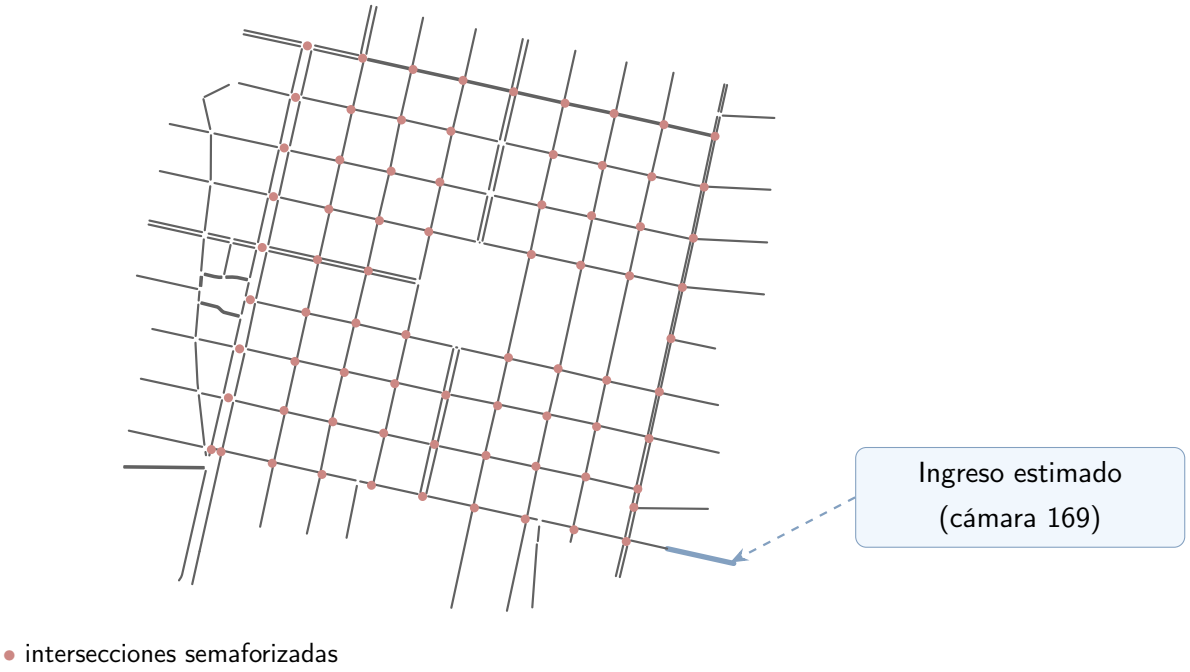


Figura 3.2: Red parcial del microcentro de Mendoza. Los extremos grises conectan con la red circundante; el tramo azul marca el ingreso estimado con el conteo de la cámara 169, que registra el flujo hacia el oeste por Av. José Vicente Zapata.

Como la cámara 169 queda fuera del límite modelado, los vehículos cuya demanda se estima a partir de su conteo se incorporan en el primer tramo interno correspondiente. Así se representa ese ingreso sin extender la red hasta el punto de medición.

Auditoría y corrección topológica con Google Maps y Street View: tras la conversión inicial desde OSM, se detectaron discrepancias entre la red y la infraestructura real. La red se revisó con imágenes satelitales de Google Maps y vistas de calle de Google Street View:

- **Restricciones de giro y maniobras prohibidas:** se implementaron las restricciones identificadas mediante la inspección de imágenes, como la prohibición de giro a la izquierda desde Av. General San Martín hacia el sur por Av. Colón. También

se conservaron los giros semaforizados permitidos, como el de calle Perú hacia Av. Las Heras.

- **Sentidos de circulación y conectividad vial:** se corrigieron los sentidos de circulación de calzadas secundarias que figuraban como bidireccionales o invertidas en OSM. También se restituyó la continuidad de los cuatro carriles de ingreso de la Avenida José Vicente Zapata hacia la trama interior.
- **Cantidad de carriles y accesos laterales:** se ajustó el número de carriles de aproximación por calzada y se habilitaron las conexiones de inserción en las calles transversales Catamarca, Buenos Aires y Leandro N. Alem.
- **Programas semafóricos:** se conservaron las secuencias de señales y los parámetros incluidos en el archivo de red. La configuración urbana combina programas fijos y programas que ajustan el verde según la detección de vehículos, como se detalla en la sección A.4. No se dispone de mediciones que permitan reconstruir los planes municipales.

Construcción de la demanda. La demanda se organiza por hora a partir del perfil promedio observado $\bar{q}_{\text{obs}}(h)$, donde h identifica una hora del día entre 0 y 23. El perfil se promedia sobre los 33 días completos de lunes a viernes. En los registros, el período diurno de 12 horas (06 : 00–18 : 00) concentra el 69, 29 % del flujo diario total, mientras que la ventana matutina de hora pico (06 : 30–08 : 30) concentra el 12, 53 %.

Para extender el perfil observado a toda la red se fijó como supuesto un volumen de referencia $V_{\text{diario}} = 260.000$ vehículos por día. Cada intervalo del perfil dura $\Delta h = 1$ hora y $\bar{q}_{\text{obs}}(h)$ se expresa en vehículos por hora. El factor de expansión f_{exp} relaciona el volumen diario supuesto con el total diario promedio del perfil:

$$f_{\text{exp}} = \frac{V_{\text{diario}}}{\sum_{h=0}^{23} \bar{q}_{\text{obs}}(h)\,\Delta h} = \frac{260.000}{25.283,85} \approx 10,2832 \tag{3.1}$$

El factor no tiene unidades, porque relaciona dos volúmenes expresados en vehículos por día. Los 260.000 vehículos son un supuesto de construcción, no una estimación identificada por el conteo de un único acceso. Para cada hora, el flujo solicitado para toda la red, $Q_{\text{red}}(h)$, se obtiene al escalar el perfil observado por ese factor:

$$Q_{\text{red}}(h) = \bar{q}_{\text{obs}}(h) \cdot f_{\text{exp}} \tag{3.2}$$

El resultado $Q_{\text{red}}(h)$ se expresa en vehículos por hora.

El parámetro `camera_retention` fija en 1 la fracción del conteo recibido que se conserva como referencia para el ingreso modelado. No se conoce cuántos vehículos observados llegan al área modelada ni cuántos se incorporan desde accesos no observados; por tanto, este valor es un supuesto de construcción, no una proporción medida.

Distribución de los viajes entre zonas. La red contiene 39 zonas de análisis de tránsito, identificadas de 0 a 38, y 1482 pares posibles entre zonas distintas. Se verificó la conectividad mediante búsqueda en anchura (*breadth-first search*, BFS), que recorre el grafo vial para identificar destinos accesibles; 520 pares pueden conectarse. Una matriz origen–destino (OD) indica qué proporción de los viajes se asigna a cada par de zonas.

De los 520 pares conectados, 473 tienen porcentajes fijados en la configuración, cuya suma representa el 60 % de los viajes. A otros 44 pares se les distribuye aleatoriamente el 40 % restante. Los tres pares restantes se excluyeron porque produjeron bloqueos durante la construcción del escenario. Esta decisión limita la demanda representada y no garantiza que otras simulaciones estén libres de bloqueos. Para formalizar la asignación, o y d denotan el origen y el destino; p_{od} es la proporción de viajes entre ambos. $\mathcal{P}_{\text{fijo}}$, $\mathcal{P}_{\text{excl}}$ y $\mathcal{P}_{\text{aleat}}$ son, respectivamente, los conjuntos de pares con porcentajes fijados, excluidos y asignados aleatoriamente. Para cada par aleatorio, u_{od} es un peso sorteado uniformemente entre cero y uno; p_{od}^{fijo} es el porcentaje definido en la configuración. La regla de asignación es:

$$p_{od} = \begin{cases} p_{od}^{\text{fijo}}, & (o,d) \in \mathcal{P}_{\text{fijo}}, \\ 0, & (o,d) \in \mathcal{P}_{\text{excl}}, \\ \dfrac{0,40\, u_{od}}{\sum_{(o',d') \in \mathcal{P}_{\text{aleat}}} u_{o'd'}}, & (o,d) \in \mathcal{P}_{\text{aleat}}. \end{cases} \tag{3.3}$$

En la tercera rama, los pesos aleatorios se normalizan para que las proporciones asignadas sumen 0,40; las proporciones fijadas suman 0,60.

Los porcentajes fijos priorizan los corredores Vicente Zapata, General San Martín, Colón–Emilio Civit, Las Heras y Manuel Belgrano, según la revisión cartográfica y los conteos disponibles.

Las rutas se generan con la semilla 104773, que controla la selección y generación de itinerarios; SUMO utiliza la semilla 104729 para inicializar la dinámica estocástica de la simulación. La primera se eligió entre las configuraciones que permitían operar doce horas sin activar el mecanismo *teleport* de SUMO; se seleccionó la que mejor se aproximaba a la distribución espacial buscada.

Llegadas y tipos de vehículos. En SUMO, los flujos por par origen–destino se generan con intervalos entre llegadas de distribución exponencial, como se describe en la ecuación (2.4). Para un par con flujo positivo, Q_{od} es el flujo solicitado en vehículos por hora y $\lambda_{od} = Q_{od}/3600$ su tasa equivalente en vehículos por segundo. Si U es un número aleatorio uniforme en $(0, 1)$, el tiempo H_{od} entre dos llegadas consecutivas, en segundos, se calcula como:

$$H_{od} = -\frac{\ln(U)}{\lambda_{od}}. \tag{3.4}$$

La flota combina un 25 % de vehículos lentos, un 60 % de vehículos estándar y un 15 % de vehículos con mayor aceleración. Si el acceso está ocupado, el vehículo ingresa cuando

hay espacio, aunque su llegada estuviera programada antes.

Comprobación operacional. La comprobación operacional forma parte de la preparación del escenario urbano: permite examinar si la red y la demanda construidas funcionan con los programas semafóricos conservados. Para ello se utilizaron dos simulaciones, sin política aprendida, en los siguientes intervalos:

1. **Período diurno (12 horas,** 06 : 00–18 : 00**):** horizonte de 43.200 segundos para representar la evolución del tránsito durante el día.

2. **Hora pico matutina (2 horas,** 06 : 30–08 : 30**):** intervalo de control comprendido entre $t_{\text{sim}} = 1800$ y $t_{\text{sim}} = 9000$ segundos, donde t_{sim} indica el tiempo de simulación.

La simulación estática de hora pico controla entre $t_{\text{sim}} = 1800$ y $t_{\text{sim}} = 9000$ (06:30–08:30); la configuración `mendoza_0630_0830.sumocfg` comienza en $t_{\text{sim}} = 1800$ y no añade calentamiento. La configuración MARL `mendoza_marl_2h.sumocfg` comienza en $t_{\text{sim}} = 1500$ (06:25) y reserva 300 segundos para calentamiento antes del mismo intervalo de control. Ambas parten de la demanda generada a partir del perfil de doce horas; la existencia de la configuración MARL no acredita que se haya ejecutado una campaña urbana. Los archivos se identifican en el apéndice A.4.

En ambos intervalos se registra cuántos vehículos incluye la demanda cargada, cuántos logran ingresar en la red y cuántos completan el viaje antes del cierre. También se calculan la tasa de inserción —vehículos ingresados sobre vehículos cargados— y la tasa de completitud de la demanda —viajes completados sobre vehículos cargados—, además de las activaciones del mecanismo de teletransporte de SUMO. Estas mediciones permiten comprobar si el escenario se mantiene operable y si la demanda logra ingresar y completar viajes; sus valores se presentan en el capítulo 4.

Comparación con el conteo observado. Para el mismo horario, V_{mod} denota el flujo horario nominal solicitado desde la zona 28 de análisis de tránsito (TAZ), asociada al acceso representado en la red, según las rutas generadas; V_{obs} es el flujo horario del perfil de la cámara 169. Ambos se expresan en vehículos por hora. El Estadístico de comparación de flujos de Geoffrey E. Havers (GEH) resume la diferencia entre estos flujos [PW16]:

$$\text{GEH} = \sqrt{\frac{2(V_{\text{mod}} - V_{\text{obs}})^2}{V_{\text{mod}} + V_{\text{obs}}}}. \tag{3.5}$$

El estadístico GEH vale cero cuando los flujos coinciden y aumenta a medida que difieren; no es un porcentaje. La comparación se refiere a la demanda programada, no a los vehículos que SUMO logró insertar ni a los que efectivamente cruzaron un detector. Como el mismo conteo se utilizó para construir el perfil, no constituye una validación independiente de la demanda de toda la ciudad.

3.3. Sistema de control multiagente

El paquete `marlTLC` integra el simulador SUMO con Ray RLlib, biblioteca que coordina la recolección de experiencias y las actualizaciones de aprendizaje [LLN+18], por medio del entorno `SumoRllibEnv`. Este entorno consulta y modifica la simulación mediante libsumo —interfaz de Python para SUMO—, organiza las mediciones como observaciones, aplica las acciones semafóricas y calcula las recompensas. PyTorch es la biblioteca que calcula y actualiza las redes neuronales. La figura 3.3 sitúa estas operaciones entre la simulación y el aprendizaje; las clases principales se describen en el apéndice A.

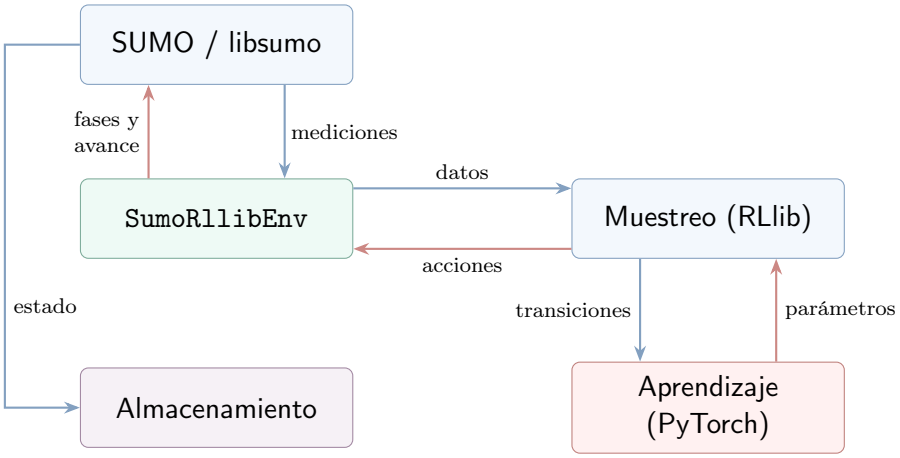


Figura 3.3: Flujo entre la simulación, el entorno y el aprendizaje. El estado de SUMO y los parámetros de las políticas se almacenan por separado.

3.3.1. Entorno de simulación

Con la topología de carriles, fases y conexiones, el entorno define los vecinos directos y los espacios de observación y acción de cada agente. El diagrama de clases y las responsabilidades de los componentes se describen en el apéndice A.1.

La tabla 3.4 resume las responsabilidades de las herramientas que intervienen en el flujo de información.

Tabla 3.4: Herramientas y responsabilidades en el sistema experimental.

Herramienta	Función
SUMO y libsumo	Simulación microscópica del tránsito.
`SumoRllibEnv`	Adaptación de la simulación a la interfaz multiagente.
Ray RLlib	Organización de los procesos de muestreo y aprendizaje.
PyTorch	Cálculo de redes neuronales y sus actualizaciones.

Al reiniciar, el entorno restaura el estado de SUMO guardado al terminar el calentamiento y entrega a cada agente su observación. En cada paso recibe las acciones, avanza

la simulación hasta la decisión siguiente y devuelve observaciones, recompensas e indicadores de finalización o truncamiento; este último señala que el episodio se interrumpió al alcanzar el límite temporal, sin que se haya producido una condición terminal del entorno. Las ejecuciones se realizan sin interfaz gráfica.

3.3.2. Observación, acciones y recompensas

MAPPO y HAPPO utilizan un crítico centralizado que recibe las observaciones conjuntas durante el entrenamiento; al ejecutar, cada actor selecciona fases a partir de la información disponible para su intersección. La entrada conjunta concatena observaciones locales; no es el estado físico completo de la red. La figura de CTDE del marco teórico (figura 2.19) presenta el principio general.

Observaciones y codificación de la espera

El codificador $e(x)$ transforma la espera x de cada carril en uno de I niveles enteros, donde I es un hiperparámetro y la salida pertenece a $\{0, \ldots, I-1\}$. El umbral de saturación por carril, M_{lane}, se expresa en vehículo·segundos (veh·s) y se fija por escenario. La observación local $o_i^{(t)}$ recibida por el agente i en el paso t combina el índice $p_i^{(t)}$ de la fase semafórica activa con las esperas codificadas de sus carriles controlados. Aquí m_i es el número de esos carriles, l identifica uno de ellos y $w_{i,l}^{(t)}$ es la suma, en veh·s, de las esperas acumuladas de los vehículos presentes que el entorno atribuye al carril l. Esa atribución evita volver a contar en otro carril la espera de un vehículo que cambió de carril. En las fases 1 y 2 de búsqueda se usa esta información local. Las políticas no conservan un historial de observaciones:

$$o_i^{(t)} = \left(p_i^{(t)}, e(w_{i,1}^{(t)}), \ldots, e(w_{i,m_i}^{(t)})\right), \tag{3.6}$$

SUMO obtiene esta magnitud mediante `getAccumulatedWaitingTime`, que conserva la espera en una ventana temporal finita; esa ventana se denomina memoria de espera [Ecl26a]. Su duración se detalla junto con los parámetros de SUMO en la sección A.4. La espera consecutiva de un vehículo detenido es otra magnitud. También lo es el tiempo de espera total por viaje registrado en el archivo `TripInfo`, que reúne datos de cada recorrido; SUMO lo almacena en el campo `waitingTime`. La salida de vehículos de un carril, los cambios de carril y el vencimiento de la ventana pueden reducir la señal observada sin que ello equivalga a reducir la espera total del conjunto de vehículos evaluado.

Para calcular $e(x)$, la implementación expresa la espera en vehículo·segundos. Sea $u = 1\,\text{veh·s}$ esa unidad; $x_{\text{num}} = x/u$ es el valor numérico de la espera $x \geq 0$, y $M_{\text{num}} = M_{\text{lane}}/u > 0$ es el umbral en la misma convención. La espera normalizada es $\hat{x} = \text{mín}(1, x_{\text{num}}/M_{\text{num}}) \in [0, 1]$. Para $I \geq 2$, se define $L = (I-1)/M_{\text{num}}$; en los dominios utilizados, $M_{\text{num}} > I - 1$, de modo que $0 < L < 1$. La función g combina un componente

logarítmico y otro lineal; $\log_2$ denota el logaritmo en base dos:

$$g(\hat{x}) = (1 - L)\frac{\log_2\left(\frac{\hat{x}}{L} + 1\right)}{\log_2\left(\frac{1}{L} + 1\right)} + L\hat{x}, \tag{3.7}$$

El parámetro adimensional L regula numéricamente la curvatura y la ponderación lineal; depende de la convención de unidades y se usa como coeficiente de la transformación. La salida se escala por el índice máximo y se redondea hacia arriba al entero más próximo, operación que se escribe $\lceil z \rceil$ o ceil(z); por ejemplo, $\lceil 2{,}3 \rceil = 3$ y $\lceil 2 \rceil = 2$. Finalmente, se limita el resultado a $I - 1$:

$$e(x) = \text{mín}\{I - 1, \text{ceil}((I - 1)g(\hat{x}))\}. \tag{3.8}$$

La expresión algebraica equivalente a la forma computada por el código y el tratamiento de valores que alcanzan el umbral se presentan en el apéndice A.6.

En esta formulación:

- **Resolución del codificador (I):** hiperparámetro `encoder_intervals`, muestreado según el dominio de cada combinación de algoritmo y escenario; fija cuántos niveles puede recibir la política.
- **Umbral de saturación (M_{lane}):** corresponde al parámetro `max_lane_value`, fijado por escenario y expresado en veh·s. Para estimarlo se realizaron simulaciones previas y se tomó el percentil 99 —el valor que deja por debajo al 99 % de las observaciones— de la mayor espera entre carriles en cada paso, después de 300 segundos de calentamiento. Los valores sintéticos son 1184, 1136 y 712 veh·s para 3×1, 3×3 y $4 \times 4_H$, respectivamente. Este criterio operativo fija la escala a partir de esperas elevadas.

La figura 3.4 muestra la codificación de la observación local. El ejemplo usa $I = 22$ solo con fines ilustrativos y $M_{\text{lane}} = 1136$ veh·s, estimado para 3×3; la comparación A/C utiliza $I = 17$.

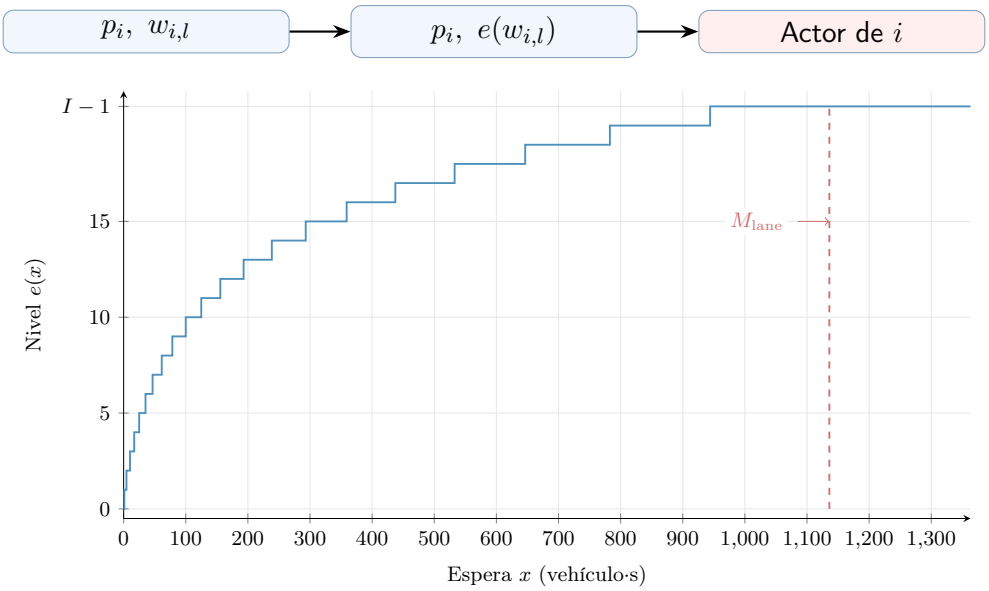


Figura 3.4: La observación conserva la fase activa y codifica la espera de cada carril en niveles discretos.

El umbral local se fijó mediante simulaciones previas, sin seleccionar una política por su rendimiento. Una vez construida la observación, el agente selecciona una fase permitida por una máscara de acciones, regla que excluye las fases que la señalización vigente no habilita.

Acciones permitidas y cambios de fase

El agente i puede elegir entre K_i fases verdes, representadas por el conjunto de acciones $\mathcal{A}_i = \{0, 1, \ldots, K_i - 1\}$. Cada acción $a \in \mathcal{A}_i$ identifica una fase permitida por la geometría y los movimientos del cruce. Cada agente tiene su propia política, sin compartir parámetros con los demás. En el instante t, $\mathcal{A}_i^{\text{adm}}(t) \subseteq \mathcal{A}_i$ es el subconjunto habilitado por la señal vigente, y $b_i(a, t)$ vale uno si a pertenece a ese subconjunto y cero en caso contrario. La red asigna a cada acción una puntuación real sin normalizar $z_i(a, t)$, llamada *logit*; la función *softmax* transforma esas puntuaciones en probabilidades que suman uno. La puntuación después de aplicar la máscara se denota $\widetilde{z}_i(a, t)$. El módulo `ActionMaskingTorchRLModule` suma al puntaje el logaritmo natural de la máscara:

$$\widetilde{z}_i(a, t) = z_i(a, t) + \log b_i(a, t), \qquad b_i(a, t) = \begin{cases} 1 & \text{si } a \in \mathcal{A}_i^{\text{adm}}(t) \\ 0 & \text{si } a \notin \mathcal{A}_i^{\text{adm}}(t) \end{cases} \tag{3.9}$$

Así, la puntuación enmascarada $\widetilde{z}_i(a, t)$ asigna probabilidad cero a una fase no habilitada: en la notación matemática, $\log(0) = -\infty$; el código reemplaza ese valor por una puntuación finita muy baja. En IPPO, MAPPO y HAPPO se restringen las puntuaciones de la política; IDQN descarta las acciones no permitidas al seleccionar y evaluar fases. El agente conserva la fase vigente durante el verde mínimo de cinco segundos y durante

la transición amarilla de cuatro segundos. La máscara limita las fases que puede elegir; SUMO ejecuta el intervalo amarillo antes de activar la fase siguiente.

Recompensas locales y vecinales

El coeficiente $\rho_{\text{coop}} \in [0, 1)$ determina cuánto pesa la señal vecinal en la recompensa combinada. En la búsqueda documentada de fase 1 se muestreó en $[0, 0{,}95]$, sin incluir exactamente cero. Cada agente recibió observaciones locales. El programa de búsqueda documentado combina como señal base la diferencia de espera acumulada y el promedio simple de las señales locales vecinales, sin dividirlas por el número de carriles. El código consultado deja por defecto la normalización por carriles desactivada. Esta evidencia describe la configuración disponible del programa y no verifica por separado todos los ensayos históricos.

En las combinaciones sintéticas documentadas de fase 2, se fijó $\rho_{\text{coop}} = 0$ y se utilizó solo la señal local. En la variante C de A/C, la señal vecinal se divide por el número de carriles de cada vecino antes de promediarla. La tabla A.7 del apéndice resume las configuraciones documentadas para cada procedimiento.

La señal local $L_i^{(t)}$ compara la espera acumulada en los carriles de una intersección entre dos pasos de tiempo. Si $W_i^{(t)}$ es la suma, de los tiempos de espera acumulados de los vehículos sobre los carriles de la intersección (agente) i, $L_i^{(t)}$ se calcula como la diferencia entre esa suma, en el paso de tiempo anterior y el actual.

1. **Señal local (L_i):** la recompensa es positiva cuando disminuye la suma de los tiempos de espera:
$$L_i^{(t)} = W_i^{(t-1)} - W_i^{(t)} \tag{3.10}$$

2. **Señal vecinal normalizada por carriles para C ($N_{i,\text{carril}}$):** sea $\mathcal{N}(i)$ el conjunto de vecinos topológicos directos del agente i, y m_j el número de carriles controlados por cada vecino j. Si hay vecinos, sus señales locales se promedian después de dividir cada una por su número de carriles; si no los hay, $N_{i,\text{carril}}^{(t)} = 0$:
$$N_{i,\text{carril}}^{(t)} = \begin{cases} \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} \frac{L_j^{(t)}}{m_j}, & |\mathcal{N}(i)| > 0, \\ 0, & |\mathcal{N}(i)| = 0. \end{cases} \tag{3.11}$$

3. **Mezcla vecinal utilizada en C:** la recompensa resultante $R_i^{(t)}$ combina la señal local y la señal vecinal normalizada por carriles con los pesos $1-\rho_{\text{coop}}$ y ρ_{coop} cuando hay vecinos; si no los hay, conserva la señal local completa:
$$R_i^{(t)} = \begin{cases} (1-\rho_{\text{coop}}) L_i^{(t)} + \rho_{\text{coop}} N_{i,\text{carril}}^{(t)}, & |\mathcal{N}(i)| > 0, \\ L_i^{(t)}, & |\mathcal{N}(i)| = 0. \end{cases} \tag{3.12}$$

En la variante C de la comparación, ρ_{coop} se fijó en $0,1$. Cada agente recibe su propia recompensa; las recompensas no se suman en un único retorno para toda la red.

3.3.3. Agentes y algoritmos de aprendizaje

RLlib vincula un módulo de aprendizaje con cada intersección. En los métodos basados en PPO, cada intersección tiene su propio actor; IDQN aprende valores de acción y deriva de ellos la regla de selección. La figura 3.5 distingue la información utilizada durante el entrenamiento y la ejecución.

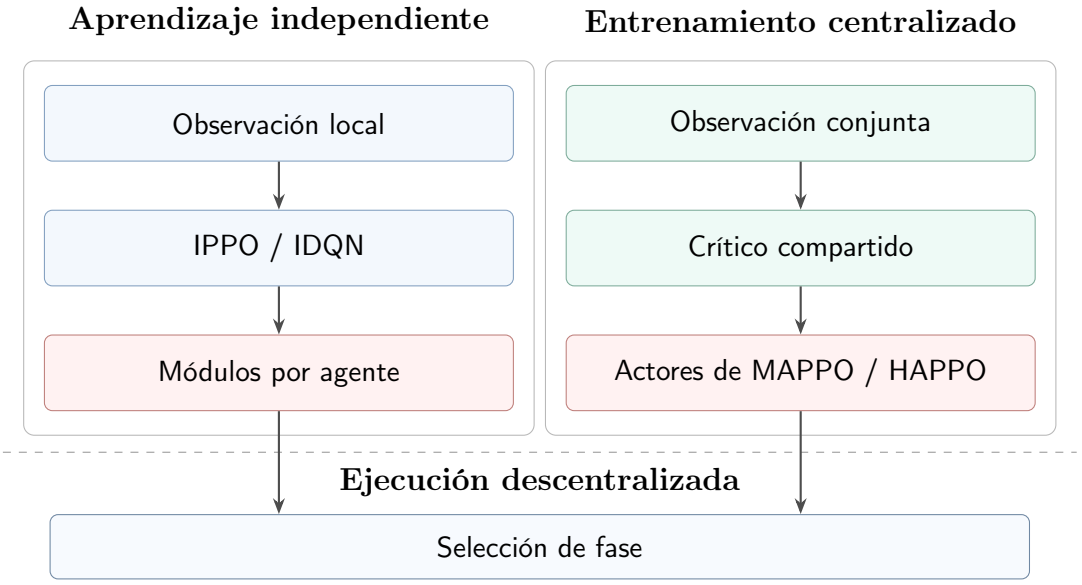


Figura 3.5: Información de los algoritmos durante el entrenamiento y la ejecución. El crítico compartido de MAPPO y HAPPO no interviene al ejecutar.

El apéndice describe la pérdida del crítico, el procesamiento de los lotes y otras operaciones de actualización que completan el recorrido de aprendizaje.

Aprendizaje independiente y centralizado

- **IDQN e IPPO (aprendizaje independiente):** IPPO mantiene un actor y un crítico locales por intersección; IDQN utiliza una red local de valores de acción y selecciona entre las fases permitidas según esas estimaciones.
- **MAPPO con crítico centralizado:** cada intersección tiene un actor propio que, durante la ejecución, utiliza su observación local. En el entrenamiento, un crítico compartido recibe la concatenación de las observaciones locales y produce un valor para cada agente. Esta entrada conjunta no representa el estado completo de la red. Para cada agente se calculan por separado la ventaja y el valor objetivo del crítico; luego se normaliza la ventaja de cada actor.
- **HAPPO:** el marco teórico describe el factor M y su actualización secuencial en la sección 2.3.6. En esta implementación, cada factor comienza con la ventaja indi-

vidual estandarizada del agente, calculada por el crítico centralizado con múltiples salidas. Los actores se actualizan uno tras otro; la razón entre las probabilidades de acción anteriores y actualizadas modifica los factores de quienes siguen pendientes. A diferencia de la formulación teórica, se utilizan ventajas individuales, se procesa el lote completo y se realiza un paso de gradiente por actor en cada *epoch*. El parámetro `greedy_selection_prob` controla la probabilidad de elegir al agente pendiente con mayor magnitud media del factor M; de lo contrario, se elige uno al azar. Regula el orden de actualización, no la fase semafórica. El coeficiente `kl_coeff` pondera la penalización por la diferencia entre las distribuciones de acción antes y después de actualizar; el código lo adapta hacia el objetivo `kl_target`.

En MAPPO y HAPPO, la pérdida del crítico limita el error cuadrático agregado, en lugar de recortar las predicciones de valor como en la variante habitual de PPO. Cuando el error agregado supera la cota, la pérdida queda saturada y su derivada respecto de ese error es cero; por tanto, esa muestra no aporta gradiente a la pérdida del crítico en esa región. No se midió con qué frecuencia se activa ni su efecto en el desempeño. La operación se detalla en el apéndice A.6.

3.4. Entrenamiento y búsqueda de hiperparámetros

La optimización de hiperparámetros (HPO) explora configuraciones dentro de un presupuesto de entrenamiento; cada ensayo es el entrenamiento de una configuración propuesta. Optuna administra las propuestas y sus resultados; su estimador de Parzen estructurado en árbol (*Tree-structured Parzen Estimator*, TPE) propone valores a partir del historial [ELR23, BBBK11, ASY+19]. Ray Tune ejecuta y registra los ensayos. En ambas fases, el algoritmo asíncrono de reducción sucesiva a la mitad (*Asynchronous Successive Halving Algorithm*, ASHA) compara el rendimiento intermedio y puede detener ensayos antes de completar su presupuesto [LJR+20]. En ASHA, el período de gracia fija cuántas iteraciones mínimas se ejecutan antes de considerar una detención; el factor de reducción determina cuánto disminuye, en cada nivel de comparación, la cantidad de ensayos que continúa.

3.4.1. Condiciones de simulación y semillas

Los horizontes de simulación dependen de la campaña y del escenario; la tabla 3.5 resume el calentamiento y la duración del control. Las comprobaciones de Mendoza con los programas semafóricos de la red no añaden calentamiento. Para la configuración MARL urbana, las dos horas de control corresponden al intervalo 06:30–08:30, precedido por cinco minutos de calentamiento.

Tabla 3.5: Períodos de calentamiento y control de las campañas.

Campaña	Calentamiento	Duración del control
HPO en redes sintéticas (fases 1 y 2)	600 s	3600 s
Cribado exploratorio de MAPPO	300 s	1800 s
Comparación A/C de MAPPO	300 s	3600 s
Mendoza con programas de la red, 12 h	No se añade	43200 s
Mendoza con programas de la red, 2 h	No se añade	7200 s
Configuración MARL de Mendoza, 2 h	300 s	7200 s

En las campañas de aprendizaje, el agente puede solicitar una fase cada cinco segundos. Un cambio solicitado activa cuatro segundos de amarillo; el verde debe mantenerse al menos cinco segundos antes de que pueda solicitarse otro cambio. El entorno conserva el estado de SUMO al terminar el calentamiento y lo restaura al iniciar cada episodio. Los parámetros aprendidos se almacenan por separado en un archivo de modelo. Las dos comprobaciones de Mendoza con los programas de la red y la configuración MARL de dos horas usan la misma demanda derivada del perfil de doce horas, con origen a las 06:00; difieren en el inicio de la simulación y en si incluyen calentamiento.

En estas campañas, la semilla de aprendizaje controla las fuentes aleatorias del algoritmo y la inicialización de sus redes; la de SUMO, la dinámica estocástica del simulador; la de rutas, la selección y generación de itinerarios; y la de TPE, la secuencia de propuestas. En la fase 1, la semilla TPE es independiente de la asignada al aprendizaje. Las semillas verificables por escenario se detallan en el apéndice.

3.4.2. Fase 1: búsqueda amplia

La fase 1 documentada exploró MAPPO, HAPPO, IPPO e IDQN en las tres redes sintéticas y usó h128 (dos capas ocultas de 128 unidades) y h256 (dos de 256 unidades): 24 estudios en total. Cada estudio admitió hasta 100 propuestas y cada ensayo, 40 iteraciones. ASHA usó una gracia de 10 iteraciones y un factor de reducción 4; el seguimiento diagnóstico, que ejecuta la política sin muestrear acciones, se realizó cada diez iteraciones. Sus resultados orientaron los dominios y las arquitecturas de la fase 2.

3.4.3. Fase 2: dominios reducidos

La fase 2 reduce los dominios de búsqueda bajo recompensa local ($\rho_{\text{coop}} = 0$) y mantiene la máscara de acciones. En las tres redes sintéticas, el presupuesto es de 100 configuraciones por cada combinación de algoritmo y escenario, contando los ensayos previos compatibles de fase 2: cuatro algoritmos en tres escenarios, 1200 configuraciones en total. Cada configuración puede completar hasta 60 iteraciones. Los fallos, las detenciones anticipadas y las interrupciones consumen parte de ese presupuesto. El archivo de estado final

del modelo conserva los parámetros aprendidos para reanudar o evaluar el entrenamiento; a este archivo se lo denomina *checkpoint*. Su existencia no libera presupuesto si el ensayo está incompleto. El número contabilizado no certifica una política apta para selección.

La fase 2 en las redes sintéticas usa ASHA, con una gracia de 30 iteraciones y un factor de reducción 2, una ejecución concurrente por combinación y un episodio de seguimiento sin exploración cada 15 iteraciones. Cada ensayo dispone de 600 segundos de calentamiento y 3600 segundos de control. Aunque Mendoza forma parte del alcance previsto, no se presenta una campaña HPO urbana como ejecutada; los presupuestos y semillas de esta fase corresponden solo a las redes sintéticas. Las fuentes históricas con rangos o arquitecturas distintos conservan su identidad y no se agregan como si procedieran de un estudio homogéneo.

La figura 3.6 resume el paso de la exploración amplia a la búsqueda con intervalos acotados.

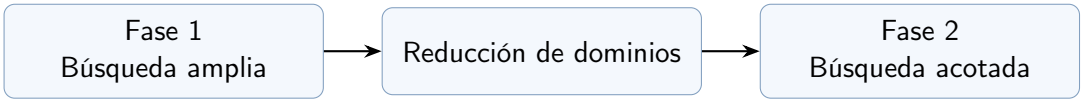


Figura 3.6: Búsqueda en dos fases: la primera orienta la reducción de dominios de la segunda.

3.4.4. Objetivo de búsqueda y evaluación diagnóstica

El objetivo de búsqueda combina la espera media actual con una penalización por los aumentos recientes. Así, una configuración que mantiene una espera baja y estable recibe una puntuación favorable.

Sea w_r la espera media por vehículo insertado en la medición r, calculada con TripInfo y expresada en segundos por vehículo. El historial incorpora las mediciones con valores numéricos válidos y finitos, es decir, definidos y no infinitos. Se descartan las primeras cinco mediciones disponibles, aparte del calentamiento de SUMO. Después se toman las últimas diez mediciones disponibles del historial restante y se calculan las diferencias consecutivas; las penalizaciones se registran como cero.

Sea n la cantidad de diferencias consecutivas disponibles tras descartar las primeras cinco mediciones y tomar hasta las diez últimas del historial restante. Se ordenan de mayor a menor los n aumentos resultantes, incluidos los ceros; $X_{(j)}$ denota el valor que ocupa la posición j en ese orden y $k = \lceil 0{,}2n \rceil$ es la cantidad de aumentos mayores que se promedia. Este promedio empírico de la cola superior se inspira en el Valor en riesgo condicional (*Conditional Value at Risk*) (CVaR), una medida que promedia los resultados de la cola más desfavorable de una distribución. Aquí se aplica a los aumentos de la espera media entre iteraciones, no a los viajes ni a las recompensas. Por el tamaño finito de la ventana y el redondeo de k, no es necesariamente una estimación exacta del CVaR de una distribución poblacional. Se define X_r como el aumento no negativo entre la medición

actual w_r y la anterior w_{r-1}, y T_r como la penalización promedio de esos aumentos; el objetivo de búsqueda J_r suma w_r y T_r:

$$\begin{aligned} X_r &= \text{máx}(0, w_r - w_{r-1}), \qquad k = \lceil 0{,}2n \rceil, \\ J_r &= w_r + T_r, \qquad T_r = \begin{cases} \frac{1}{k}\sum_{j=1}^{k} X_{(j)}, & n > 0, \\ 0, & n = 0. \end{cases} \end{aligned} \tag{3.13}$$

Las magnitudes w_r, X_r, T_r y J_r se expresan en segundos por vehículo. La búsqueda minimiza J_r; la recompensa del agente se calcula por separado.

En los episodios de seguimiento ya descritos, la política actúa sin muestrear acciones: IPPO, MAPPO y HAPPO eligen la fase permitida con mayor probabilidad; IDQN selecciona la fase permitida con mayor valor estimado. Estos episodios permiten observar el comportamiento durante el entrenamiento, pero no constituyen una evaluación independiente.

3.4.5. Selección de configuraciones de HPO

Los criterios cuantitativos siguientes corresponden a las doce combinaciones sintéticas de fase 2 documentadas en este capítulo y el apéndice; no definen el presupuesto ni las semillas de la búsqueda de Mendoza. Antes de evaluar una configuración sintética, se comprueba que el ensayo haya alcanzado las 60 iteraciones, que sus métricas sean numéricas y finitas —no ausentes ni infinitas— y que incluya al menos 15 registros de métricas de tránsito con valores numéricos finitos. También se verifica que la configuración efectiva y el archivo final del modelo estén completos y sean legibles. El estado `COMPLETE` de Optuna solo indica que el ensayo terminó sin registrar un fallo; por sí mismo no demuestra que alcanzara las 60 iteraciones ni que los archivos estén completos. Por combinación de algoritmo y escenario sintético se consideran hasta tres candidatos técnicamente admisibles, procurando incluir ambas arquitecturas cuando sea posible. Para cada combinación se construye una configuración de referencia a partir de los cinco mejores ensayos de fase 1, ordenados por el objetivo de búsqueda: se usa la mediana de cada parámetro numérico —el valor central al ordenar las observaciones— y la moda de cada parámetro categórico —la categoría más frecuente—. Esa configuración se entrena desde cero; no se importan puntuaciones, estados del modelo ni bases de Optuna de fase 1.

La selección sintética evalúa cada archivo de estado final del modelo con tres semillas SUMO comunes para todos los algoritmos de un mismo escenario: 950001, 950003 y 950007. Cada episodio debe registrar cero teletransportes, una Tasa de completitud de vehículos insertados (*Completion of Inserted Vehicles*) (CI) —viajes completados divididos por vehículos insertados— de al menos 0,95 y un número de vehículos insertados no inferior al 95 % de la referencia empírica del escenario y la semilla. La referencia es el máximo de vehículos insertados observado en el conjunto inicial de episodios de evaluación

usado para fijar este umbral; se mantiene igual al comparar candidatos y no constituye una medición independiente de la demanda total.

Entre los candidatos sintéticos que cumplen los tres criterios se elige el de menor espera media en las tres evaluaciones. En caso de empate exacto, se priorizan la CI media más alta, el menor tiempo observado de entrenamiento y, después, el identificador del ensayo. Si falta evidencia para completar la auditoría o el conjunto de evaluaciones de una combinación sintética, se informa como incompleta; si hay candidatos evaluables pero ninguno cumple los criterios, se informa que no hay candidato admisible. No se inicia otra búsqueda ni un entrenamiento adicional de una configuración definitiva.

El resultado sintético se interpreta como la mejor configuración observada dentro del presupuesto documentado, no como un óptimo global. Este procedimiento selecciona configuraciones de HPO; no las valida mediante nuevos entrenamientos independientes.

3.5. Comparación de variantes locales y vecinales de MAPPO

Antes de la comparación ampliada se realizó un cribado exploratorio —prueba preliminar para elegir qué formulación evaluar después, no confirmatoria— de cinco formulaciones en la red $4 \times 4_H$, con MAPPO de arquitectura h128 y tres semillas de entrenamiento, durante veinte iteraciones por política. Cada episodio duró 1800 segundos, después de 300 segundos de calentamiento. Se aplicaron las mismas semillas a las cinco formulaciones; las repeticiones en el entorno local y en el clúster de cómputo Toko (grupo de servidores que ejecuta trabajos en paralelo) no se contaron como semillas independientes adicionales.

La tabla 3.6 resume las cinco formulaciones. La observación local contiene las señales de la propia intersección; la vecinal incorpora, para cada vecino directo, información agregada sobre su espera. En B, C y D, $\rho = 0{,}1$ es el coeficiente que pondera la contribución vecinal y $1 - \rho = 0{,}9$ el que pondera la local; estos coeficientes no representan proporciones efectivas de la recompensa porque las señales pueden tener escalas distintas. La variante D añade a la observación el logaritmo natural de uno más el número de carriles del vecino y utiliza la misma regla de recompensa que C. E mantiene la observación local; la tabla define su señal de recompensa.

Tabla 3.6: Variantes examinadas en el cribado exploratorio.

Variante	Observación	Recompensa
A	Local: fase activa y señales de carriles propios.	Diferencia local de espera acumulada.
B	Local y suma de espera acumulada de cada vecino.	Diferencia local y vecinal sin normalización por carriles; $\rho = 0{,}1$.
C	Local y vecinal: espera media por carril de cada vecino.	Diferencia local y vecinal normalizada por carriles; $\rho = 0{,}1$.
D	Local, suma de espera vecinal y logaritmo natural (de base e) de uno más el número de carriles del vecino.	Igual que C.
E	Local: fase activa y señales de carriles propios.	Negativo de la diferencia absoluta entre el número de vehículos detenidos en carriles de entrada y el de los carriles de salida; esta señal se denomina recompensa de presión local.

La evaluación ampliada comparó dos variantes de MAPPO con la misma arquitectura y los mismos hiperparámetros. Ambas usan un crítico centralizado durante el entrenamiento.

La condición C se conserva como brazo experimental de la comparación A/C; la configuración final del controlador utiliza observación y recompensa locales. En C, la observación añade para cada vecino topológico directo la media por carril de su espera acumulada. El modelo identifica como vecinos las intersecciones cuyos carriles controlados están unidos por una conexión vial. Si W_j es la suma de las esperas acumuladas atribuidas a los carriles del vecino j y m_j es su cantidad de carriles, la entrada vecinal se codifica como $e(W_j/m_j)$ con un umbral propio M_{vec}. Este umbral normaliza y limita la señal antes de cuantizarla; se expresa en veh·s por carril y se estimó mediante el percentil 99 de los valores por intersección y paso en simulaciones previas. Los valores y las semillas de calibración se presentan en el apéndice.

En C, $R_{i,C}^{(t)}$ denota la recompensa recibida en el paso t por el agente i. Se calcula con la señal local $L_i^{(t)}$ y el promedio de las señales de sus vecinos, dividido por el número de carriles controlados por cada vecino. La forma explícita usa $\rho_{\text{coop}} = 0{,}1$ y se aplica cuando $|\mathcal{N}(i)| > 0$:

$$R_{i,C}^{(t)} = 0{,}9L_i^{(t)} + 0{,}1\frac{1}{|\mathcal{N}(i)|}\sum_{j\in\mathcal{N}(i)}\frac{L_j^{(t)}}{m_j}, \qquad |\mathcal{N}(i)| > 0. \tag{3.14}$$

Si el agente no tiene vecinos, recibe la señal local completa, como se establece en la

ecuación (3.12).

En cada una de las tres redes se entrenaron 24 pares A/C, cada par con la misma semilla de entrenamiento para ambas políticas. El tamaño se fijó tomando como referencia una desviación estándar pareada histórica aproximada de dos segundos por vehículo, que daba una semiamplitud t esperada cercana a 0,85 segundos por vehículo para el intervalo del 95 %. Es una referencia de planificación, no una garantía de potencia ni un resultado de esta campaña.

Cada política se entrenó durante cuarenta iteraciones y se evaluó cinco veces a partir del modelo de la última iteración. En estas evaluaciones, los contadores se reinician tras restaurar el estado posterior al calentamiento. La CI cuenta solo vehículos insertados durante el control, y `TripInfo` filtra esos viajes por su instante real de inserción en $[300, 3900)$.

En total se reunieron 72 pares, 144 políticas y 720 episodios de evaluación. Los hiperparámetros se mantuvieron fijos entre redes; solo se recalcularon los umbrales de las observaciones. Los valores y las semillas están en el apéndice A.6.1, y el análisis de los pares se explica en la sección 3.6.2.

El análisis principal comprende los 48 pares de 3×1 y $4 \times 4_H$. Los 24 pares de 3×3 se conservan y se presentan por separado. Esta exclusión del análisis principal se decidió después de observar los resultados; se trata como una limitación y no se atribuye a un mecanismo causal.

3.6. Métricas y análisis de las evaluaciones

La evaluación utiliza registros vehiculares y medidas por intersección calculadas durante la ventana controlada. La recompensa guía el aprendizaje, el objetivo J permite comparar ensayos de HPO y las métricas siguientes describen episodios ejecutados sin actualizar la política.

3.6.1. Medidas de desempeño y vehículos evaluados

La medida principal es la espera media durante la ventana de evaluación $[t_{\text{inicio}}, t_{\text{fin}})$, donde t_{inicio} y t_{fin} son los instantes de apertura y cierre, en segundos de simulación. El archivo `TripInfo` de SUMO reúne registros por viaje; el atributo `depart` indica el instante efectivo de inserción y `waitingTime` registra en segundos el tiempo a una velocidad menor o igual a 0,1 metros por segundo, salvo las detenciones programadas [Ecl26b]. Se incluyen los vehículos cuyo `depart` pertenece a la ventana, aunque su viaje siga inconcluso al cierre. El atributo `departDelay` registra el retraso previo a la inserción y no integra la espera del viaje:

- **Espera media de los vehículos insertados ($\bar{w}_{\text{ins}}$):** sea $\mathcal{V}_{\text{ins}}$ el conjunto de vehículos insertados dentro de la ventana y w_v la espera en segundos registrada para el

vehículo v. Su cantidad es $|\mathcal{V}_{\text{ins}}|$; la media se calcula como:

$$\bar{w}_{\text{ins}} = \frac{\sum_{v \in \mathcal{V}_{\text{ins}}} w_v}{|\mathcal{V}_{\text{ins}}|}, \qquad w_v = \texttt{waitingTime}_v. \tag{3.15}$$

La medida se expresa en segundos por vehículo; si el conjunto está vacío, no se asigna una puntuación.

- **Tasa de completitud (*Completion Rate*) (CR):** proporción de los vehículos incluidos en la demanda que completa su itinerario dentro del horizonte de evaluación. Esta proporción se denota por $\text{CR}_{\text{ventana}}$. Sea $N_{\text{completados}}$ el número de viajes terminados antes del cierre y N_{cargados} el total de vehículos incorporados a la demanda de SUMO. La tasa también contabiliza los vehículos que no lograron ingresar por sobresaturación:

$$\text{CR}_{\text{ventana}} = \frac{N_{\text{completados}}}{N_{\text{cargados}}} \tag{3.16}$$

  Cuando no se descartan vehículos, el total cargado coincide con los insertados y los que siguen esperando en el origen al cierre. En la salida de SUMO, `departed` cuenta los vehículos insertados y `pending_vehicles_end` los que aún no ingresaron [DLR26]. La Tasa de inserción (*Insertion Rate*) (IR) es el número de vehículos insertados dividido por el de cargados; la CI es el número de viajes completados dividido por el de vehículos insertados. CR incluye la demanda que no logró ingresar; CI describe la finalización entre quienes sí ingresaron. Estas medidas complementan la espera y evitan evaluar solo los vehículos que alcanzaron a entrar en la red [JLW+24].

- **Desigualdad espacial de espera (coeficiente de Gini, G_{wait}):** en cada paso, el entorno calcula la espera acumulada atribuida a los carriles de cada intersección y la divide por el número de vehículos presentes; si no hay vehículos, asigna cero. Luego promedia esta medida por intersección durante la ventana de control. Sea N la cantidad de intersecciones y $\bar{w}_{(i)}$ la espera media temporal de la intersección situada en la posición i al ordenar esas medias de menor a mayor ($i = 1, \ldots, N$). Cada intersección recibe el mismo peso:

$$G_{\text{wait}} = \frac{2\sum_{i=1}^{N} i \cdot \bar{w}_{(i)} - (N+1)\sum_{i=1}^{N} \bar{w}_{(i)}}{N \sum_{i=1}^{N} \bar{w}_{(i)}} \tag{3.17}$$

  En el código, la medida por paso corresponde al campo `mean_acc_waiting_time`. El índice describe la desigualdad entre las medias por intersección, no entre vehículos; el subíndice entre paréntesis indica la posición en el orden, no la identidad del cruce. Su valor va de cero a menos de uno: cero indica medias iguales entre intersecciones y los valores más altos, mayor desigualdad. Los promedios de espera y su desviación

estándar entre intersecciones (σ_{wait}) se expresan en segundos; el coeficiente de Gini es adimensional. Si la suma de medias es cero, la implementación informa Gini cero.

- **Pérdida temporal ($\bar{\ell}$):** sea ℓ_v la pérdida temporal del viaje del vehículo v, registrada por SUMO en `timeLoss`. $\bar{\ell}$ es la media de esa cantidad entre los vehículos insertados y se expresa en segundos por vehículo. Estima el tiempo perdido frente al viaje a la velocidad ideal, ajustada por el multiplicador individual que modifica la velocidad deseada del vehículo respecto del límite vial, y sin contar detenciones programadas; se informa como medida secundaria distinta de la espera [Ecl26b].

Los registros de vehículos inconclusos se conservan mediante la opción `-tripinfo-output.write-unfinished`. Además de IR, CI y CR, se registran las pérdidas temporales y los teletransportes que notifica SUMO: estos últimos señalan bloqueos operacionales y no se interpretan como viajes completados.

3.6.2. Comparación por pares y criterio de decisión

Para cada política se promedian las cinco evaluaciones, que usan semillas SUMO comunes a A y C. Si $w_{s,b,q}$ es la espera media por vehículo insertado, en segundos, para el par de entrenamiento s, la condición $b \in \{A, C\}$ y la semilla de evaluación $q \in \{1, \ldots, 5\}$, la media por política es $\bar{w}_{s,b} = \frac{1}{5}\sum_{q=1}^{5} w_{s,b,q}$. Para cada par, d_s denota la diferencia absoluta C menos A, en segundos por vehículo, y z_s la diferencia relativa adimensional con respecto a A; un valor negativo favorece C. Los efectos reportados son las medias entre pares, $\bar{d}$ y $\bar{z}$; $100\bar{z}$ expresa el cambio porcentual medio. La diferencia relativa no está definida cuando $\bar{w}_{s,A} = 0$ y debe marcarse como tal. La unidad de inferencia es el par de políticas entrenadas, no cada episodio de evaluación [ASC+21]. Las dos diferencias se calculan como:

$$d_s = \bar{w}_{s,C} - \bar{w}_{s,A}, \qquad z_s = \frac{\bar{w}_{s,C} - \bar{w}_{s,A}}{\bar{w}_{s,A}}. \tag{3.18}$$

No se sustituye la media de diferencias relativas por el cociente entre medias globales, que ponderaría implícitamente los pares de otra manera.

La incertidumbre se estima mediante remuestreo con reemplazo (*bootstrap*) de los 24 pares completos de cada red; el efecto se recalcula en 20.000 muestras. Se informan intervalos percentiles del 95 % y del 98,33 %; este último aplica Bonferroni a las tres comparaciones por red. Como comprobación se calcula también un intervalo t de Student sobre las diferencias pareadas. Ambos intervalos aproximan la incertidumbre; se conservan los valores extremos y se revisa su influencia.

El criterio de decisión para A/C usa el intervalo del 98,33 %: su límite superior para el efecto relativo en espera debe ser menor que $-0,05$, y el límite inferior para la diferencia de CI (C menos A) debe superar $-0,005$, equivalente a 0,5 puntos porcentuales en la tasa. Además, C no debe producir más teletransportes que A en ninguna evaluación pareada. Los márgenes del 5 % y de 0,5 puntos porcentuales son decisiones prácticas de este análisis,

no umbrales universales. El intervalo del 95 % se informa como descripción y análisis de sensibilidad.

Los intervalos describen variación entre entrenamientos, condicionada a las cinco semillas de evaluación compartidas; cinco episodios de una política no equivalen a cinco entrenamientos independientes y no cubren toda demanda posible. La ausencia de evidencia de una reducción práctica no demuestra equivalencia. Las semillas de evaluación son fijas. Para los viajes que siguen inconclusos al cierre no se observa cuánto tardarían en completarse fuera del horizonte simulado.

3.7. Reproducibilidad y alcance

Para reproducir los experimentos se conservan archivos de red y rutas, configuración SUMO, semillas, arquitectura, parámetros efectivos, versiones de código y archivos de estado del modelo (*checkpoints*). Los protocolos, manifiestos y códigos de verificación calculados a partir de los archivos permiten comprobar qué procedimiento produjo cada registro. La sección A.5 reúne las fuentes y valores que completan las decisiones del capítulo.

3.7.1. Entorno de cómputo y envío de trabajos

Las búsquedas principales y la comparación A/C se ejecutaron en el clúster de cómputo Toko. El cribado exploratorio se realizó en un entorno local y en Toko. Las exploraciones anteriores en el clúster Mendieta no forman parte de los análisis presentados. Los scripts solicitaron diez núcleos de la unidad central de procesamiento (CPU, del inglés *Central Processing Unit*); el manual institucional disponible describe una partición de Toko —grupo de recursos que el sistema asigna para ejecutar trabajos— con hasta ocho, por lo que no permite confirmar los recursos físicos asignados a cada trabajo [Tok24].

Los trabajos se enviaron con `sbatch`, comando del gestor de trabajos Slurm —sigla de *Simple Linux Utility for Resource Management*— que los incorpora a la cola de ejecución. Para la fase 1 se utilizó el script `submit_phase1.sh` y, para la fase 2, `submit_phase2_refined.sh`. Cada trabajo correspondió a una combinación de algoritmo, red y arquitectura neuronal. Los scripts solicitaron un nodo (servidor de cómputo), una tarea (proceso de ejecución), diez núcleos de CPU y una partición con recursos de unidad de procesamiento gráfico (GPU, del inglés *Graphics Processing Unit*). En la fase 2 también solicitaron asignación exclusiva del nodo y un límite de tiempo explícito. Ray ejecutó como máximo un ensayo de búsqueda a la vez en cada trabajo; ASHA determinó las detenciones anticipadas con los parámetros propios de cada fase descritos arriba.

Los resultados se guardaron en almacenamiento persistente y se copiaron al espacio temporal del nodo durante la ejecución, lo que permite reanudar un trabajo después de una interrupción. Aunque los scripts configuraron un recurso de GPU para Ray, no solicitaron una tarjeta física mediante un identificador de recurso de Slurm; por ello, los archivos del

proyecto no permiten asociar cada trabajo con un modelo específico de GPU.

3.7.2. Alcance de las conclusiones

La comparación A/C modifica conjuntamente observación y recompensa, por lo que sus resultados no identifican el efecto aislado de ninguno de esos cambios. Como se especifica en la sección 3.5, los 24 pares de 3×3 se excluyeron del análisis principal después de observar los resultados y se presentan por separado; la comparación principal comprende las otras dos redes.

La búsqueda adaptativa produce coberturas y presupuestos efectivos desiguales: igualar iteraciones no iguala pasos de gradiente ni costo de cómputo entre PPO, HAPPO e IDQN. Las arquitecturas seleccionadas no permiten atribuir causalmente diferencias de desempeño ni garantizan superioridad fuera de los escenarios examinados. La implementación de HAPPO con recompensas y ventajas individuales y objetivo recortado tampoco hereda la garantía teórica de mejora monótona para recompensas compartidas.

La demanda de Mendoza incorpora un volumen nominal, una matriz espacial supuesta y una semilla seleccionada durante la construcción del escenario. La comprobación usa los programas semafóricos de SUMO, no planes municipales; tampoco equivale a evaluar una política aprendida. Las conclusiones se limitan a las redes, flotas, rutas, semillas, medidas y presupuestos documentados. La espera se calcula para los vehículos insertados y, en los viajes inconclusos, queda censurada al cierre de la simulación, como se define en la sección 3.6.

4

Análisis y discusión de resultados

Este capítulo organiza los resultados por procedimiento: búsqueda de hiperparámetros, comparación pareada A/C de MAPPO y comprobación operacional de Mendoza. La búsqueda y la comparación corresponden a unidades experimentales distintas; sus métricas y criterios de selección se interpretan por separado.

La comprobación de Mendoza corresponde a la red urbana con los programas semafóricos incluidos en ella. Sus conteos y tasas describen esa operación y no se interpretan como una evaluación de políticas aprendidas.

4.1. Resultados del ajuste de hiperparámetros

La presentación de HPO distingue la exploración amplia de fase 1 de los dominios informados para fase 2. Las configuraciones candidatas y su selección se interpretan mediante los criterios especificados en la metodología, sin confundir el objetivo de búsqueda con la evaluación física.

4.1.1. Estado de la consolidación y de las búsquedas

4.1.2. Resultados del ajuste y análisis de sensibilidad

4.1.3. Selección de configuraciones de HPO

4.2. Evaluación de políticas y comprobación de escenarios

La comparación A/C se presenta por separado de la selección de configuraciones mediante HPO. El análisis conserva los pares de entrenamiento y presenta por separado la red 3×3, excluida del análisis principal después de observar sus resultados, como se describe en la sección 3.5.

4.2.1. Criterios de evaluación y métricas de comparación

4.2.2. Desempeño en escenarios sintéticos: 3×1, 3×3, 4×4 heterogéneo

4.2.3. Comprobación operacional del escenario de la Ciudad de Mendoza

Se comprobó el funcionamiento de la red modelada de la Ciudad de Mendoza en los dos horizontes definidos en la sección 3.2.3. La tabla 4.1 presenta conteos y tasas obtenidos con los programas semafóricos de la red; caracterizan su operación, no el desempeño de políticas aprendidas. La fila de dos horas corresponde a `mendoza_0630_0830.sumocfg`, que comienza en $t = 1800$ (06:30) y no incluye el calentamiento de 300 segundos de la configuración MARL `mendoza_marl_2h.sumocfg`, que comienza en $t = 1500$ (06:25).

Tabla 4.1: Métricas operacionales del escenario urbano de la Ciudad de Mendoza con los programas semafóricos de la red.

Ventana temporal	**Cargados**	**Insertados**	**Completados**	**CR**	**Inserción**	**Finalización**	*teleport*
2 horas (06 : 30–08 : 30)	30.629	23.735	22.702	0,7412	0,7749	0,9565	0
12 horas (06 : 00–18 : 00)	180.163	143.819	143.022	0,7938	0,7983	0,9945	0

4.3. Discusión e interpretación de resultados

En esta sección se integran los resultados presentados y se discuten sus principales implicancias en relación con los objetivos del trabajo.

5

Conclusiones

Este capítulo sintetiza los hallazgos del estudio, relaciona la evidencia experimental con los objetivos planteados y delimita el alcance de las conclusiones. También identifica las limitaciones del modelo y propone líneas de trabajo futuro. Las afirmaciones finales deben interpretarse dentro de los escenarios, métricas y protocolos descritos en la metodología.

A

Arquitectura de software y reproducción experimental

Este apéndice describe el diseño orientado a objetos del paquete de software `marlTLC` mediante diagramas de clases UML y presenta los patrones previstos para separar la simulación, el aprendizaje por refuerzo y el cálculo de recompensas. Los diagramas representan el diseño inicial y no una correspondencia exhaustiva entre clases y archivos ejecutables. Los apartados finales reúnen parámetros, dominios y semillas necesarios para reproducir los procedimientos descritos.

A.1. Diseño orientado a objetos del entorno de simulación

El diseño del entorno separa las responsabilidades de sus componentes. En la especificación, la clase abstracta `BaseSumoEnvironment` encapsula la inicialización de SUMO, el control temporal y la interacción de bajo nivel con la API `libsumo` / `traci`. A partir de ella, `SumoRllibEnv` se define como adaptador de la simulación a la interfaz `MultiAgentEnv` de Ray RLlib.

La figura A.1 presenta las entidades del entorno y sus relaciones con los agentes semafóricos. En un diagrama de clases, cada recuadro contiene el nombre de la clase, sus atributos y sus operaciones. El signo "−" identifica miembros privados, "+" públicos y "#" protegidos, disponibles en la clase y sus derivadas. El texto después de los dos puntos indica el tipo de dato. La cursiva señala una clase u operación abstracta, que debe concretarse en otra clase. La flecha "hereda de" apunta desde la clase especializada hacia la

general; la multiplicidad 1..∗ indica “uno o más” objetos relacionados.

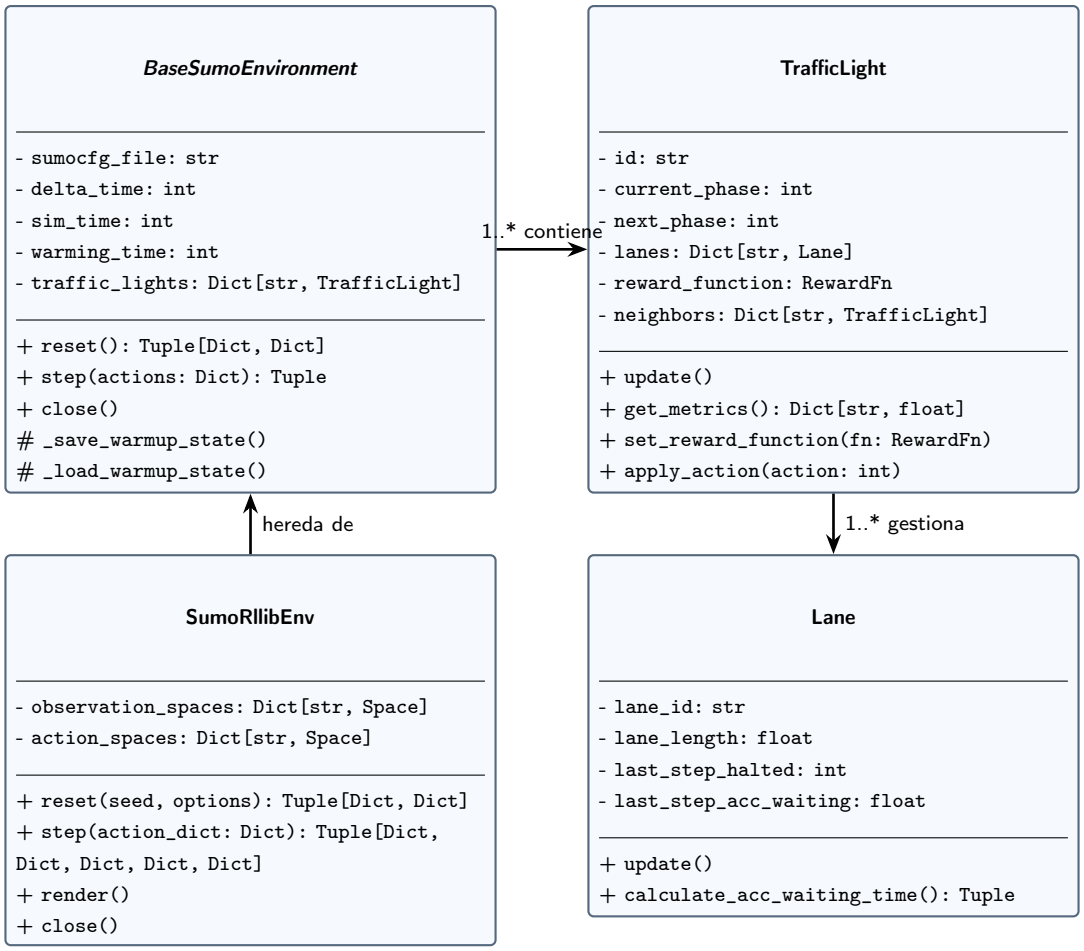


Figura A.1: Diagrama de clases UML del entorno. `SumoRllibEnv` adapta la clase abstracta `BaseSumoEnvironment`; el entorno contiene uno o más objetos `TrafficLight`, y cada uno gestiona objetos `Lane`. En cada clase se muestran sus principales atributos y métodos.

La figura A.1 se organiza desde la abstracción general hacia las entidades de la red. En la columna izquierda, `BaseSumoEnvironment` concentra el archivo de configuración, los parámetros temporales, los semáforos y las operaciones generales de reinicio, avance y cierre. Debajo, `SumoRllibEnv` especializa esa clase para recibir un diccionario de acciones de los agentes y devolver las observaciones, recompensas y estados definidos por la interfaz de Ray RLlib. A la derecha, el diseño asigna a `TrafficLight` la fase actual, la siguiente fase, sus carriles, su función de recompensa y sus vecinos; también le asigna la actualización de métricas y la aplicación de acciones. Finalmente, `Lane` representa la longitud y las medidas recientes de detención y espera. Las flechas muestran las dependencias previstas entre la interfaz de aprendizaje, el entorno, los semáforos y sus carriles.

A.2. Jerarquía de funciones de recompensa y patrón *Strategy*

En la especificación, el cálculo de la recompensa se desacopla mediante el patrón ***Strategy***. Esta organización permite sustituir la métrica de optimización sin modificar el bucle principal de simulación.

La clase base abstracta `RewardFn` define la interfaz común `compute_reward(tl_id)`. En el diseño se contemplan formulaciones locales basadas en tres señales: demora acumulada, longitud de cola y presión de red. La coordinación distribuida se representa mediante el patrón ***Composite***: una clase de recompensa compuesta combina la señal local y la señal promedio de los semáforos adyacentes.

La figura A.2 detalla las clases que intervienen en la recompensa cooperativa. Las demás estrategias locales no se dibujan para conservar la legibilidad. Se mantienen las convenciones de la figura A.1; en este segundo diagrama, la flecha discontinua "compone" indica que una clase usa otra en su cálculo, no que herede sus atributos y métodos.

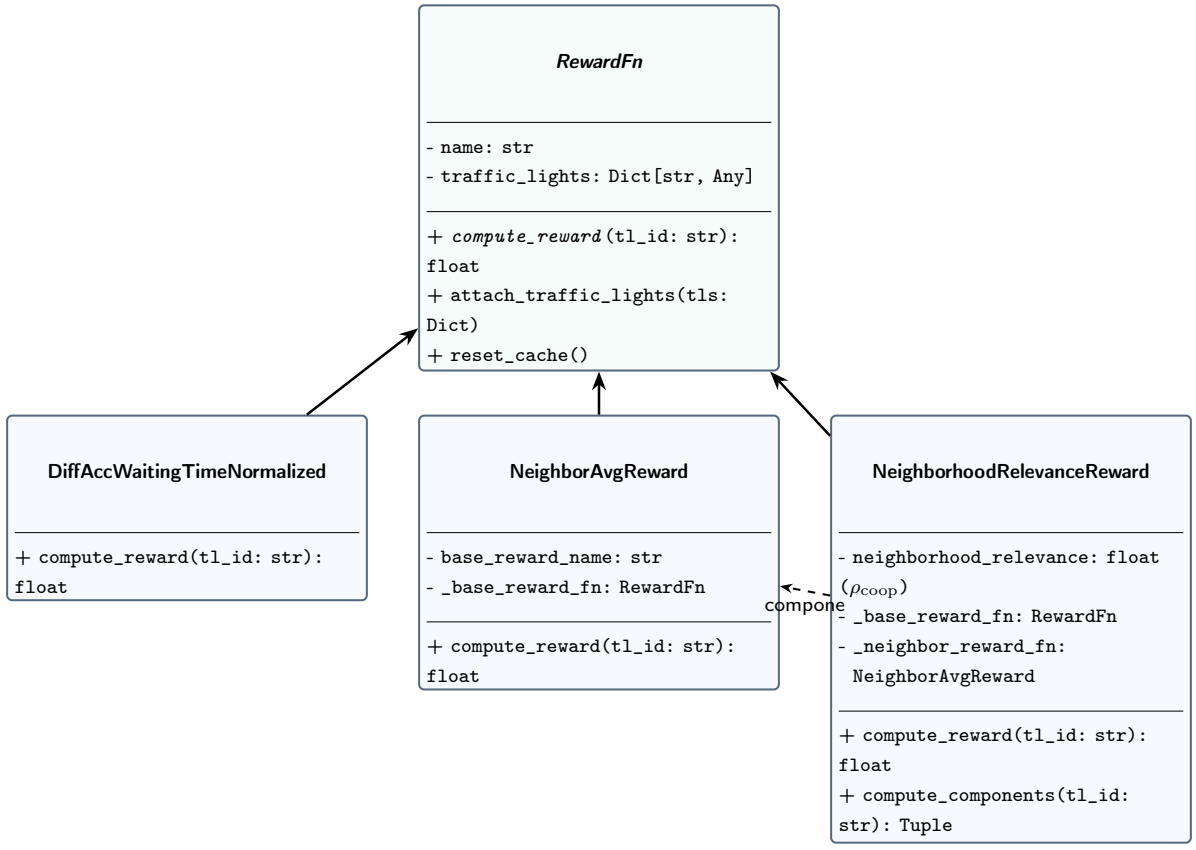


Figura A.2: Especificación UML del módulo de recompensas. Las clases inferiores realizan la interfaz `RewardFn`; la clase compuesta incorpora la señal promedio de los agentes vecinos.

La figura A.2 sitúa la clase abstracta `RewardFn` en la parte superior; esta define la operación abstracta `compute_reward`, que recibe un identificador de semáforo y devuelve

un valor numérico. Las flechas indican las clases que implementan variantes de esa operación. `DiffAccWaitingTimeNormalized` representa una señal local normalizada a partir de la variación de la espera acumulada; `NeighborAvgReward` promedia la señal base de los vecinos; y `NeighborhoodRelevanceReward` combina ambas según ρ_{coop}. La flecha discontinua indica la composición entre las dos últimas clases. Esta separación permite sustituir la formulación de la recompensa sin cambiar el bucle de simulación.

A.3. Patrones de diseño de la arquitectura

A continuación se resumen los patrones contemplados en la especificación de `marlTLC`. La lista describe responsabilidades y relaciones de diseño; no certifica la presencia de una implementación ejecutable de cada componente en el repositorio.

1. **Patrón de estrategia (*Strategy*):**
   - **Propósito:** encapsular los algoritmos de recompensa para intercambiarlos sin modificar el módulo que aplica las señales.
   - **Rol en el diseño:** interfaz base `RewardFn` y variantes concretas de recompensa.
2. **Método de fábrica y registro (*Factory Method* y *Registry*):**
   - **Propósito:** instanciar funciones de recompensa y módulos neuronales a partir de cadenas de configuración o especificaciones compuestas (`RewardSpec`). También permite registrar nuevas funciones sin modificar la fábrica, de acuerdo con el principio abierto/cerrado.
   - **Rol en el diseño:** registro mediante `RewardFactory` y `@register_reward`, que identifica el mecanismo previsto para asociar nombres de configuración con funciones de recompensa.
3. **Patrón de objeto compuesto (*Composite*):**
   - **Propósito:** combinar recompensas locales y vecinales mediante una interfaz uniforme.
   - **Rol en el diseño:** clases de recompensa compuesta que agregan señales locales y vecinales.
4. **Protocolos y tipado estructural (*Protocol* / *Structural Subtyping*):**
   - **Propósito:** desacoplar el proveedor de métricas vecinales del tipo concreto `TrafficLight`, facilitando las pruebas unitarias con objetos simulados (*mocks*) y evitando dependencias circulares.

- **Rol en el diseño:** protocolo `NeighborInfoProvider` para describir la información vecinal requerida sin depender de una clase concreta. El detalle del decorador de comprobación en tiempo de ejecución no forma parte de la evidencia disponible en este repositorio.

5. **Patrón de adaptador (*Adapter*):**

   - **Propósito:** adaptar la API procedural y basada en comandos de TraCI / `libsumo` a la interfaz orientada a objetos de `MultiAgentEnv` (Ray RLlib).
   - **Rol en el diseño:** `SumoRllibEnv` como punto de adaptación entre el simulador y la interfaz de aprendizaje.

6. **Patrón de decorador (*Decorator*):**

   - **Propósito:** extender dinámicamente las capacidades de los módulos sin alterar su jerarquía de herencia.
   - **Rol en el diseño:** decoradores de registro y una envoltura para los módulos de acción en PyTorch; los nombres concretos de esas piezas se mantienen como identificadores de la especificación.

A.4. Parámetros de SUMO y programas semafóricos de referencia

Los archivos de red contienen las secuencias de señales utilizadas antes de aplicar las decisiones de los agentes. En las redes sintéticas, todos los programas son de tiempo fijo y tienen un ciclo de 90 segundos; sus duraciones aparecen en la Tabla 3.2.

La red de Mendoza contiene 73 programas: 17 de tiempo fijo y 56 accionados, es decir, que responden a la detección de vehículos. La tabla A.1 agrupa las secuencias de duraciones nominales del archivo `gran_mendoza2_fixed.net.xml`. Cada fila reúne programas con la misma secuencia de duraciones, aunque sus movimientos habilitados pueden diferir.

Tabla A.1: Duraciones nominales de las fases en el archivo de red de Mendoza, en segundos.

Tipo	Cantidad	Secuencia de duraciones
Accionado	1	33, 6, 6, 6, 33, 6
Accionado	5	39, 6, 39, 6
Accionado	19	40, 5, 40, 5
Accionado	5	41, 4, 41, 4
Accionado	24	42, 3, 42, 3
Accionado	1	81, 4, 5
Accionado	1	82, 3, 5
Fijo	1	24, 3, 24, 3, 6, 3, 24, 3
Fijo	2	39, 6, 39, 6
Fijo	3	40, 5, 40, 5
Fijo	1	41, 4, 41, 4
Fijo	8	42, 3, 42, 3
Fijo	2	82, 3, 5

En los programas accionados, la duración efectiva puede variar. Los atributos *minDur* y *maxDur* establecen los límites de cada verde: los mínimos registrados son 5, 7, 10 o 13 segundos según la fase, y el máximo es de 50 segundos. Por eso, sumar las duraciones nominales de la tabla no permite afirmar que todos los ciclos ejecutados duren lo mismo.

A.4.1. Identificadores del escenario urbano de Mendoza

En la figura 3.2, el acceso de Avenida José Vicente Zapata se representa mediante el tramo interno `93904440#0`, ubicado en la TAZ 28. La continuidad de los carriles de ingreso se restablece entre los tramos `93904440#0` y `93904440#1`. La configuración MARL de la hora pico parte de la demanda generada con el perfil de doce horas, cuyo origen es 06:00. La simulación comienza en $t = 1500$ segundos (06:25), incluye 300 segundos de calentamiento y controla entre $t = 1800$ y $t = 9000$ (06:30–08:30). Se utiliza `nets/Mendoza/mendoza_marl_2h.sumocfg`; la configuración estática `mendoza_0630_0830.sumocfg` no incluye el calentamiento.

La comprobación estática de doce horas controla desde $t = 0$ hasta $t = 43200$ (06:00–18:00). La comprobación estática de dos horas comienza en $t = 1800$ y controla hasta $t = 9000$, sin calentamiento previo; ambas utilizan la misma demanda de doce horas que la configuración MARL. Mendoza forma parte del diseño de búsqueda para los cuatro algoritmos, pero los dominios y presupuestos detallados en este apéndice corresponden a las combinaciones sintéticas documentadas y no se atribuyen a la búsqueda urbana.

El parámetro `waiting_time_memory` determina cuántos segundos de historia conserva SUMO al consultar la espera acumulada de un vehículo. En los experimentos de aprendizaje se fija en 1000 segundos. Esta ventana afecta la observación del agente, mientras que el atributo `waitingTime` de TripInfo acumula la espera durante el viaje [Ecl26a, Ecl26b].

En Mendoza, la fracción `camera_retention` se fija en 1 para conservar todo el conteo como referencia de ingreso; no es una proporción medida.

En los episodios de aprendizaje, el entorno guarda el estado de SUMO al terminar el calentamiento y lo carga al reiniciar el episodio; también puede pasar una semilla al iniciar SUMO. Guardar y cargar ese estado vehicular no demuestra por sí solo que se restaure toda fuente de aleatoriedad. En el código de inicialización y restauración consultado no aparece una opción explícita para guardar el estado de los generadores aleatorios; por eso, aquí no se afirma que el reinicio reproduzca todas las fuentes estocásticas.

A.5. Especificaciones para reproducir los experimentos

Los parámetros, dominios y semillas se contrastaron con las configuraciones experimentales y los archivos de ejecución del paquete `marl-tlc`. En este apéndice, fase 1 y fase 2 designan etapas de búsqueda de hiperparámetros.

A.5.1. Semillas de fase 1 en las redes sintéticas

La tabla A.2 reúne las semillas observadas en las configuraciones serializadas de fase 1 y las asociadas a los perfiles canónicos de generación de rutas. La semilla de entrenamiento inicializa el aprendizaje y la simulación SUMO durante el entrenamiento; la de evaluación controla SUMO en las evaluaciones diagnósticas.

Tabla A.2: Semillas por escenario sintético en la fase 1.

Escenario	Aprendizaje y SUMO de entrenamiento	SUMO de evaluación	Perfil de rutas
3×1	101	100101	1333938385
3×3	137	100137	810840844
$4 \times 4_H$	179	100179	1871007461

El código de fase 1 define 7001 como semilla TPE por defecto, independiente de las semillas de entrenamiento. El consolidado histórico no registra la semilla TPE por estudio; por tanto, no permite confirmar que ese valor se aplicara a todas las ejecuciones.

A.5.2. Arquitecturas y semillas de la búsqueda informada

Tabla A.3: Arquitecturas y semillas asignadas a las doce combinaciones de algoritmo y escenario de las redes sintéticas en fase 2. La semilla de evaluación es común a los algoritmos del mismo escenario; las semillas de TPE no son réplicas de entrenamiento.

Algoritmo	Escenario	Arquitectura	Semilla de entrenamiento	Semilla de evaluación	Semilla TPE
IPPO	3×1	h256	910001	920001	930001
IPPO	3×3	h128	910002	920003	930002
IPPO	$4 \times 4_H$	h128	910003	920004	930003
MAPPO	3×1	h128	910004	920001	930004
MAPPO	3×3	h128	910005	920003	930005
MAPPO	$4 \times 4_H$	h128	910006	920004	930006
HAPPO	3×1	h256	910007	920001	930007
HAPPO	3×3	h256	910008	920003	930008
HAPPO	$4 \times 4_H$	h128	910009	920004	930009
IDQN	3×1	h256	910010	920001	930010
IDQN	3×3	h128	910011	920003	930011
IDQN	$4 \times 4_H$	h256	910012	920004	930012

Las arquitecturas h128 y h256 contienen dos capas ocultas de 128 y 256 unidades, respectivamente. La arquitectura asignada a cada una de las doce combinaciones se consigna en la tabla. Son preferencias de búsqueda derivadas de la fase 1, no evidencia de superioridad bajo la recompensa local de fase 2. Las semillas TPE controlan las propuestas; las semillas SUMO, la simulación. Ninguna sustituye una réplica de entrenamiento.

A.5.3. Dominios completos y parámetros efectivos

Las tablas A.4–A.6 presentan los doce dominios v3, uno por combinación de algoritmo, escenario sintético y arquitectura asignada para fase 2. Sus valores se redondean para facilitar la lectura; los límites exactos figuran en `experiments/marl_experiments/tuning/phase2_refined/informed_domains_rho0_v3.json`. La red de Mendoza forma parte del diseño HPO, pero no de estas doce combinaciones sintéticas; no se le atribuyen los valores de presupuesto o semillas de este manifiesto. Las distribuciones U y LU indican muestreo continuo uniforme y loguniforme; R indica dominio entero con límite superior exclusivo y C, selección categórica. Para U y LU, los corchetes representan límites del dominio. En las columnas, cada dominio corresponde a la combinación indicada.

Tabla A.4: Dominios de búsqueda de HAPPO por escenario y arquitectura.

Parámetro	3×1 (h256)	3×3 (h256)	$4 \times 4_{\mathrm{H}}$ (h128)
`clip_param` U	[0,1; 0,22]	[0,1; 0,22]	[0,1; 0,22]
`encoder_intervals` R	[4; 27)	[5; 37)	[4; 65)
`entropy_coeff` U	[0,0043479; 0,0079967]	[0; 0,01]	[0; 0,01]
`grad_clip` LU	[0,64487; 4,7566]	[0,26841; 1,3289]	[0,25; 5]
`greedy_selection_prob` U	[0,40484; 0,82866]	[0,44174; 0,82189]	[0,48405; 0,77601]
`kl_coeff` LU	[0,05; 0,5]	[0,066096; 0,19175]	[0,05; 0,5]
`kl_target` LU	[0,015783; 0,03]	[0,005; 0,03]	[0,005; 0,03]
`lambda_gae` U	[0,88683; 0,93862]	[0,90514; 0,9443]	[0,88; 0,99]
`lr` LU	$[2{,}707 \times 10^{-4}; 1 \times 10^{-3}]$	$[3 \times 10^{-5}; 1 \times 10^{-3}]$	$[3 \times 10^{-5}; 1 \times 10^{-3}]$
`minibatch_size` C*	{256; 512}	{128; 512}	{256; 512}
`num_epochs` R	[5; 19)	[5; 19)	[5; 19)
`train_batch_episodes` R	[2; 13)	[2; 13)	[2; 13)
`vf_clip_param` LU	[8,402; 18,471]	[6,674; 20]	[5; 11,218]

Tabla A.5: Dominios de búsqueda de IDQN por escenario y arquitectura.

Parámetro	3×1 (h256)	3×3 (h128)	$4 \times 4_{\mathrm{H}}$ (h256)
`encoder_intervals` R	[10; 62)	[7; 33)	[4; 65)
`epsilon_final` U	[0,02; 0,047523]	[0,023555; 0,066094]	[0,02; 0,1]
`epsilon_timesteps` C	{10 000; 20 000}	{10 000; 20 000}	{10 000; 20 000}
`grad_clip` C	{0,5; 1; 2; 5; 10}	{0,5; 1; 2}	{1; 2; 5; 10}
`lr` LU	$[9{,}879 \times 10^{-5}; 1 \times 10^{-3}]$	$[1 \times 10^{-5}; 1 \times 10^{-3}]$	$[8{,}135 \times 10^{-5}; 7{,}889 \times 10^{-4}]$
`n_step` C	{3; 5}	{1; 3}	{1; 3; 5}
`num_steps_sampled_before_learning_starts` C	{1 000; 2 000; 5 000}	{1 000; 2 000; 5 000}	{1 000; 2 000; 5 000}
`target_network_update_freq` C	{500; 1 000; 2 000; 5 000}	{500; 2 000; 5 000}	{500; 1 000; 2 000}
`train_batch_size_per_learner` C	{64; 128; 256}	{256; 512}	{64; 128; 256}

Tabla A.6: Dominios de búsqueda de IPPO y MAPPO.

IPPO			
Parámetro	3×1 (h256)	3×3 (h128)	$4 \times 4_{\mathrm{H}}$ (h128)
`clip_param` U	[0,1; 0,22]	[0,1; 0,22]	[0,15327; 0,19443]
`encoder_intervals` R	[4; 65)	[4; 65)	[4; 65)
`entropy_coeff` U	[0,0038889; 0,0095621]	[0,00067906; 0,0074136]	[0,0036258; 0,01]
`grad_clip` LU	[0,25; 5]	[0,25; 5]	[0,42722; 1,5143]
`kl_coeff` LU	[0,055016; 0,21664]	[0,05; 0,2904]	[0,05; 0,5]
`kl_target` LU	[0,005; 0,03]	[0,005; 0,03]	[0,005; 0,03]
`lambda_gae` U	[0,88; 0,99]	[0,89174; 0,96008]	[0,88; 0,99]
`lr` LU	$[3 \times 10^{-5}; 1 \times 10^{-3}]$	$[1{,}136 \times 10^{-4}; 9{,}113 \times 10^{-4}]$	$[1{,}322 \times 10^{-4}; 4{,}772 \times 10^{-4}]$
`minibatch_size` C	{128; 512}	{128; 512}	{128; 512}
`num_epochs` R	[5; 16)	[5; 16)	[5; 16)
`train_batch_episodes` R	[2; 13)	[2; 13)	[2; 13)
`vf_clip_param` LU	[5; 20]	[5; 20]	[5; 20]

MAPPO			
Parámetro	3×1 (h128)	3×3 (h128)	$4 \times 4_{\mathrm{H}}$ (h128)
`clip_param` U	[0,1; 0,22]	[0,1; 0,22]	[0,1; 0,22]
`encoder_intervals` R	[4; 65)	[8; 45)	[4; 65)
`entropy_coeff` U	[0,00015809; 0,0062478]	[0; 0,01]	[0,0021511; 0,0080125]
`grad_clip` LU	[0,28422; 0,80645]	[0,25; 5]	[1,1032; 5]
`kl_coeff` LU	[0,05; 0,13242]	[0,05; 0,5]	[0,14905; 0,38494]
`kl_target` LU	[0,005; 0,03]	[0,01384; 0,028938]	[0,005; 0,01363]
`lambda_gae` U	[0,88703; 0,92246]	[0,88; 0,99]	[0,88; 0,99]
`lr` LU	$[4{,}285 \times 10^{-5}; 1{,}695 \times 10^{-4}]$	$[1{,}947 \times 10^{-4}; 1 \times 10^{-3}]$	$[3 \times 10^{-5}; 1 \times 10^{-3}]$
`minibatch_size` C	{256; 512}	{128; 512}	{128; 256; 512}
`num_epochs` R	[5; 16)	[6; 13)	[5; 16)
`train_batch_episodes` R	[2; 13)	[2; 13)	[2; 13)
`vf_clip_param` LU	[5; 20]	[5; 20]	[5; 20]

Para cada algoritmo, escenario y arquitectura, los dominios se derivan de los ensayos de fase 1 ordenados por su objetivo. El vector inicial combina la mediana de los parámetros numéricos y la moda de los categóricos entre los cinco mejores; puede no coincidir con una configuración histórica y se entrena desde cero. No se incorporan al optimizador puntuaciones, checkpoints ni bases Optuna de fase 1.

En parámetros numéricos se parte del intervalo de los diez mejores ensayos, se amplía cada extremo en un 10 % del ancho del dominio original y se limita a sus fronteras. Los parámetros loguniformes se tratan en escala logarítmica. Se conserva el dominio original

si los veinte mejores ensayos cubren al menos el 70 % de su ancho o si los diez mejores alcanzan uno de sus extremos. Los límites enteros se redondean hacia afuera, con extremo superior exclusivo. En parámetros categóricos se consideran las alternativas de los veinte mejores; si solo aparece una, se añade una alternativa vecina del dominio original.

El asterisco de `minibatch_size` en la tabla A.4 indica que el parámetro se registra, pero HAPPO procesa el lote completo por actor. En PPO, `encoder_intervals` fija los niveles del codificador, `clip_param` limita la razón de probabilidades del actor y `entropy_coeff` pondera la entropía. `kl_coeff` pondera la penalización KL y `kl_target` es el objetivo hacia el que se adapta ese coeficiente en HAPPO. `num_epochs` indica los recorridos de optimización del lote; `minibatch_size` fija el tamaño de sus subdivisiones cuando se usan.

Para PPO, el código multiplica `train_batch_episodes` por los pasos de control del episodio ($3600/5 = 720$) para calcular el tamaño del lote por componente de aprendizaje; ese cálculo no incluye un factor adicional por el número de procesos de simulación. En IDQN, `num_steps_sampled_before_learning_starts` fija los pasos muestreados antes de iniciar el aprendizaje; `target_network_update_freq` fija el intervalo de actualización de la red objetivo; `train_batch_size_per_learner` es el tamaño del lote de transiciones; `n_step`, el horizonte del valor objetivo temporal; `epsilon_final`, el nivel final de exploración; y `epsilon_timesteps`, el horizonte de su programación. $c_v =$ `vf_clip_param` es la cota superior de la pérdida cuadrática de valor, no un recorte PPO convencional de predicciones.

A.6. Detalles del aprendizaje y de la codificación

En los módulos de aprendizaje propios de MAPPO y HAPPO, primero se promedia el error cuadrático entre los N agentes para cada muestra, luego se limita al intervalo $[0, c_v]$ y finalmente se promedia sobre las B muestras válidas del lote:

$$\ell_b = \frac{1}{N}\sum_{i=1}^{N}(V_{b,i} - y_{b,i})^2, \qquad \mathcal{L}_V = \frac{1}{B}\sum_{b=1}^{B}\text{mín}\{c_v, \text{máx}(0, \ell_b)\}. \tag{A.1}$$

Aquí $V_{b,i}$ es la predicción del crítico para el agente i en la muestra b, $y_{b,i}$ su valor objetivo, N el número de agentes, B el número de muestras no enmascaradas del lote y $c_v =$ `vf_clip_param` la cota superior configurada. Este recorte del error cuadrático no es el *value clipping* convencional de PPO, que compara predicciones de valor recortadas y no recortadas; tampoco se afirma con qué frecuencia se activa.

IDQN reutiliza transiciones de un búfer de experiencia priorizado y aprende valores para las acciones; no tiene un actor PPO, y la selección se realiza entre acciones permitidas. Los métodos tipo PPO recolectan un lote de trayectorias. En IPPO y MAPPO, un *minibatch* es una subdivisión del lote que produce una actualización; HAPPO procesa el

lote completo por actor. Los críticos y optimizadores son componentes de entrenamiento, mientras que en PPO la ejecución de cada actor depende de su observación y de la máscara vigente.

La Figura A.3 separa las contribuciones del codificador utilizado en la sección 3.3.2. Permite comprobar la equivalencia algebraica, sin atribuir al ejemplo un resultado de control.

Para $0 \leq x_{\text{num}} < M_{\text{num}}$, la mezcla escalada de la sección 3.3.2 se expresa en la forma implementada por el código:

$$e(x) = \text{ceil}\left(F \cdot \log_2\left(\frac{x_{\text{num}}}{M_{\text{num}} \cdot L} + 1\right) + L^2 \cdot x_{\text{num}}\right), \tag{A.2}$$

donde $F = (1 - L)(I - 1)/\log_2(1/L + 1)$. Para $x_{\text{num}} \geq M_{\text{num}}$, el codificador devuelve directamente $I - 1$. Esta expresión es algebraicamente equivalente a la ecuación (3.7); aquí se conserva para documentar la operación computacional.

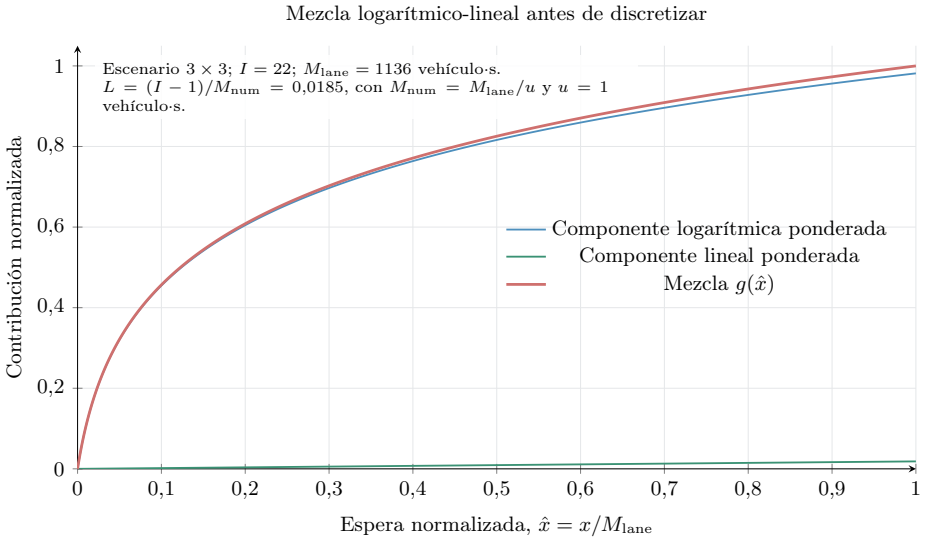


Figura A.3: Ejemplo ilustrativo con $I = 22$ y umbral $M_{\text{num}} = 1136$ calibrado para el escenario 3×3; $I = 22$ no corresponde a la configuración de la comparación A/C.

La tabla A.7 resume las configuraciones de recompensa documentadas para las búsquedas y las dos condiciones de la comparación A/C. La fila de fase 1 describe el programa de búsqueda examinado, no verifica individualmente todos sus ensayos históricos. La fase 2 sintética documentada usa solo la señal local; en A/C, C combina la señal local con la media vecinal normalizada por carriles.

Tabla A.7: Señales de recompensa por procedimiento documentado.

Procedimiento	Observación	Señal de aprendizaje
HPO, fase 1	Local	Diferencia de espera acumulada local mezclada con el promedio simple de las diferencias locales vecinales, sin dividir por carriles; el peso vecinal se explora.
HPO, fase 2 sintética	Local	Diferencia de espera acumulada local, con $\rho_{\text{coop}} = 0$.
A/C, condición A	Local	Diferencia de espera acumulada entre pasos consecutivos en la propia intersección: $R_i = L_i$.
A/C, condición C	Vecinal	Combinación de la diferencia local, con peso 0,9, y la media de las diferencias vecinales divididas por carriles, con peso 0,1, según la ecuación (3.14).

A.6.1. Configuración fija de la comparación multiescenario

La tabla A.8 conserva los hiperparámetros de MAPPO h128 seleccionados en un ensayo histórico de $4 \times 4_H$ y aplicados sin cambios entre las tres redes. El ensayo `49cfbba0` está registrado en `rho_screening_recipes/mappo_4x4_H_h128.json` e incorporado a los manifiestos de campaña. Los valores continuos se muestran redondeados; la fuente conserva precisión completa.

Tabla A.8: Hiperparámetros fijos de aprendizaje de la comparación A/C.

Parámetro	Valor
Arquitectura	dos capas de 128 unidades
Tasa de aprendizaje (`lr`)	$4,95235885 \times 10^{-5}$
Recorte del actor (`clip_param`)	0,20148266
GAE (`lambda_gae`)	0,90059980
Coeficiente de entropía	0,00483113
Recorte del gradiente	1,48846291
Coeficiente KL	0,24180060
Objetivo KL	0,01053851
Cota de pérdida de valor	10,11299284
Epochs por iteración	8
Episodios por lote	3
Tamaño de *minibatch*	128
Niveles del codificador (I)	17
Descuento (γ)	0,99

La comparación fija el peso de la recompensa vecinal en cero para A y en 0,1 para C; también asigna observación local o vecinal y normaliza por carriles, según la condición.

Los umbrales del codificador corresponden a cada escenario. Los tres episodios de cada lote representan $3 \times (3600/5) = 2160$ pasos de entorno por componente de aprendizaje. El número de procesos de simulación no multiplica este tamaño de lote.

Los manifiestos de la comparación registran SUMO 1.26.0, Ray 2.54.1 y PyTorch 2.10.0 con CUDA 12.8. Estas versiones corresponden a esa campaña y no se atribuyen automáticamente a las búsquedas históricas. Los archivos `manifest.json` de cada par conservan su configuración y versiones efectivas.

La calibración vecinal específica se conserva junto con la configuración de la comparación; otros archivos de calibración pueden corresponder a protocolos distintos.

A.6.2. Calibraciones y semillas de cooperación

El umbral local se calculó con las semillas 42–44; para 3×1, 3×3 y $4 \times 4_H$ se obtuvieron 1184, 1136 y 712, respectivamente. La calibración de la señal de espera vecinal incorporada en la observación C utilizó las semillas 51001–51003 y produjo umbrales 9660, 5329 y 4652 en esas redes. En ambos casos se empleó el percentil 99 y 300 segundos de calentamiento. Para la señal vecinal, el percentil se calculó sobre la medida media agrupada por intersección y paso. Los umbrales corresponden a señales codificadas, no a tiempos máximos de viaje ni a valores de recompensa.

La comparación en las tres redes utiliza las semillas 61001–61024 para el entrenamiento y 71001–71005 para la evaluación. Las semillas de evaluación son comunes a las dos variantes y se mantienen separadas de las de entrenamiento.

A.6.3. Archivos utilizados para las figuras

Las figuras de topología se generaron a partir de las redes sintéticas y urbanas utilizadas en los experimentos. En la Figura 3.2, el punto de conteo se representa de forma esquemática y no indica sus coordenadas exactas.

Bibliografía

[ACS24] Albrecht, Stefano V., Filippos Christianos y Lukas Schäfer: *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. MIT Press, 2024. https://www.marl-book.com/, Consultado en 2025.

[AHL16] Abel, David, D. Ellis Hershkowitz y Michael L. Littman: *Near Optimal Behavior via Approximate State Abstraction*. En *Proceedings of the 33rd International Conference on Machine Learning*, volumen 48 de *Proceedings of Machine Learning Research*, páginas 2915–2923. PMLR, 2016.

[AOS20] Amato, Christopher, Frans A. Oliehoek y Matthijs T.J. Spaan: *A Survey of Decentralized Partially Observable Markov Decision Processes and Applications*. Journal of Artificial Intelligence Research, 69:1–72, 2020.

[AS22] Ault, James y Guni Sharon: *Cooperative Multi-agent Reinforcement Learning Applied to Multi-intersection Traffic Signal Control*. En *Proceedings of the Third Workshop on Data-driven Intelligent Transportation, co-located with CIKM '22*, 2022. https://people.engr.tamu.edu/guni/papers/CIKM22-MATCS.pdf, Fuente: [7, 8] Destaca el "éxito limitado"de SARL y los enfoques independientes en redes.

[ASC+21] Agarwal, Rishabh, Max Schwarzer, Pablo Samuel Castro, Aaron C. Courville y Marc G. Bellemare: *Deep Reinforcement Learning at the Edge of the Statistical Precipice*. En *Advances in Neural Information Processing Systems 34*, páginas 29304–29320, 2021. https://proceedings.neurips.cc/paper_files/paper/2021/hash/f514cec81cb148559cf475e7426eed5e-Abstract.html.

[Aso18] Asociación de Fábricas Argentinas de Componentes: *Flota circulante en Argentina 2017*. Informe técnico, Asociación de Fábricas Argentinas de Componentes, mayo 2018. https://cdn.motor1.com/pdf-files/afac-informe-parque-circulante-2017pdf.pdf, Informe elaborado con datos de AFAC y Promotive.

[Aso26] Asociación de Fábricas Argentinas de Componentes: *Flota vehicular circulante en Argentina*. Informe técnico, Asociación de Fábricas Argentinas de Componentes, 2026. https://www.automotrix.com.ar/adm/enviosmasivos/Plantillas/PRENSA/2026/AFAC_PRENSA_FLOTACIRCULANTE_2025.pdf, Datos al cierre de 2025; informe elaborado en el marco del convenio entre AFAC y Promotive S.A.

[ASY+19] Akiba, Takuya, Shotaro Sano, Toshihiko Yanase, Takeru Ohta y Masanori Koyama: *Optuna: A next-generation hyperparameter optimization framework*. En *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, páginas 2623–2631, 2019.

[BB24] Bishop, Christopher M. y Hugh Bishop: *Deep Learning: Foundations and Concepts*. Springer, Cham, 2024.

[BBBK11] Bergstra, James S., Rémi Bardenet, Yoshua Bengio y Balázs Kégl: *Algorithms for Hyper-Parameter Optimization*. En *Advances in Neural Information Processing Systems 24*, 2011. https://proceedings.neurips.cc/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html.

[BV02] Bowling, Michael y Manuela Veloso: *Multiagent learning using a variable learning rate*. Artificial Intelligence, 136(2):215–250, 2002.

[CWCL19] Chu, Tianshu, Jie Wang, Lara Codecà y Zhaojian Li: *Multi-agent deep reinforcement learning for large-scale traffic signal control*. IEEE Transactions on Intelligent Transportation Systems, 21(3):1086–1095, 2019.

[CYL+24] Chen, Wubing, Shangdong Yang, Wenbin Li, Yujing Hu, Xiao Liu y Yang Gao: *Learning Multi-Intersection Traffic Signal Control via Coevolutionary Multi-Agent Reinforcement Learning*. IEEE Transactions on Intelligent Transportation Systems, 25(11):15947–15963, 2024.

[CyMR18] Mayor R., Rafael Cal y: *Ingeniería de tránsito: Fundamentos y aplicaciones*. Alfaomega, 9na edición, 2018.

[DLR26] DLR – Institute of Transportation Systems: *SUMO User Documentation*, 2026. https://sumo.dlr.de/docs/, Documentación web consultada en 2026.

[DLSZ26] Deng, Maojiang, Shoufeng Lu, Jiazhao Shi y Wen Zhang: *Adaptive Traffic Signal Control Optimization Using a Novel Road Partition and Multi-Channel State Representation Method*. Urban Lifeline, 4(1):9, 2026.

[dWGM+20] Witt, Christian Schroeder de, Tarun Gupta, Denys Makoviichuk, Viktor Maksimov, Edward Ivanov, Michael Schmid, Gregory Yakovlev, Philip Torr *y cols.*: *Is independent learning all you need in the starcraft multi-agent challenge?* arXiv preprint arXiv:2011.09533, 2020.

[Ecl26a] Eclipse SUMO: *SUMO Documentation: TraCI: Vehicle Value Retrieval*. `https://sumo.dlr.de/docs/TraCI/Vehicle_Value_Retrieval.html`, 2026. Documentación en línea, consultada el 8 de octubre de 2026.

[Ecl26b] Eclipse SUMO: *SUMO Documentation: TripInfo*. `https://sumo.dlr.de/docs/Simulation/Output/TripInfo.html`, 2026. Documentación en línea, consultada el 8 de octubre de 2026.

[ELR23] Eimer, Theresa, Marius Lindauer y Roberta Raileanu: *Hyperparameters in Reinforcement Learning and How To Tune Them*. En *Proceedings of the 40th International Conference on Machine Learning*, volumen 202 de *Proceedings of Machine Learning Research*, páginas 9104–9149. PMLR, 2023. `https://proceedings.mlr.press/v202/eimer23a.html`.

[FA11] Fernández A., Rodrigo: *Elementos de la teoría del tráfico vehicular*. Fondo Editorial de la Pontificia Universidad Católica del Perú, 2011.

[Fed08] Federal Highway Administration: *Traffic Signal Timing Manual*. Informe técnico FHWA-HOP-08-024, Federal Highway Administration, Office of Operations, 2008. Capítulo 4: Traffic Signal Design. Disponible en: `https://ops.fhwa.dot.gov/publications/fhwahop08024/chapter4.htm`.

[FNF+17] Foerster, Jakob, Nantas Nardelli, Gregory Farquhar, Philip H.S. Torr, Pushmeet Kohli y Shimon Whiteson: *Stabilising Experience Replay for Deep Multi-Agent Reinforcement Learning*. En *Proceedings of the 34th International Conference on Machine Learning (ICML)*, páginas 1146–1155. PMLR, 2017.

[FYX+23] Fang, Jie, Ya You, Mengyun Xu, Juanmeizi Wang y Sibin Cai: *Multi-Objective Traffic Signal Control Using Network-Wide Agent Coordinated Reinforcement Learning*. Expert Systems with Applications, 229:120535, 2023.

[GBC16] Goodfellow, Ian, Yoshua Bengio y Aaron Courville: *Deep Learning*. MIT Press, Cambridge, MA, 2016. `https://www.deeplearningbook.org/`.

[GR18] Genders, Wade y Saiedeh Razavi: *Evaluating Reinforcement Learning State Representations for Adaptive Traffic Signal Control*. Procedia Computer Science, 130:26–33, 2018.

[HHA22] Haddad, Tarek Amine, Djalal Hedjazi y Sofiane Aouag: *A Deep Reinforcement Learning-Based Cooperative Approach for Multi-Intersection Traffic Signal Control*. Engineering Applications of Artificial Intelligence, 114:105019, 2022.

[JLW+24] Jiang, Haoyuan, Ziyue Li, Hua Wei, Xuantang Xiong, Jingqing Ruan, Jiaming Lu, Hangyu Mao y Rui Zhao: *X-Light: Cross-City Traffic Signal Control Using Transformer on Transformer as Meta Multi-Agent Reinforcement Learner*. En *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, páginas 94–102, 2024.

[KCW+22] Kuba, Jakub Grudzien, Ruiqing Chen, Muning Wen, Ying Wen, Fuchun Sun, Jun Wang y Yaodong Yang: *Trust region policy optimisation in multi-agent reinforcement learning*. En *International Conference on Learning Representations (ICLR)*, 2022. `https://arxiv.org/abs/2109.11251`.

[KKBA23] Kolat, Máté, Bálint Kővári, Tamás Bécsi y Szilárd Aradi: *Multi-Agent Reinforcement Learning for Traffic Signal Control: A Cooperative Approach*. Sustainability, 15(4):3479, 2023.

[KL02] Kakade, Sham y John Langford: *Approximately optimal reinforcement learning*. En *International Conference on Machine Learning (ICML)*, páginas 267–274, 2002.

[Kra98] Krauß, Stefan: *Microscopic Modeling of Traffic Flow: Investigation of Collision Free Vehicle Dynamics*. DLR-Forschungsbericht 98-08, Deutsches Zentrum für Luft- und Raumfahrt, 1998. `https://elib.dlr.de/8380/`.

[KT03] Konda, Vijay R. y John N. Tsitsiklis: *On Actor-Critic Algorithms*. SIAM Journal on Control and Optimization, 42(4):1143–1166, 2003.

[LFYZ25] Lu, Qiong, Haoda Fang, Zhangcheng Yin y Guliang Zhu: *HAPS-PPO: A Multi-Agent Reinforcement Learning Architecture for Coordinated Regional Control of Traffic Signals in Heterogeneous Road Networks*. Applied Sciences, 15(20):10945, 2025.

[LJR+20] Li, Liam, Kevin Jamieson, Afshin Rostamizadeh, Ekaterina Gonina, Jonathan Ben-Tzur, Moritz Hardt, Benjamin Recht y Ameet Talwalkar: *A system for massively parallel hyperparameter tuning*. En *Proceedings of Machine Learning and Systems (MLSys)*, volumen 2, páginas 230–246, 2020.

[LLN+18] Liang, Eric, Richard Liaw, Robert Nishihara, Philipp Moritz, Roy Fox, Ken Goldberg, Joseph Gonzalez, Michael Jordan y Ion Stoica: *RLlib: Abstrac-*

tions for distributed reinforcement learning. En *International Conference on Machine Learning (ICML)*, páginas 3053–3062. PMLR, 2018.

[LML+25] Liang, Jiaxin, Haotian Miao, Kai Li, Jianheng Tan, Xi Wang, Rui Luo y Yueqiu Jiang: *A Review of Multi-Agent Reinforcement Learning Algorithms*. Electronics, 14(4):820, 2025. https://doi.org/10.3390/electronics14040820.

[LWT+17] Lowe, Ryan, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel y Igor Mordatch: *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*. En *Advances in Neural Information Processing Systems (NeurIPS)*, páginas 6379–6390, 2017.

[LZF+25] Liu, Wanting, Chengwei Zhang, Wanqing Fang, Kailing Zhou, Yihong Li, Furui Zhan, Qi Wang, Wanli Xue y Rong Chen: *Vehicle-Level Fairness-Oriented Constrained Multi-Agent Reinforcement Learning for Adaptive Traffic Signal Control*. IEEE Transactions on Intelligent Transportation Systems, 26(4):4878–4890, 2025.

[MKS+15] Mnih, V., K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg y D. Hassabis: *Human-level control through deep reinforcement learning*. Nature, 518(7540):529–533, 2015. https://doi.org/10.1038/nature14236.

[MMLK25] Michailidis, Panagiotis, Iakovos Michailidis, Charalampos Rafail Lazaridis y Elias Kosmatopoulos: *Traffic Signal Control via Reinforcement Learning: A Review on Applications and Innovations*. Infrastructures, 10(5):114, 2025. Fuente: [4, 5] Revisión que documenta la evolución de RL como framework para TSC.

[PW16] Pande, Anurag y Brian Wolshon: *Traffic Engineering Handbook*. John Wiley & Sons, Inc., 7th edición, 2016.

[RDZ+24] Ren, Fuyue, Wei Dong, Xiaodong Zhao, Fan Zhang, Yaguang Kong y Qiang Yang: *Two-Layer Coordinated Reinforcement Learning for Traffic Signal Control in Traffic Network*. Expert Systems with Applications, 235:121111, 2024.

[RSDW+18] Rashid, Tabish, Mikayel Samvelyan, Christian Schroeder De Witt, Gregory Farquhar, Jakob Foerster y Shimon Whiteson: *QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning*.

En *Proceedings of the 35th International Conference on Machine Learning (ICML)*, páginas 4295–4304. PMLR, 2018.

[SB18] Sutton, R.S. y A.G. Barto: *Reinforcement learning: An introduction*. The MIT Press, Cambridge, MA, 2nd edición, 2018.

[Sha53] Shapley, Lloyd S: *Stochastic games*. Proceedings of the National Academy of Sciences, 39(10):1095–1100, 1953.

[SLA+15] Schulman, John, Sergey Levine, Pieter Abbeel, Michael Jordan y Philipp Moritz: *Trust region policy optimization*. En *International Conference on Machine Learning (ICML)*, páginas 1889–1897. PMLR, 2015.

[SLG+18] Sunehag, Peter, Guy Lever, Audrunas Gruslys, Wojciech M. Czarnecki, Vinicius Zambaldi, Max Jaderberg, Marc Lanctot, Nicolas Sonnerat, Joel Z. Leibo, Karl Tuyls y Thore Graepel: *Value-Decomposition Networks For Cooperative Multi-Agent Learning*. En *Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS)*, páginas 2085–2087. International Foundation for Autonomous Agents and Multiagent Systems, 2018.

[SML+16] Schulman, John, Philipp Moritz, Sergey Levine, Michael Jordan y Pieter Abbeel: *High-dimensional continuous control using generalized advantage estimation*. En *International Conference on Learning Representations (ICLR)*, 2016.

[SWD+17] Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford y Oleg Klimov: *Proximal policy optimization algorithms*. arXiv preprint arXiv:1707.06347, 2017.

[TMK+17] Tampuu, Ardi, Tambet Matiisen, Dorian Kodelja, Ilya Zauss, Kristjan Korjus, Juhan Aru, Jaan Birk y Raul Vicente: *Multiagent cooperation and competition with deep reinforcement learning*. PloS one, 12(4):e0172395, 2017.

[Tok24] Toko: *Manual de uso del clúster Toko*. `https://toko.uncu.edu.ar/files/Manual_Toko-20240401.pdf`, 2024.

[W+19] Wei, H. *y cols.*: *A survey on traffic signal control methods*, 2019. Available at: `https://arxiv.org/abs/1904.08117`.

[WCZ+19] Wei, Hua, Chacha Chen, Guanjie Zheng, Kan Wu, Vikash Gayah, Kai Xu y Zhenhui Li: *PressLight: Learning Max Pressure Control to Coordinate Traffic Signals in Arterial Network*. En *Proceedings of the 25th ACM SIGKDD*

International Conference on Knowledge Discovery & Data Mining, páginas 1290–1298, 2019.

[Wie00] Wiering, Marco A: *Multi-agent reinforcement learning for traffic light control*. Machine learning, 2000.

[WWS+22] Wu, Qiang, Jianqing Wu, Jun Shen, Bo Du, Akbar Telikani, Mahdi Fahmideh y Chao Liang: *Distributed Agent-Based Deep Reinforcement Learning for Large Scale Traffic Signal Control*. Knowledge-Based Systems, 241:108304, 2022.

[WXZ+19] Wei, Hua, Nan Xu, Huichu Zhang, Guanjie Zheng, Xinshi Zang, Chacha Chen, Weinan Zhang, Yanmin Zhu, Kai Xu y Zhenhui Li: *CoLight: Learning Network-level Cooperation for Traffic Signal Control*. En *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, páginas 1913–1922, 2019.

[WZK+23] Wang, Xihuai, Lingpan Zheng, Jakub Grudzien Kuba, Ruiqing Chen, Li Shen, Yidong Cao, Jun Wang y Yaodong Yang: *Order matters: Agent-by-agent policy optimization*. En *International Conference on Learning Representations (ICLR)*, 2023.

[Yan23] Yang, Shantian: *Hierarchical Graph Multi-Agent Reinforcement Learning for Traffic Signal Control*. Information Sciences, 634:55–72, 2023.

[YVV+22] Yu, Chao, Akash Velu, Eugene Vinitsky, Jiaxuan Gao, Yu Wang, Alexandre Bayen y Yi Wu: *The surprising effectiveness of ppo in cooperative multi-agent games*. Advances in Neural Information Processing Systems, 35:24611–24624, 2022.

[ZKF+24] Zhong, Yifan, Jakub Grudzien Kuba, Xidong Feng, Siyi Hu, Jiaming Ji y Yaodong Yang: *Heterogeneous-Agent Reinforcement Learning*. Journal of Machine Learning Research, 25:1–67, 2024. `https://www.jmlr.org/papers/v25/23-0488.html`.

[ZLFS24] Zhang, Yifeng, Peizhuo Li, Mingfeng Fan y Guillaume Sartoretti: *HeteroLight: A General and Efficient Learning Approach for Heterogeneous Traffic Signal Control*. En *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, páginas 1010–1017, 2024.

[ZLYZ23] Zhou, Mengdi, Xiaoteng Li, Yaodong Yang y Weinan Zhang: *A Survey on Centralized Training for Decentralized Execution in Multi-Agent Reinforcement Learning*. Artificial Intelligence Review, 2023.

[ZWL24] Zhang, Haipeng, Zhiwen Wang y Na Li: *MATLight: Traffic Signal Coordinated Control Algorithm Based on Heterogeneous-Agent Mirror Learning With Transformer*. En *Proceedings of the 23rd International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, páginas 2582–2584, 2024.